

Curso Profesional de Entity Framework Core 8

Módulo 3 — Prácticas: Consultas con LINQ

Autor: JAIME GALLO

12 puntos – 6 horas

El Módulo 3 profundiza en las consultas con LINQ sobre el proyecto AceriaData. Se parte de la arquitectura limpia configurada en el Módulo 2 y se añaden casos de uso que aprovechan las capacidades de LINQ y EF Core: filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga de relaciones y composición de consultas.

Punto 3.1 - Fundamentos de LINQ to Entities

Práctica

Ejercicio: Explorar los fundamentos de LINQ to Entities en el proyecto AceriaData. Crear un caso de uso que ejecute consultas LINQ con diferentes operadores, analizar el SQL generado con ToString y comparar el comportamiento de IEnumerable e IQueryable.

Contexto del proyecto: En el Módulo 2 se configuró la arquitectura limpia con cuatro proyectos y se aplicaron los patrones Repositorio y Unidad de Trabajo. En este punto se exploran los fundamentos de LINQ sobre el proyecto AceriaData. Esta exploración se usará en el punto 3.2 para las consultas básicas con filtros y ordenaciones.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir un caso de uso para consultas LINQ

Crear el archivo src/AceriaData.Application/UseCases/ConsultasLinqUseCase.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Application.UseCases;

public class ConsultasLinqUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasLinqUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");

        DemostrarIEnumerableVsIQueryable();
        DemostrarEjecucionDiferida();
        DemostrarMaterializacion();
        DemostrarToString();
    }

    private void DemostrarIEnumerableVsIQueryable()
    {
        Console.WriteLine("\n-- IEnumerable vs IQueryable --");

        // IEnumerable: carga todo en memoria y filtra en C#
        var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas()
            .Where(o => o.Cliente == "Constructora del Norte")
            .ToList();

        // IQueryable: filtra en SQL
        var ordenesQueryable = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Cliente == "Constructora del Norte")
            .ToList();

        Console.WriteLine($"IEnumerable: {ordenesEnumerable.Count} órdenes");
    }
}
```

```

        Console.WriteLine($"Queryable: {ordenesQueryable.Count} órdenes");
    }

    private void DemostrarEjecucionDiferida()
    {
        Console.WriteLine("\n--- Ejecución diferida ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable();

        Console.WriteLine("Consulta construida, sin ejecutar.");

        var cliente = "Constructora del Norte";
        if (!string.IsNullOrEmpty(cliente))
        {
            consulta = consulta.Where(o => o.Cliente == cliente);
        }

        Console.WriteLine("Filtro añadido, sin ejecutar.");

        var resultados = consulta.ToList();

        Console.WriteLine($"Consulta materializada: {resultados.Count} órdenes");
    }

    private void DemostrarMaterializacion()
    {
        Console.WriteLine("\n--- Materialización ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable();

        var lista = consulta.ToList();
        Console.WriteLine($"ToList: {lista.Count} órdenes");

        var primera = consulta.FirstOrDefault();
        Console.WriteLine($"FirstOrDefault: {primera?.NumeroOrden}");

        var total = consulta.Count();
        Console.WriteLine($"Count: {total} órdenes");

        var existe = consulta.Any();
        Console.WriteLine($"Any: {existe}");
    }

    private void DemostrarToQueryString()
    {
        Console.WriteLine("\n--- ToQueryString ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Cliente == "Constructora del Norte")
            .OrderBy(o => o.FechaCreacion);

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core para ToQueryString.

Línea 5: namespace AceriaData.Application.UseCases; → espacio de nombres de los casos de uso.

Línea 7: public class ConsultasLinqUseCase → declara el caso de uso.

Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 11: public ConsultasLinqUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 13: _unidad = unidad; → asigna el parámetro al campo.

Línea 16: `public void Ejecutar()` → declara el método principal.

Línea 18: `Console.WriteLine("=== FUNDAMENTOS DE LINQ TO ENTITIES ===");` → muestra la cabecera.

Línea 20: `DemostrarIEnumerableVsIQueryable();` → llama al método de comparación.

Línea 21: `DemostrarEjecucionDiferida();` → llama al método de ejecución diferida.

Línea 22: `DemostrarMaterializacion();` → llama al método de materialización.

Línea 23: `DemostrarToQueryString();` → llama al método de análisis del SQL.

Línea 26: `private void DemostrarIEnumerableVsIQueryable()` → declara el método.

Línea 28: `Console.WriteLine("\n--- IEnumerable vs IQueryable ---");` → muestra la cabecera.

Línea 31: `var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas()` → obtiene todas las órdenes como lista.

Línea 32: `.Where(o => o.Cliente == "Constructora del Norte")` → filtra en memoria.

Línea 33: `.ToList();` → materializa la lista.

Línea 36: `var ordenesQueryable = _unidad.Ordenes.ObtenerQueryable()` → obtiene la consulta como IQueryable.

Línea 37: `.Where(o => o.Cliente == "Constructora del Norte")` → filtra en SQL.

Línea 38: `.ToList();` → materializa la consulta.

Línea 40: `Console.WriteLine($"Enumerable: {ordenesEnumerable.Count} órdenes");` → muestra el número.

Línea 41: `Console.WriteLine($"Queryable: {ordenesQueryable.Count} órdenes");` → muestra el número.

Línea 44: `private void DemostrarEjecucionDiferida()` → declara el método.

Línea 46: `Console.WriteLine("\n--- Ejecución diferida ---");` → muestra la cabecera.

Línea 48: `var consulta = _unidad.Ordenes.ObtenerQueryable();` → crea la consulta base.

Línea 50: `Console.WriteLine("Consulta construida, sin ejecutar.");` → mensaje.

Línea 52: `var cliente = "Constructora del Norte";` → declara el cliente.

Línea 53: `if (!string.IsNullOrEmpty(cliente))` → comprueba si el cliente no está vacío.

Línea 55: `consulta = consulta.Where(o => o.Cliente == cliente);` → añade el filtro.

Línea 58: `Console.WriteLine("Filtro añadido, sin ejecutar.");` → mensaje.

Línea 60: `var resultados = consulta.ToList();` → materializa la consulta.

Línea 62: `Console.WriteLine($"Consulta materializada: {resultados.Count} órdenes");` → muestra el número.

Línea 65: `private void DemostrarMaterializacion()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Materialización ---");` → muestra la cabecera.

Línea 69: `var consulta = _unidad.Ordenes.ObtenerQueryable();` → crea la consulta base.

Línea 71: `var lista = consulta.ToList();` → materializa con ToList.

Línea 72: `Console.WriteLine($"ToList: {lista.Count} órdenes");` → muestra el número.

Línea 74: `var primera = consulta.FirstOrDefault();` → obtiene la primera orden.

Línea 75: `Console.WriteLine($"FirstOrDefault: {primera?.NumeroOrden}");` → muestra el número de orden.

Línea 77: `var total = consulta.Count();` → cuenta las órdenes.

Línea 78: `Console.WriteLine($"Count: {total} órdenes");` → muestra el total.

Línea 80: `var existe = consulta.Any();` → comprueba si hay órdenes.

Línea 81: `Console.WriteLine($"Any: {existe}");` → muestra el resultado.

Línea 84: `private void DemostrarToQueryString()` → declara el método.

Línea 86: `Console.WriteLine("\n--- ToQueryString ---");` → muestra la cabecera.

Línea 88: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → crea la consulta.

Línea 89: `.Where(o => o.Cliente == "Constructora del Norte")` → filtra por cliente.

Línea 90: `.OrderBy(o => o.FechaCreacion);` → ordena por fecha.

Línea 92: `var sql = consulta.ToQueryString();` → obtiene el SQL sin ejecutarlo.

Línea 93: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si la interfaz `IOrdenRepositorio` no tiene un método `ObtenerQueryable`, este código no compila. Se debe añadir el método a la interfaz.

■ Paso 3: Añadir el método `ObtenerQueryable` a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 3: `namespace AceriaData.Application.Interfaces;` → espacio de nombres.

Línea 5: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 7: `OrdenFabricacion? ObtenerPorId(int id);` → método que obtiene por Id.

Línea 8: `List ObtenerTodas();` → método que obtiene todas.

Línea 9: `IQueryable ObtenerQueryable();` → método que devuelve `IQueryable` para composición de consultas.

Línea 10: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → método que obtiene por número.

Línea 11: `void Agregar(OrdenFabricacion orden);` → método que agrega.

Línea 12: `void Eliminar(OrdenFabricacion orden);` → método que elimina.

Error común: exponer `IQueryable` en el repositorio es un anti-patrón según el documento de evolución. Sin embargo, para el Módulo 3, que se centra en LINQ, es necesario para demostrar la composición de consultas. En el Módulo 4 se refactorizará para encapsular las consultas en métodos específicos.

Paso 4: Implementar ObtenerQueryable en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```
public IQueryable<OrdenFabricacion> ObtenerQueryable()
{
    return _context.OrdenesFabricacion.AsQueryable();
}
```

Línea 1: public IQueryable ObtenerQueryable() → declara el método.

Línea 3: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta como IQueryable.

Error común: si se devuelve _context.OrdenesFabricacion.ToList().AsQueryable(), se materializa la consulta antes de tiempo y se pierde la ventaja de la traducción a SQL.

Paso 5: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<ConsultasLinqUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasLinqUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 7: Insertar datos de prueba

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes de prueba:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = DateTime.Now }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... } → cuarta orden.

Línea 15: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 16: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa AK_OrdenesFabricacion_NumeroOrden rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa la diferencia entre IEnumerable e IQueryable, la ejecución diferida, la materialización y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== FUNDAMENTOS DE LINQ TO ENTITIES ===
```

```
--- IEnumerable vs IQueryable ---
```

```
Enumerable: 2 órdenes
```

```
Queryable: 2 órdenes
```

```
--- Ejecución diferida ---
```

```
Consulta construida, sin ejecutar.
```

```
Filtro añadido, sin ejecutar.
```

```
Consulta materializada: 2 órdenes
```

```
--- Materialización ---
```

```
ToList: 4 órdenes
```

```
FirstOrDefault: 0F-2024-0001
```

```
Count: 4 órdenes
```

```
Any: True
```

```
--- ToQueryString ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
```

```
FROM [OrdenesFabricacion] AS [o]
```

```
WHERE [o].[Cliente] = N'Constructora del Norte'
```

```
ORDER BY [o].[FechaCreacion]
```

La primera sección muestra que ambas consultas devuelven dos órdenes, pero la primera filtra en memoria y la segunda en SQL. La segunda sección muestra que la consulta no se ejecuta hasta ToList. La tercera sección muestra los diferentes métodos de materialización. La cuarta sección muestra el SQL generado.

Observaciones: el SQL generado por ToQueryString no se ha ejecutado contra la base de datos. Solo se ha construido. Esto demuestra que ToQueryString no materializa la consulta.

Paso 10: Diagnosticar un error común

Modificar el método DemostrarIEnumerableVsIQueryable para usar ToList antes del filtro:

```
csharp
```

```
var ordenesEnumerable = _unidad.Ordenes.ObtenerTodas()
```

```
.ToList()
```

```
.Where(o => o.Cliente == "Constructora del Norte")
```

```
.ToList();
```

Resultado esperado: el resultado es el mismo, pero la consulta SQL ejecutada carga todas las órdenes en memoria y después filtra. Si la tabla tuviera un millón de filas, la consulta sería muy lenta.

Solución: aplicar ToList solo al final, después de todos los filtros.

Errores comunes del ejercicio Error Causa Solución Filtro se aplica en memoria Se llamó a ToList antes del filtro Aplicar ToList al final Expresión no traducida Se usó un método personalizado Reescribir con métodos traducibles Múltiples consultas Se materializó varias veces Materializar una sola vez al final ToQueryString ejecuta la consulta No, solo la construye Verificar que no se llama a ToList IQueryable expuesto en el repositorio Anti-patrón Encapsular en métodos específicos (Módulo 4) Falta el using de EF Core El método ToQueryString no está disponible Añadir using Microsoft.EntityFrameworkCore;

Retos resueltos: Analizar el SQL de una consulta con filtros y ordenaciones

Reto: Crear un método que construya una consulta LINQ con filtros por cliente y estado, ordenación por fecha, y proyección a un tipo anónimo. Obtener el SQL con ToQueryString y analizar su estructura.

Solución paso a paso:

Paso 1: Añadir el método AnalizarSql al caso de uso:

```
private void AnalizarSql()
{
    Console.WriteLine("\n--- Análisis del SQL generado ---");

    var consulta = _unidad Ordenes.ObtenerQueryable()
        .Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente")
        .OrderByDescending(o => o.FechaCreacion)
        .Select(o => new { o.NumeroOrden, o.Cliente, o.Estado, o.FechaCreacion });

    var sql = consulta.ToQueryString();
    Console.WriteLine(sql);
}
```

Línea 1: `private void AnalizarSql()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Análisis del SQL generado ---");` → muestra la cabecera.

Línea 5: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 6: `.Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente")` → filtra por cliente y estado.

Línea 7: `.OrderByDescending(o => o.FechaCreacion)` → ordena por fecha descendente.

Línea 8: `.Select(o => new { o.NumeroOrden, o.Cliente, o.Estado, o.FechaCreacion })` → proyecta a un tipo anónimo.

Línea 10: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 11: `Console.WriteLine(sql);` → imprime el SQL.

Paso 2: Llamar al método desde Ejecutar:

```
AnalizarSql();
Paso 3: Ejecutar dotnet run y analizar el SQL generado.
```

Resultado esperado: el SQL incluye el filtro `WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[Estado] = N'Pendiente'`, la ordenación `ORDER BY [o].[FechaCreacion] DESC` y la proyección de las columnas seleccionadas.

Analogía final

LINQ to Entities en una acería es como el lenguaje que usa el jefe de planta para pedir informes al archivo central. El jefe no va al archivo a buscar los documentos: envía una orden con las condiciones (cliente, estado, fecha) y el archivo devuelve solo los documentos que cumplen esas condiciones. Si el jefe pidiera todos los documentos y luego los filtrara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se usa `IEnumerable` en lugar de `IQueryable`: se traen todos los datos a memoria y se filtran en el cliente. La ejecución diferida es como preparar la orden de búsqueda antes de enviarla: el jefe puede añadir condiciones hasta que esté seguro, y solo entonces la envía. La materialización es el momento en que se envía la orden y se recibe la respuesta. `ToQueryString` es como pedir una copia de la orden sin enviarla: el jefe puede revisar qué se va a pedir antes de ejecutarlo. Así funciona LINQ to Entities: se construye la consulta, se traduce a SQL y se ejecuta en el servidor, aprovechando toda la potencia de la base de datos.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado el caso de uso `ConsultasLinqUseCase`.

Añadido el método `ObtenerQueryable` a la interfaz del repositorio.

Implementado el método en el repositorio.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba.

Ejecutado las demostraciones de IEnumerable vs IQueryable, ejecución diferida, materialización y ToQueryString.

Analizado el SQL generado por una consulta con filtros y ordenaciones.

Diagnosticado el error de aplicar ToList antes del filtro.

Creado el método AnalizarSql.

Punto 3.2 - Consultas básicas: Where, OrderBy y ThenBy

Práctica

Ejercicio: Añadir métodos específicos al repositorio de órdenes para encapsular consultas con filtros y ordenaciones. Crear un caso de uso que ejecute estas consultas, analice el SQL generado y compare los resultados con diferentes combinaciones de filtros y ordenaciones.

Contexto del proyecto: En el punto 3.1 se exploraron los fundamentos de LINQ to Entities. En este punto se añaden consultas básicas con Where, OrderBy, ThenBy, OrderByDescending y ThenByDescending. Estas consultas se encapsulan en métodos del repositorio para evitar exponer IQueryable fuera de la infraestructura. Esta encapsulación se usará en el punto 3.3 para las proyecciones con Select.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de consulta a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 3: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 5: public interface IOrdenRepositorio → declara la interfaz.

Línea 7: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 8: List ObtenerTodas(); → método que obtiene todas.

Línea 9: IQueryable ObtenerQueryable(); → método que devuelve IQueryable para el punto 3.1.

Línea 10: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 11: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las órdenes pendientes de un cliente.

Línea 12: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene las órdenes por estado ordenadas por fecha.

Línea 13: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene las órdenes por rango de fechas.

Línea 14: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene las órdenes de un cliente ordenadas.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o entidades concretas.

| Paso 3: Implementar los métodos en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories.OrdenRepositorio.cs`:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.Find(id);
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }

    public IQueryable<OrdenFabricacion> ObtenerQueryable()
    {
        return _context.OrdenesFabricacion.AsQueryable();
    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }
}
```

```

public List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)
{
    return _context.OrdenesFabricacion
        .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente)
        .OrderBy(o => o.Cliente)
        .ThenByDescending(o => o.FechaCreacion)
        .ToList();
}

public void Agregar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Add(orden);
}

public void Eliminar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Remove(orden);
}
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz del repositorio.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 3: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorio : IOrdenRepositorio → declara la implementación.

Línea 9: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 11: public OrdenRepositorio(AceriaDbContext context) → constructor.

Línea 13: _context = context; → asigna el parámetro al campo.

Línea 16: public OrdenFabricacion? ObtenerPorId(int id) → método que obtiene por Id.

Línea 18: return _context.OrdenesFabricacion.Find(id); → busca por clave primaria.

Línea 21: public List ObtenerTodas() → método que obtiene todas.

Línea 23: return _context.OrdenesFabricacion.ToList(); → materializa la consulta.

Línea 26: public IQueryable ObtenerQueryable() → método que devuelve IQueryable.

Línea 28: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta sin materializar.

Línea 31: public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden) → método que obtiene por número.

Línea 33: return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → filtra por número y devuelve la primera coincidencia.

Línea 36: public List ObtenerPendientesPorCliente(string cliente) → método que obtiene las órdenes pendientes de un cliente.

Línea 38: return _context.OrdenesFabricacion → inicia la consulta.

Línea 39: .Where(o => o.Cliente == cliente && o.Estado == "Pendiente") → filtra por cliente y estado.

Línea 40: .OrderBy(o => o.FechaCreacion) → ordena por fecha ascendente.

Línea 41: `.ToList();` → materializa la consulta.

Línea 44: `public List ObtenerPorEstadoOrdenadasPorFecha(string estado)` → método que obtiene las órdenes por estado ordenadas por fecha.

Línea 46: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 47: `.Where(o => o.Estado == estado)` → filtra por estado.

Línea 48: `.OrderByDescending(o => o.FechaCreacion)` → ordena por fecha descendente.

Línea 49: `.ToList();` → materializa la consulta.

Línea 52: `public List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)` → método que obtiene las órdenes por rango de fechas.

Línea 54: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 55: `.Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)` → filtra por rango de fechas.

Línea 56: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha ascendente.

Línea 57: `.ToList();` → materializa la consulta.

Línea 60: `public List ObtenerPorClienteOrdenadas(string cliente)` → método que obtiene las órdenes de un cliente ordenadas.

Línea 62: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 63: `.Where(o => o.Cliente == cliente)` → filtra por cliente.

Línea 64: `.OrderBy(o => o.Cliente)` → ordena por cliente ascendente.

Línea 65: `.ThenByDescending(o => o.FechaCreacion)` → añade un criterio secundario por fecha descendente.

Línea 66: `.ToList();` → materializa la consulta.

Línea 69: `public void Agregar(OrdenFabricacion orden)` → método que agrega una orden.

Línea 71: `_context.OrdenesFabricacion.Add(orden);` → registra la entidad.

Línea 74: `public void Eliminar(OrdenFabricacion orden)` → método que elimina una orden.

Línea 76: `_context.OrdenesFabricacion.Remove(orden);` → marca la entidad para eliminar.

Error común: si se llama a `ToList` antes de aplicar los filtros y las ordenaciones, se carga toda la tabla en memoria y se pierde la ventaja de la traducción a SQL.

| Paso 4: Crear el caso de uso de consultas básicas

Crear el archivo `src/AceriaData.Application/UseCases/ConsultasBasicasUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ConsultasBasicasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS BÁSICAS CON LINQ ===");
    }
}
```

```

        DemostrarWhereSimple();
        DemostrarWhereCompuesto();
        DemostrarOrdenacionSimple();
        DemostrarOrdenacionMultiple();
        DemostrarCombinacion();
    }

    private void DemostrarWhereSimple()
    {
        Console.WriteLine("\n--- Where simple: órdenes pendientes del cliente 'Constructora del Norte'
        ---");

        var ordenes = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} |
            {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarWhereCompuesto()
    {
        Console.WriteLine("\n--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha
        descendente ---");

        var ordenes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} |
            {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarOrdenacionSimple()
    {
        Console.WriteLine("\n--- Ordenación simple: órdenes por rango de fechas ---");

        var desde = new DateTime(2024, 1, 1);
        var hasta = new DateTime(2025, 12, 31);

        var ordenes = _unidad.Ordenes.ObtenerPorRangoDeFechas(desde, hasta);
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/
            yyyy}");
        }
    }

    private void DemostrarOrdenacionMultiple()
    {
        Console.WriteLine("\n--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte'
        ---");

        var ordenes = _unidad.Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/
            yyyy}");
        }
    }

    private void DemostrarCombinacion()
    {
        Console.WriteLine("\n--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy
        ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.Cliente)
            .ThenByDescending(o => o.FechaCreacion);

        var sql = consulta.ToQueryString();
    }

```

```

        Console.WriteLine(sql);
    }
}

```

- Línea 1: `using AceriaData.Application.Interfaces;` → importa las interfaces de la aplicación.
- Línea 3: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres.
- Línea 5: `public class ConsultasBasicasUseCase` → declara el caso de uso.
- Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → campo de la unidad de trabajo.
- Línea 9: `public ConsultasBasicasUseCase(IUnidadDeTrabajo unidad)` → constructor.
- Línea 11: `_unidad = unidad;` → asigna el parámetro al campo.
- Línea 14: `public void Ejecutar()` → declara el método principal.
- Línea 16: `Console.WriteLine("=== CONSULTAS BÁSICAS CON LINQ ===");` → muestra la cabecera.
- Línea 18: `DemostrarWhereSimple();` → llama al método de Where simple.
- Línea 19: `DemostrarWhereCompuesto();` → llama al método de Where compuesto.
- Línea 20: `DemostrarOrdenacionSimple();` → llama al método de ordenación simple.
- Línea 21: `DemostrarOrdenacionMultiple();` → llama al método de ordenación múltiple.
- Línea 22: `DemostrarCombinacion();` → llama al método de combinación.
- Línea 25: `private void DemostrarWhereSimple()` → declara el método.
- Línea 27: `Console.WriteLine("\n--- Where simple: órdenes pendientes del cliente 'Constructora del Norte' ---");` → muestra la cabecera.
- Línea 29: `var ordenes = _unidad.Ordenes.ObtenerPendientesPorCliente("Constructora del Norte");` → llama al método del repositorio.
- Línea 30: `foreach (var orden in ordenes)` → itera sobre las órdenes.
- Línea 32: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.
- Línea 35: `private void DemostrarWhereCompuesto()` → declara el método.
- Línea 37: `Console.WriteLine("\n--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha descendente ---");` → muestra la cabecera.
- Línea 39: `var ordenes = _unidad.Ordenes.ObtenerPorEstadoOrdenadasPorFecha("Pendiente");` → llama al método del repositorio.
- Línea 40: `foreach (var orden in ordenes)` → itera sobre las órdenes.
- Línea 42: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.
- Línea 45: `private void DemostrarOrdenacionSimple()` → declara el método.
- Línea 47: `Console.WriteLine("\n--- Ordenación simple: órdenes por rango de fechas ---");` → muestra la cabecera.
- Línea 49: `var desde = new DateTime(2024, 1, 1);` → declara la fecha desde.
- Línea 50: `var hasta = new DateTime(2025, 12, 31);` → declara la fecha hasta.
- Línea 52: `var ordenes = _unidad.Ordenes.ObtenerPorRangoDeFechas(desde, hasta);` → llama al método del repositorio.
- Línea 53: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 55: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 58: `private void DemostrarOrdenacionMultiple()` → declara el método.

Línea 60: `Console.WriteLine("\n--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte' ---");` → muestra la cabecera.

Línea 62: `var ordenes = _unidad Ordenes.ObtenerPorClienteOrdenadas("Constructora del Norte");` → llama al método del repositorio.

Línea 63: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 65: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 68: `private void DemostrarCombinacion()` → declara el método.

Línea 70: `Console.WriteLine("\n--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy ---");` → muestra la cabecera.

Línea 72: `var consulta = _unidad Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 73: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 74: `.OrderBy(o => o.Cliente)` → ordena por cliente ascendente.

Línea 75: `.ThenByDescending(o => o.FechaCreacion);` → añade un criterio secundario por fecha descendente.

Línea 77: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 78: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ConsultasBasicasUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasBasicasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 7: Insertar datos de prueba

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes de prueba:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... } → quinta orden.

Línea 16: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 17: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa AK_OrdenesFabricacion_NumeroOrden rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan los filtros por cliente y estado, las ordenaciones ascendentes y descendentes, y el SQL generado por la combinación de Where, OrderBy y ThenBy.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

=== CONSULTAS BÁSICAS CON LINQ ===

--- Where simple: órdenes pendientes del cliente 'Constructora del Norte' ---

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0005 | Constructora del Norte | Pendiente | 12/05/2024
```

--- Where compuesto: órdenes en estado 'Pendiente' ordenadas por fecha descendente ---

```
OF-2024-0005 | Constructora del Norte | Pendiente | 12/05/2024
OF-2024-0004 | Constructora del Este | Pendiente | 05/04/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
```

--- Ordenación simple: órdenes por rango de fechas ---

```
OF-2024-0001 | Constructora del Norte | 15/01/2024
OF-2024-0002 | Constructora del Sur | 20/02/2024
OF-2024-0003 | Constructora del Norte | 10/03/2024
OF-2024-0004 | Constructora del Este | 05/04/2024
OF-2024-0005 | Constructora del Norte | 12/05/2024
```

--- Ordenación múltiple: órdenes del cliente 'Constructora del Norte' ---

```
OF-2024-0005 | Constructora del Norte | 12/05/2024
OF-2024-0003 | Constructora del Norte | 10/03/2024
OF-2024-0001 | Constructora del Norte | 15/01/2024
```

--- Combinación: SQL generado por una consulta con Where, OrderBy y ThenBy ---

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[Cliente], [o].[FechaCreacion] DESC
```

La primera sección muestra las órdenes pendientes del cliente "Constructora del Norte". La segunda sección muestra las órdenes pendientes ordenadas por fecha descendente. La tercera sección muestra las órdenes ordenadas por fecha ascendente. La cuarta sección muestra las órdenes del cliente "Constructora del Norte" ordenadas por cliente y fecha descendente. La quinta sección muestra el SQL generado.

Observaciones: el SQL generado incluye la cláusula WHERE con el filtro por estado y la cláusula ORDER BY con dos columnas. El orden de las cláusulas es el correcto: SELECT, FROM, WHERE, ORDER BY.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerPendientesPorCliente para usar ToList antes del filtro:

```
csharp
public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
{
    return _context.OrdenesFabricacion
        .ToList()
        .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}
```

Resultado esperado: el resultado es el mismo, pero la consulta SQL ejecutada carga todas las órdenes en memoria y después filtra. Si la tabla tuviera un millón de filas, la consulta sería muy lenta.

Solución: aplicar ToList solo al final, después de todos los filtros y ordenaciones.

Errores comunes del ejercicio Error Causa Solución Filtro se aplica en memoria Se llamó a ToList antes del filtro Aplicar ToList al final Ordenación no aplicada Se llamó a OrderBy después de ToList Aplicar OrderBy antes de ToList Múltiples consultas Se materializó varias veces Materializar una sola vez al final IQueryable expuesto en el repositorio Anti-patrón Encapsular en métodos específicos (Módulo 4) Filtro por fecha incorrecto Se usó == en lugar de >= y <= Usar operadores de comparación ThenBy sin OrderBy Se llamó a ThenBy sin OrderBy previo Llamar a OrderBy primero Falta el using de EF Core El método ToQueryString no está disponible Añadir using Microsoft.EntityFrameworkCore;

Reto resuelto: Consulta con filtro por rango de fechas y ordenación múltiple

Reto: Crear un método en el repositorio que obtenga las órdenes de un cliente en un rango de fechas, ordenadas por estado ascendente y fecha de creación descendente. Añadir el método a la interfaz, implementarlo y usarlo desde el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);
```

Línea 1: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
        .OrderBy(o => o.Estado)
        .ThenByDescending(o => o.FechaCreacion)
        .ToList();
}
```

Línea 1: public List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta) → filtra por cliente y rango de fechas.

Línea 5: .OrderBy(o => o.Estado) → ordena por estado ascendente.

Línea 6: .ThenByDescending(o => o.FechaCreacion) → añade un criterio secundario por fecha descendente.

Línea 7: .ToList(); → materializa la consulta.

Paso 3: Usar el método desde el caso de uso:

```
private void DemostrarConsultaCombinada()
{
    Console.WriteLine("\n--- Consulta combinada: cliente y rango de fechas ---");

    var desde = new DateTime(2024, 1, 1);
    var hasta = new DateTime(2024, 12, 31);

    var ordenes = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", desde, hasta);
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: private void DemostrarConsultaCombinada() → declara el método.

Línea 3: Console.WriteLine("\n--- Consulta combinada: cliente y rango de fechas ---"); → muestra la cabecera.

Línea 5: `var desde = new DateTime(2024, 1, 1);` → declara la fecha desde.

Línea 6: `var hasta = new DateTime(2024, 12, 31);` → declara la fecha hasta.

Línea 8: `var ordenes = _unidad.Ordenes.ObtenerPorClienteYRangoDeFechas("Constructora del Norte", desde, hasta);` → llama al método del repositorio.

Línea 9: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 11: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarConsultaCombinada();
Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran ordenadas por estado y fecha descendente.
```

Resultado esperado: las órdenes del cliente "Constructora del Norte" en el rango de fechas se muestran ordenadas por estado ascendente y, dentro de cada estado, por fecha descendente.

| Analogía final

Las consultas básicas con LINQ en una acería son como las órdenes de búsqueda que el jefe de planta envía al archivo central. Where es el filtro: "solo quiero las órdenes del cliente Constructora del Norte". OrderBy es el criterio de ordenación: "ordénalas por fecha de creación". ThenBy es el criterio secundario: "y dentro de cada fecha, ordénalas por estado". El archivo central recibe la orden, busca en sus índices y devuelve exactamente lo que se pidió. Si el jefe pidiera todos los documentos y los ordenara él mismo, perdería tiempo y esfuerzo. Eso es lo que pasa cuando se materializa antes de filtrar. Las consultas encapsuladas en el repositorio son como las ventanillas especializadas del archivo: cada una sabe qué buscar y cómo ordenarlo. El jefe no necesita conocer el archivo por dentro: solo pide lo que necesita a la ventanilla correspondiente. Así funcionan las consultas básicas con LINQ: se filtran y se ordenan en el servidor, y los resultados se devuelven ya preparados.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de consulta a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ConsultasBasicasUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba.

Ejecutado las demostraciones de `Where`, `OrderBy`, `ThenBy`, `OrderByDescending` y `ThenByDescending`.

Analizado el SQL generado por una consulta combinada.

Diagnosticado el error de aplicar `ToList` antes del filtro.

Creado el método `ObtenerPorClienteYRangoDeFechas` y su demostración.

| Conexión con el siguiente punto

En este punto se han añadido consultas básicas con `Where`, `OrderBy`, `ThenBy`, `OrderByDescending` y `ThenByDescending` al proyecto `AceriaData`. Las consultas se han encapsulado en métodos del repositorio para evitar exponer `IQueryable` fuera de la infraestructura. En el siguiente punto se estudiarán las proyecciones con `Select`, incluyendo tipos anónimos, DTOs y proyecciones parciales.

Punto 3.3 - Proyecciones con Select y tipos anónimos

Práctica

Ejercicio: Añadir proyecciones al repositorio de órdenes del proyecto AceriaData. Crear métodos que devuelvan tipos anónimos y DTOs. Crear un caso de uso que ejecute estas proyecciones, analice el SQL generado y compare el rendimiento de proyectar versus cargar entidades completas.

Contexto del proyecto: En el punto 3.2 se añadieron consultas básicas con Where, OrderBy y ThenBy. En este punto se añaden proyecciones con Select, incluyendo tipos anónimos y DTOs. Estas proyecciones se usarán en el punto 3.4 para las proyecciones a DTOs y en el punto 3.5 para las consultas de agregación.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO OrdenResumenDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenResumenDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenResumenDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres de los DTOs.

Línea 3: public class OrdenResumenDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → propiedad que almacena el estado.

Línea 8: public DateTime FechaCreacion { get; set; } → propiedad que almacena la fecha de creación.

Error común: si el DTO tiene propiedades que no se proyectan, EF Core no las incluye en el SELECT. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO OrdenConTotalesDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConTotalesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConTotalesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres de los DTOs.

Línea 3: public class OrdenConTotalesDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public int TotalPlanchas { get; set; } → propiedad que almacena el total de planchas.

Línea 8: public decimal PesoTotal { get; set; } → propiedad que almacena el peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

Paso 4: Añadir los métodos de proyección a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: `IQueryable ObtenerQueryable();` → método que devuelve `IQueryable`.

Línea 11: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → método que obtiene por número.

Línea 12: `List ObtenerPendientesPorCliente(string cliente);` → método que obtiene las pendientes de un cliente.

Línea 13: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene por estado ordenadas.

Línea 14: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene por rango de fechas.

Línea 15: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene por cliente ordenadas.

Línea 16: `List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);` → método que obtiene por cliente y rango de fechas.

Línea 18: `List ObtenerClientesUnicos();` → método que obtiene los clientes únicos.

Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes de todas las órdenes.

Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado.

Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales de planchas y peso.

Línea 23: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 24: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 5: Implementar los métodos de proyección en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.Find(id);
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }

    public IQueryable<OrdenFabricacion> ObtenerQueryable()
    {
        return _context.OrdenesFabricacion.AsQueryable();
    }
}
```



```

    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
            .OrderBy(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente)
            .OrderBy(o => o.Cliente)
            .ThenByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde,
DateTime hasta)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
            .OrderBy(o => o.Estado)
            .ThenByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<string> ObtenerClientesUnicos()
    {
        return _context.OrdenesFabricacion
            .Select(o => o.Cliente)
            .Distinct()
            .OrderBy(c => c)
            .ToList();
    }

    public List<OrdenResumenDto> ObtenerResumenes()
    {
        return _context.OrdenesFabricacion
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList();
    }
}

```

```

public List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

public List<OrdenConTotalesDto> ObtenerOrdenesConTotales()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConTotalesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            TotalPlanchas = o.Planchas.Count(),
            PesoTotal = o.Planchas.Sum(p => p.Peso)
        })
        .ToList();
}

public void Agregar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Add(orden);
}

public void Eliminar(OrdenFabricacion orden)
{
    _context.OrdenesFabricacion.Remove(orden);
}
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 6: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 8: public class OrdenRepositorio : IOrdenRepositorio → declara la implementación.

Línea 10: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 12: public OrdenRepositorio(AceriaDbContext context) → constructor.

Línea 14: _context = context; → asigna el parámetro al campo.

Línea 17: public OrdenFabricacion? ObtenerPorId(int id) → método que obtiene por Id.

Línea 19: return _context.OrdenesFabricacion.Find(id); → busca por clave primaria.

Línea 22: public List ObtenerTodas() → método que obtiene todas.

Línea 24: return _context.OrdenesFabricacion.ToList(); → materializa la consulta.

Línea 27: public IQueryable ObtenerQueryable() → método que devuelve IQueryable.

Línea 29: return _context.OrdenesFabricacion.AsQueryable(); → devuelve la consulta sin materializar.

Línea 32: public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden) → método que obtiene por número.

Línea 34: `return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → filtra por número.

Línea 37: `public List ObtenerPendientesPorCliente(string cliente)` → método que obtiene las pendientes de un cliente.

Línea 39: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 40: `.Where(o => o.Cliente == cliente && o.Estado == "Pendiente")` → filtra por cliente y estado.

Línea 41: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 42: `.ToList();` → materializa la consulta.

Línea 45: `public List ObtenerPorEstadoOrdenadasPorFecha(string estado)` → método que obtiene por estado.

Línea 47: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 48: `.Where(o => o.Estado == estado)` → filtra por estado.

Línea 49: `.OrderByDescending(o => o.FechaCreacion)` → ordena por fecha descendente.

Línea 50: `.ToList();` → materializa la consulta.

Línea 53: `public List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)` → método que obtiene por rango de fechas.

Línea 55: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 56: `.Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)` → filtra por rango.

Línea 57: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 58: `.ToList();` → materializa la consulta.

Línea 61: `public List ObtenerPorClienteOrdenadas(string cliente)` → método que obtiene por cliente.

Línea 63: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 64: `.Where(o => o.Cliente == cliente)` → filtra por cliente.

Línea 65: `.OrderBy(o => o.Cliente)` → ordena por cliente.

Línea 66: `.ThenByDescending(o => o.FechaCreacion)` → añade criterio secundario.

Línea 67: `.ToList();` → materializa la consulta.

Línea 70: `public List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta)` → método que combina cliente y rango.

Línea 72: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 73: `.Where(o => o.Cliente == cliente && o.FechaCreacion >= desde && o.FechaCreacion <= hasta)` → filtra por cliente y rango.

Línea 74: `.OrderBy(o => o.Estado)` → ordena por estado.

Línea 75: `.ThenByDescending(o => o.FechaCreacion)` → añade criterio secundario.

Línea 76: `.ToList();` → materializa la consulta.

Línea 79: `public List ObtenerClientesUnicos()` → método que obtiene los clientes únicos.

Línea 81: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 82: `.Select(o => o.Cliente)` → proyecta a la propiedad Cliente.

Línea 83: `.Distinct()` → elimina duplicados.

Línea 84: `.OrderBy(c => c)` → ordena alfabéticamente.

Línea 85: .ToList(); → materializa la consulta.

Línea 88: public List ObtenerResumenes() → método que obtiene los resúmenes.

Línea 90: return _context.OrdenesFabricacion → inicia la consulta.

Línea 91: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 92: .Select(o => new OrdenResumenDto → proyecta a OrdenResumenDto.

Línea 93: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 94: Cliente = o.Cliente, → asigna el cliente.

Línea 95: Estado = o.Estado, → asigna el estado.

Línea 96: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 97: }) → cierra la proyección.

Línea 98: .ToList(); → materializa la consulta.

Línea 101: public List ObtenerResumenesPorEstado(string estado) → método que obtiene los resúmenes por estado.

Línea 103: return _context.OrdenesFabricacion → inicia la consulta.

Línea 104: .Where(o => o.Estado == estado) → filtra por estado.

Línea 105: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 106: .Select(o => new OrdenResumenDto → proyecta a OrdenResumenDto.

Línea 107: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 108: Cliente = o.Cliente, → asigna el cliente.

Línea 109: Estado = o.Estado, → asigna el estado.

Línea 110: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 111: }) → cierra la proyección.

Línea 112: .ToList(); → materializa la consulta.

Línea 115: public List ObtenerOrdenesConTotales() → método que obtiene las órdenes con totales.

Línea 117: return _context.OrdenesFabricacion → inicia la consulta.

Línea 118: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 119: .Select(o => new OrdenConTotalesDto → proyecta a OrdenConTotalesDto.

Línea 120: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 121: Cliente = o.Cliente, → asigna el cliente.

Línea 122: TotalPlanchas = o.Planchas.Count(), → calcula el total de planchas.

Línea 123: PesoTotal = o.Planchas.Sum(p => p.Peso) → calcula el peso total.

Línea 124: }) → cierra la proyección.

Línea 125: .ToList(); → materializa la consulta.

Línea 128: public void Agregar(OrdenFabricacion orden) → método que agrega una orden.

Línea 130: _context.OrdenesFabricacion.Add(orden); → registra la entidad.

Línea 133: public void Eliminar(OrdenFabricacion orden) → método que elimina una orden.

Línea 135: _context.OrdenesFabricacion.Remove(orden); → marca la entidad para eliminar.

Error común: si se proyecta a un DTO con una propiedad que no existe en la entidad, el compilador lanza un error. Se deben proyectar solo propiedades que existen.

| Paso 6: Crear el caso de uso de proyecciones

Crear el archivo src/AceriaData.Application/UseCases/ProyeccionesUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProyeccionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProyeccionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES CON SELECT ===");

        DemostrarClientesUnicos();
        DemostrarResumenes();
        DemostrarResumenesPorEstado();
        DemostrarOrdenesConTotales();
        DemostrarSqlProyeccion();
    }

    private void DemostrarClientesUnicos()
    {
        Console.WriteLine("\n--- Clientes únicos ---");

        var clientes = _unidad.Ordenes.ObtenerClientesUnicos();
        foreach (var cliente in clientes)
        {
            Console.WriteLine($" {cliente}");
        }
    }

    private void DemostrarResumenes()
    {
        Console.WriteLine("\n--- Resúmenes de órdenes ---");

        var resumenes = _unidad.Ordenes.ObtenerResumenes();
        foreach (var resumen in resumenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarResumenesPorEstado()
    {
        Console.WriteLine("\n--- Resúmenes de órdenes pendientes ---");

        var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstado("Pendiente");
        foreach (var resumen in resumenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarOrdenesConTotales()
    {
        Console.WriteLine("\n--- Órdenes con totales de planchas y peso ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotales();
        foreach (var orden in ordenes)
    }
}
```

```

        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
{orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg");
        }
    }

    private void DemostrarSqlProyeccion()
    {
        Console.WriteLine("\n--- SQL generado por una proyección ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .Select(o => new
            {
                o.NumeroOrden,
                o.Cliente,
                o.Estado
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class ProyeccionesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public ProyeccionesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== PROYECCIONES CON SELECT ==="); → muestra la cabecera.

Línea 18: DemostrarClientesUnicos(); → llama al método de clientes únicos.

Línea 19: DemostrarResumenes(); → llama al método de resúmenes.

Línea 20: DemostrarResumenesPorEstado(); → llama al método de resúmenes por estado.

Línea 21: DemostrarOrdenesConTotales(); → llama al método de órdenes con totales.

Línea 22: DemostrarSqlProyeccion(); → llama al método de SQL de proyección.

Línea 25: private void DemostrarClientesUnicos() → declara el método.

Línea 27: Console.WriteLine("\n--- Clientes únicos ---"); → muestra la cabecera.

Línea 29: var clientes = _unidad.Ordenes.ObtenerClientesUnicos(); → llama al método del repositorio.

Línea 30: foreach (var cliente in clientes) → itera sobre los clientes.

Línea 32: Console.WriteLine(\$" {cliente}"); → muestra el cliente.

Línea 35: private void DemostrarResumenes() → declara el método.

Línea 37: Console.WriteLine("\n--- Resúmenes de órdenes ---"); → muestra la cabecera.

Línea 39: var resumenes = _unidad.Ordenes.ObtenerResumenes(); → llama al método del repositorio.

Línea 40: foreach (var resumen in resumenes) → itera sobre los resúmenes.

Línea 42: Console.WriteLine(\$" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 45: `private void DemostrarResumenesPorEstado()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Resúmenes de órdenes pendientes ---");` → muestra la cabecera.

Línea 49: `var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstado("Pendiente");` → llama al método del repositorio.

Línea 50: `foreach (var resumen in resumenes)` → itera sobre los resúmenes.

Línea 52: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 55: `private void DemostrarOrdenesConTotales()` → declara el método.

Línea 57: `Console.WriteLine("\n--- Órdenes con totales de planchas y peso ---");` → muestra la cabecera.

Línea 59: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotales();` → llama al método del repositorio.

Línea 60: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 62: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg");` → muestra los datos.

Línea 65: `private void DemostrarSqlProyeccion()` → declara el método.

Línea 67: `Console.WriteLine("\n--- SQL generado por una proyección ---");` → muestra la cabecera.

Línea 69: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 70: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 71: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 72: `.Select(o => new` → proyecta a un tipo anónimo.

Línea 73: `o.NumeroOrden,` → incluye el número de orden.

Línea 74: `o.Cliente,` → incluye el cliente.

Línea 75: `o.Estado` → incluye el estado.

Línea 76: `});` → cierra la proyección.

Línea 78: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 79: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProyeccionesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas de prueba:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };

    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 11: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 12: `context.SaveChanges();` → inserta las órdenes.

Línea 14: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 15: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 16: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 18: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 19: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada proyección. Se observan los clientes únicos, los resúmenes de órdenes, los resúmenes por estado, las órdenes con totales de planchas y peso, y el SQL generado por la proyección.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROYECCIONES CON SELECT ===
```

```
--- Clientes únicos ---
```

```
Constructora del Norte
Constructora del Sur
```

```
--- Resúmenes de órdenes ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
OF-2024-0003 | Constructora del Norte | EnProceso | 10/03/2024
```

```
--- Resúmenes de órdenes pendientes ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024
```

```
--- Órdenes con totales de planchas y peso ---
```

```
OF-2024-0001 | Constructora del Norte | Planchas: 2 | Peso total: 651.3 kg
OF-2024-0002 | Constructora del Sur | Planchas: 1 | Peso total: 125.6 kg
OF-2024-0003 | Constructora del Norte | Planchas: 0 | Peso total: 0 kg
```

```
--- SQL generado por una proyección ---
```

```
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[FechaCreacion]
```

La primera sección muestra los clientes únicos. La segunda sección muestra los resúmenes de todas las órdenes. La tercera sección muestra los resúmenes de las órdenes pendientes. La cuarta sección muestra las órdenes con totales de planchas y peso. La quinta sección muestra el SQL generado por la proyección.

Observaciones: el SQL generado solo incluye las columnas proyectadas, no todas las columnas de la entidad. Esto reduce el volumen de datos transferidos.

Paso 12: Diagnosticar un error común

Modificar el método `ObtenerResumenes` para proyectar solo algunas propiedades y acceder a una propiedad no proyectada:

```
csharp
public List<OrdenResumenDto> ObtenerResumenes()
{
    return _context.OrdenesFabricacion
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente
        })
        .ToList();
}
```

```

    })
    .ToList();
}

```

Resultado esperado: el compilador lanza un error porque las propiedades Estado y FechaCreacion no se han asignado en la proyección. El DTO requiere que todas sus propiedades se asignen.

Solución: proyectar todas las propiedades del DTO o marcar las no proyectadas como anulables.

Errores comunes del ejercicio Error Causa Solución Propiedad no proyectada Se accede a una propiedad que no está en la proyección Proyectar todas las propiedades necesarias Tipo anónimo fuera de ámbito Se intenta usar un tipo anónimo fuera del método Proyectar a un DTO con nombre Función no traducida Se usa una función que EF Core no puede traducir Usar funciones traducibles Proyección antes del filtro Se proyecta antes de filtrar Filtrar y ordenar antes de proyectar Colección de navegación sin ToList Se proyecta una colección sin materializarla Añadir ToList dentro de la proyección Distinct no aplicado Se olvidó el operador Distinct Añadir .Distinct() antes de ToList

Retos resueltos: Proyección de órdenes con cliente y número de planchas

Reto: Crear un método en el repositorio que proyecte las órdenes a un DTO con el número de orden, el cliente, el estado y el número de planchas. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO OrdenConNumeroPlanchasDto:

```

namespace AceriaData.Application.Dtos;

public class OrdenConNumeroPlanchasDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public int NumeroPlanchas { get; set; }
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConNumeroPlanchasDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → propiedad que almacena el número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → propiedad que almacena el cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → propiedad que almacena el estado.

Línea 8: public int NumeroPlanchas { get; set; } → propiedad que almacena el número de planchas.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```

List<OrdenConNumeroPlanchasDto> ObtenerOrdenesConNumeroPlanchas();

```

Línea 1: List ObtenerOrdenesConNumeroPlanchas(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<OrdenConNumeroPlanchasDto> ObtenerOrdenesConNumeroPlanchas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConNumeroPlanchasDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            NumeroPlanchas = o.Planchas.Count()
        })
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesConNumeroPlanchas() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 5: .Select(o => new OrdenConNumeroPlanchasDto → proyecta al DTO.

Línea 6: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 7: Cliente = o.Cliente, → asigna el cliente.

Línea 8: Estado = o.Estado, → asigna el estado.

Línea 9: NumeroPlanchas = o.Planchas.Count() → calcula el número de planchas.

Línea 10: }) → cierra la proyección.

Línea 11: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarOrdenesConNumeroPlanchas()
{
    Console.WriteLine("\n--- Órdenes con número de planchas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConNumeroPlanchas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas:
{orden.NumeroPlanchas}");
    }
}
```

Línea 1: private void DemostrarOrdenesConNumeroPlanchas() → declara el método.

Línea 3: Console.WriteLine("\n--- Órdenes con número de planchas ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerOrdenesConNumeroPlanchas(); → llama al método del repositorio.

Línea 6: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 8: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.NumeroPlanchas}"); → muestra los datos.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarOrdenesConNumeroPlanchas();
Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con su número de planchas.
```

Resultado esperado: las órdenes se muestran con su número de orden, cliente, estado y número de planchas.

| Analogía final

Las proyecciones con Select en una acería son como los resúmenes que el jefe de planta pide al archivo central. En lugar de recibir la carpeta completa de cada orden con todos sus documentos, el jefe pide solo los datos que necesita: el número de orden, el cliente y el estado. El archivo central prepara un resumen con esos datos y lo entrega. El jefe no necesita los documentos completos para tomar decisiones: solo los datos clave. Proyectar a un tipo anónimo es como pedir un resumen informal: se usa en el momento y no se archiva. Proyectar a un DTO es como pedir un formulario estandarizado: tiene nombre, se puede archivar y se puede usar en otros departamentos. Proyectar con Distinct es como pedir la lista de clientes únicos: sin duplicados. Proyectar con funciones de agregación es como pedir el total de planchas y el peso total: valores calculados en el archivo central. Así funcionan las proyecciones en EF Core: se seleccionan solo los datos necesarios, se reduce el volumen de información y se mejora el rendimiento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `OrdenResumenDto` y `OrdenConTotalesDto`.

Añadido los métodos de proyección a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ProyeccionesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas.

Ejecutado las demostraciones de clientes únicos, resúmenes, resúmenes por estado y órdenes con totales.

Analizado el SQL generado por una proyección.

Diagnosticado el error de acceder a una propiedad no proyectada.

Creado el DTO `OrdenConNumeroPlanchasDto` y su método.

| Conexión con el siguiente punto

En este punto se han añadido proyecciones con Select al proyecto `AceriaData`, incluyendo tipos anónimos, DTOs con nombre, proyecciones de propiedades individuales, Distinct y funciones de agregación. Se ha analizado el SQL generado por las proyecciones y se ha comparado el rendimiento de proyectar versus cargar entidades completas. En el siguiente punto se estudiarán las proyecciones a DTOs en detalle, incluyendo el uso de constructores, la proyección de colecciones de navegación y la proyección con `SelectMany`.

Punto 3.4 - Proyecciones a DTOs

Práctica

Ejercicio: Añadir DTOs y métodos de proyección al repositorio de órdenes del proyecto AceriaData. Crear DTOs para resúmenes, órdenes con planchas, órdenes con detalle y órdenes con totales. Crear un caso de uso que ejecute estas proyecciones, analice el SQL generado y compare el rendimiento de proyectar versus cargar entidades completas.

Contexto del proyecto: En el punto 3.3 se añadieron proyecciones básicas con Select y tipos anónimos. En este punto se profundiza en las proyecciones a DTOs, incluyendo proyecciones con constructor, colecciones de navegación, DTOs anidados y SelectMany. Estas proyecciones se usarán en el punto 3.5 para las consultas de agregación.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO PlanchaDto

Crear el archivo src/AceriaData.Application/Dtos/PlanchaDto.cs:

```
namespace AceriaData.Application.Dtos;

public class PlanchaDto
{
    public int Id { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class PlanchaDto → declara el DTO.

Línea 5: public int Id { get; set; } → identificador de la plancha.

Línea 6: public double Espesor { get; set; } → espesor.

Línea 7: public double Ancho { get; set; } → ancho.

Línea 8: public double Largo { get; set; } → largo.

Línea 9: public decimal Peso { get; set; } → peso.

Línea 10: public bool Activa { get; set; } → indica si está activa.

Error común: si el DTO tiene propiedades que no se proyectan, EF Core deja las propiedades con su valor por defecto. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO OrdenConPlanchasDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConPlanchasDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConPlanchasDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConPlanchasDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → estado.

Línea 8: public List Planchas { get; set; } = new(); → colección de planchas.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

Paso 4: Crear el DTO DetalleDto

Crear el archivo src/AceriaData.Application/Dtos/DetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class DetalleDto
{
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class DetalleDto → declara el DTO.

Línea 5: public string ComposicionQuimica { get; set; } = string.Empty; → composición química.

Línea 6: public double TemperaturaColada { get; set; } → temperatura de la colada.

Línea 7: public string? Notas { get; set; } → notas opcionales.

Error común: si el DTO tiene una propiedad que no se proyecta y se accede a ella, el valor es null o por defecto. Se deben proyectar todas las propiedades necesarias.

Paso 5: Crear el DTO OrdenConDetalleDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConDetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConDetalleDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: `public class OrdenConDetalleDto` → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public string Estado { get; set; } = string.Empty;` → estado.

Línea 8: `public DetalleDto? Detalle { get; set; }` → detalle opcional.

Error común: si el DTO no marca la propiedad `Detalle` como anulable, el compilador lanza un warning. Se debe marcar como `?` si puede ser null.

| Paso 6: Crear el DTO `OrdenConTotalesDto`

Crear el archivo `src/AceriaData.Application/Dtos/OrdenConTotalesDto.cs`:

```
namespace AceriaData.Application.Dtos;

public class OrdenConTotalesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
    public double PesoPromedio { get; set; }
}
```

Línea 1: `namespace AceriaData.Application.Dtos;` → declara el espacio de nombres.

Línea 3: `public class OrdenConTotalesDto` → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public int TotalPlanchas { get; set; }` → total de planchas.

Línea 8: `public decimal PesoTotal { get; set; }` → peso total.

Línea 9: `public double PesoPromedio { get; set; }` → peso promedio.

Error común: si el DTO usa `int` para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar `decimal` para los pesos.

| Paso 7: Añadir los métodos de proyección a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime
hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();

    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades del dominio.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable.

Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente.

Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas.

Línea 14: List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta); → método que obtiene por rango de fechas.

Línea 15: List ObtenerPorClienteOrdenadas(string cliente); → método que obtiene por cliente ordenadas.

Línea 16: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → método que combina cliente y rango.

Línea 18: List ObtenerClientesUnicos(); → método que obtiene los clientes únicos.

Línea 19: List ObtenerResumenes(); → método que obtiene los resúmenes.

Línea 20: List ObtenerResumenesPorEstado(string estado); → método que obtiene los resúmenes por estado.

Línea 21: List ObtenerOrdenesConTotales(); → método que obtiene las órdenes con totales.

Línea 23: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.

Línea 24: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.

Línea 25: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.

Línea 27: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 28: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 8: Implementar los métodos de proyección en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories.OrdenRepositorio.cs`:

```
public List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList()
        }).ToList();
}

public List<OrdenConDetalleDto> ObtenerOrdenesConDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        }).ToList();
}

public List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConTotalesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            TotalPlanchas = o.Planchas.Count(),
            PesoTotal = o.Planchas.Sum(p => p.Peso),
            PesoPromedio = o.Planchas.Average(p => (double)p.Peso)
        })
}
```

```

    })
    .ToList();
}

```

Línea 1: `public List ObtenerOrdenesConPlanchas()` → declara el método que obtiene las órdenes con planchas.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.Select(o => new OrdenConPlanchasDto` → proyecta al DTO.

Línea 6: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 7: `Cliente = o.Cliente,` → asigna el cliente.

Línea 8: `Estado = o.Estado,` → asigna el estado.

Línea 9: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta la colección de planchas.

Línea 10: `Id = p.Id,` → asigna el Id.

Línea 11: `Espesor = p.Espesor,` → asigna el espesor.

Línea 12: `Ancho = p.Ancho,` → asigna el ancho.

Línea 13: `Largo = p.Largo,` → asigna el largo.

Línea 14: `Peso = p.Peso,` → asigna el peso.

Línea 15: `Activa = p.Activa` → asigna el estado activo.

Línea 16: `}).ToList()` → materializa la colección de planchas.

Línea 17: `})` → cierra la proyección.

Línea 18: `.ToList();` → materializa la consulta.

Línea 21: `public List ObtenerOrdenesConDetalle()` → declara el método que obtiene las órdenes con detalle.

Línea 23: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 24: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 25: `.Select(o => new OrdenConDetalleDto` → proyecta al DTO.

Línea 26: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 27: `Cliente = o.Cliente,` → asigna el cliente.

Línea 28: `Estado = o.Estado,` → asigna el estado.

Línea 29: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta.

Línea 30: `ComposicionQuimica = o.Detalle.ComposicionQuimica,` → asigna la composición química.

Línea 31: `TemperaturaColada = o.Detalle.TemperaturaColada,` → asigna la temperatura.

Línea 32: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 33: `}` → cierra la proyección del detalle.

Línea 34: `})` → cierra la proyección.

Línea 35: `.ToList();` → materializa la consulta.

Línea 38: `public List ObtenerOrdenesConTotalesDetallados()` → declara el método que obtiene las órdenes con totales detallados.

Línea 40: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 41: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 42: `.Select(o => new OrdenConTotalesDto` → proyecta al DTO.

Línea 43: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 44: `Cliente = o.Cliente,` → asigna el cliente.

Línea 45: `TotalPlanchas = o.Planchas.Count(),` → calcula el total de planchas.

Línea 46: `PesoTotal = o.Planchas.Sum(p => p.Peso),` → calcula el peso total.

Línea 47: `PesoPromedio = o.Planchas.Average(p => (double)p.Peso)` → calcula el peso promedio.

Línea 48: `})` → cierra la proyección.

Línea 49: `.ToList();` → materializa la consulta.

Error común: si se proyecta una colección sin `ToList`, EF Core puede no materializar la colección correctamente. Se debe llamar a `ToList` dentro de la proyección para materializar la colección.

| Paso 9: Crear el caso de uso de proyecciones a DTOs

Crear el archivo `src/AceriaData.Application/UseCases/ProyeccionesDtoUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProyeccionesDtoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES A DTOS ===");

        DemostrarOrdenesConPlanchas();
        DemostrarOrdenesConDetalle();
        DemostrarOrdenesConTotalesDetallados();
        DemostrarSqlProyeccionConPlanchas();
    }

    private void DemostrarOrdenesConPlanchas()
    {
        Console.WriteLine("\n--- Órdenes con planchas ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchas();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.Planchas.Count}");
            foreach (var plancha in orden.Planchas)
            {
                Console.WriteLine($"   Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");
            }
        }
    }

    private void DemostrarOrdenesConDetalle()
    {
        Console.WriteLine("\n--- Órdenes con detalle ---");
    }
}
```

```

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConDetalle();
        foreach (var orden in ordenes)
        {
            var detalle = orden.Detalle == null ? "Sin detalle" :
            $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Detalle:
            {detalle}");
        }
    }

    private void DemostrarOrdenesConTotalesDetallados()
    {
        Console.WriteLine("\n--- Órdenes con totales detallados ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotalesDetallados();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
            {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg | Peso promedio: {orden.PesoPromedio:F2} kg");
        }
    }

    private void DemostrarSqlProyeccionConPlanchas()
    {
        Console.WriteLine("\n--- SQL generado por una proyección con colección de navegación ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .OrderBy(o => o.NumeroOrden)
            .Select(o => new
            {
                o.NumeroOrden,
                o.Cliente,
                Planchas = o.Planchas.Select(p => new { p.Id, p.Espesor, p.Peso }).ToList()
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class ProyeccionesDtoUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public ProyeccionesDtoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== PROYECCIONES A DTOS ==="); → muestra la cabecera.

Línea 18: DemostrarOrdenesConPlanchas(); → llama al método de órdenes con planchas.

Línea 19: DemostrarOrdenesConDetalle(); → llama al método de órdenes con detalle.

Línea 20: DemostrarOrdenesConTotalesDetallados(); → llama al método de órdenes con totales detallados.

Línea 21: DemostrarSqlProyeccionConPlanchas(); → llama al método de SQL de proyección con planchas.

Línea 24: private void DemostrarOrdenesConPlanchas() → declara el método.

Línea 26: Console.WriteLine("\n--- Órdenes con planchas ---"); → muestra la cabecera.

Línea 28: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchas(); → llama al método del repositorio.

Línea 29: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 31: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Planchas: {orden.Planchas.Count}");` → muestra los datos.

Línea 32: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 34: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");` → muestra los datos de cada plancha.

Línea 38: `private void DemostrarOrdenesConDetalle()` → declara el método.

Línea 40: `Console.WriteLine("\n--- Órdenes con detalle ---");` → muestra la cabecera.

Línea 42: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConDetalle();` → llama al método del repositorio.

Línea 43: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 45: `var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";` → comprueba si el detalle existe.

Línea 46: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | Detalle: {detalle}");` → muestra los datos.

Línea 50: `private void DemostrarOrdenesConTotalesDetallados()` → declara el método.

Línea 52: `Console.WriteLine("\n--- Órdenes con totales detallados ---");` → muestra la cabecera.

Línea 54: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConTotalesDetallados();` → llama al método del repositorio.

Línea 55: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 57: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.TotalPlanchas} | Peso total: {orden.PesoTotal} kg | Peso promedio: {orden.PesoPromedio:F2} kg");` → muestra los datos.

Línea 61: `private void DemostrarSqlProyeccionConPlanchas()` → declara el método.

Línea 63: `Console.WriteLine("\n--- SQL generado por una proyección con colección de navegación ---");` → muestra la cabecera.

Línea 65: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 66: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 67: `.Select(o => new` → proyecta a un tipo anónimo.

Línea 68: `o.NumeroOrden,` → incluye el número de orden.

Línea 69: `o.Cliente,` → incluye el cliente.

Línea 70: `Planchas = o.Planchas.Select(p => new { p.Id, p.Espesor, p.Peso }).ToList()` → proyecta la colección de planchas.

Línea 71: `});` → cierra la proyección.

Línea 73: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 74: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 10: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProyeccionesDtoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 11: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesDtoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 12: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas y detalle:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };

    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 17: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 19: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 20: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 22: `context.SaveChanges();` → inserta las planchas y el detalle.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 13: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada proyección. Se observan las órdenes con sus planchas, las órdenes con detalle, las órdenes con totales detallados y el SQL generado por la proyección con colección de navegación.

Paso 14: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROYECCIONES A DTOS ===
```

```
--- Órdenes con planchas ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | Planchas: 2
  Plancha 1 | Espesor: 10.5 | Peso: 370.5 kg
  Plancha 2 | Espesor: 12.0 | Peso: 280.8 kg
OF-2024-0002 | Constructora del Sur | Pendiente | Planchas: 1
  Plancha 3 | Espesor: 8.0 | Peso: 125.6 kg
```

```
--- Órdenes con detalle ---
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C
OF-2024-0002 | Constructora del Sur | Pendiente | Detalle: Sin detalle
```

```
--- Órdenes con totales detallados ---
```

```
OF-2024-0001 | Constructora del Norte | Planchas: 2 | Peso total: 651.3 kg | Peso promedio: 325.65 kg
OF-2024-0002 | Constructora del Sur | Planchas: 1 | Peso total: 125.6 kg | Peso promedio: 125.60 kg
```

```
--- SQL generado por una proyección con colección de navegación ---
```

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Id], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[NumeroOrden], [o].[Id], [p].[Id]
```

La primera sección muestra las órdenes con sus planchas. La segunda sección muestra las órdenes con detalle. La tercera sección muestra las órdenes con totales detallados. La cuarta sección muestra el SQL generado por la proyección con colección de navegación.

Observaciones: el SQL generado incluye un `LEFT JOIN` con la tabla de planchas. Los filtros globales de

Soft Delete se aplican automáticamente a ambas tablas. El orden de las cláusulas es el correcto: SELECT, FROM, LEFT JOIN, WHERE, ORDER BY.

Paso 15: Diagnosticar un error común

Modificar el método ObtenerOrdenesConDetalle para acceder a una propiedad del detalle sin comprobar si existe:

```
csharp
public List<OrdenConDetalleDto> ObtenerOrdenesConDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Detalle = new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}
```

Resultado esperado: si alguna orden no tiene detalle, se produce una `NullReferenceException` en tiempo de ejecución porque `o.Detalle` es null. EF Core no puede traducir la proyección a SQL y lanza una excepción.

Solución: comprobar si `o.Detalle` es null antes de proyectarlo, como en la implementación original.

Errores comunes del ejercicio Error Causa Solución `NullReferenceException` en DTO anidado No se comprobó si la entidad relacionada es null Usar operador ternario antes de proyectar Colección no materializada Se olvidó `ToList` dentro de la proyección Añadir `ToList` dentro de la proyección Propiedad no proyectada Se accede a una propiedad que no está en la proyección Proyectar todas las propiedades necesarias `GroupBy` sin agregación Se proyecta una propiedad que no está en el `GroupBy` Usar la clave del grupo o funciones de agregación `SelectMany` con colección vacía La entidad principal no aparece en el resultado Usar `DefaultIfEmpty` `Average` sobre colección vacía Devuelve null Usar `DefaultIfEmpty` o comprobar el resultado

■ Reto resuelto: Proyección de órdenes con planchas y detalle combinados

Reto: Crear un DTO `OrdenCompletaDto` que contenga el número de orden, el cliente, el estado, la colección de planchas y el detalle. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO `OrdenCompletaDto`:

```
namespace AceriaData.Application.Dtos;

public class OrdenCompletaDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: `namespace AceriaData.Application.Dtos;` → declara el espacio de nombres.

Línea 3: `public class OrdenCompletaDto` → declara el DTO.

Línea 5: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden.

Línea 6: `public string Cliente { get; set; } = string.Empty;` → cliente.

Línea 7: `public string Estado { get; set; } = string.Empty;` → estado.

Línea 8: `public List Planchas { get; set; } = new();` → colección de planchas.

Línea 9: `public DetalleDto? Detalle { get; set; }` → detalle opcional.

Paso 2: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<OrdenCompletaDto> ObtenerOrdenesCompletas();
```

Línea 1: `List ObtenerOrdenesCompletas();` → declara el método.

Paso 3: Implementar el método en `OrdenRepositorio`:

```
public List<OrdenCompletaDto> ObtenerOrdenesCompletas()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenCompletaDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesCompletas();` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.Select(o => new OrdenCompletaDto` → proyecta al DTO.

Línea 6: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 7: `Cliente = o.Cliente,` → asigna el cliente.

Línea 8: `Estado = o.Estado,` → asigna el estado.

Línea 9: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta la colección de planchas.

Línea 17: `}).ToList(),` → materializa la colección de planchas.

Línea 18: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta.

Línea 24: `})` → cierra la proyección.

Línea 25: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarOrdenesCompletas()
{
    Console.WriteLine("\n--- Órdenes completas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"      Planchas: {orden.Planchas.Count}");
        var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
        Console.WriteLine($"      Detalle: {detalle}");
    }
}
```

Línea 1: private void DemostrarOrdenesCompletas() → declara el método.

Línea 3: Console.WriteLine("\n--- Órdenes completas ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletas(); → llama al método del repositorio.

Línea 6: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 8: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos de la orden.

Línea 9: Console.WriteLine(\$" Planchas: {orden.Planchas.Count}"); → muestra el número de planchas.

Línea 10: var detalle = orden.Detalle == null ? "Sin detalle" : \$"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C"; → comprueba si el detalle existe.

Línea 11: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarOrdenesCompletas();
Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas y detalle.
```

Resultado esperado: las órdenes se muestran con su número, cliente, estado, número de planchas y detalle.

■ Analogía final

Las proyecciones a DTOs en una acería son como los formularios estandarizados que se usan para transferir información entre departamentos. En lugar de enviar la carpeta completa de una orden con todos sus documentos, se envía un formulario con los campos que el departamento receptor necesita. El formulario tiene un nombre, un formato fijo y se puede archivar. Proyectar a un DTO con inicializador de objeto es como rellenar un formulario campo a campo. Proyectar a un DTO con constructor es como usar un formulario preimpreso que garantiza que todos los campos estén rellenos. Proyectar una colección de navegación dentro de un DTO es como adjuntar una lista de planchas al formulario de la orden. Proyectar un DTO anidado es como incluir un formulario de detalle dentro del formulario principal. SelectMany es como aplanar varias listas en una sola. GroupBy con agregaciones es como pedir un resumen por cliente con totales. Así funcionan las proyecciones a DTOs en EF Core: se seleccionan solo los datos necesarios, se estructuran en formularios estandarizados y se transfieren entre capas de forma eficiente.

■ Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `PlanchaDto`, `OrdenConPlanchasDto`, `DetalleDto`, `OrdenConDetalleDto` y `OrdenConTotalesDto`.

Añadido los métodos de proyección a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ProyeccionesDtoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas y detalle.

Ejecutado las demostraciones de órdenes con planchas, órdenes con detalle y órdenes con totales detallados.

Analizado el SQL generado por una proyección con colección de navegación.

Diagnosticado el error de no comprobar si la entidad relacionada es null.

Creado el DTO `OrdenCompletaDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido proyecciones a DTOs al proyecto `AceriaData`, incluyendo proyecciones con inicializador de objeto, proyecciones con constructor, proyecciones de colecciones de navegación, proyecciones de DTOs anidados y el operador `SelectMany`. Se ha analizado el SQL generado por las proyecciones y se ha comprobado que EF Core genera un `LEFT JOIN` para las colecciones de navegación. En el siguiente punto se estudiarán las consultas de agregación con `Count`, `Sum`, `Average`, `Min`, `Max` y `GroupBy`.

Punto 3.5 - Consultas de agregación: Count, Sum, Average, Min y Max

Práctica

Ejercicio: Añadir métodos de agregación al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen Count, Sum, Average, Min, Max y GroupBy. Crear un caso de uso que ejecute estas agregaciones, analice el SQL generado y compare el rendimiento de agregar en el servidor versus agregar en memoria.

Contexto del proyecto: En el punto 3.4 se añadieron proyecciones a DTOs, incluyendo colecciones de navegación y DTOs anidados. En este punto se añaden consultas de agregación que calculan resúmenes directamente en el servidor. Estas agregaciones se usarán en el punto 3.6 para las agrupaciones con proyección y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO ResumenPorClienteDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteDto
{
    public string Cliente { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
    public DateTime FechaMasReciente { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public int TotalPlanchas { get; set; } → total de planchas.

Línea 8: public decimal PesoTotal { get; set; } → peso total.

Línea 9: public DateTime FechaMasReciente { get; set; } → fecha más reciente.

Error común: si el DTO tiene una propiedad que no se proyecta, EF Core deja la propiedad con su valor por defecto. Se deben proyectar todas las propiedades que se van a usar.

Paso 3: Crear el DTO ResumenPorEstadoDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorEstadoDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorEstadoDto
{
    public string Estado { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorEstadoDto → declara el DTO.

Línea 5: public string Estado { get; set; } = string.Empty; → estado.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public decimal PesoTotal { get; set; } → peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

■ Paso 4: Añadir los métodos de agregación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
    List<OrdenCompletaDto> ObtenerOrdenesCompletas();

    int ContarOrdenes();
    int ContarOrdenesPorEstado(string estado);
    bool ExisteAlgunaOrden();
    bool TodasLasOrdenesPendientes();
    decimal ObtenerPesoTotalDePlanchas();
    double ObtenerPesoPromedioDePlanchas();
    decimal ObtenerPesoMinimoDePlanchas();
    decimal ObtenerPesoMaximoDePlanchas();
    DateTime ObtenerFechaMasAntigua();
    DateTime ObtenerFechaMasReciente();

    List<ResumenPorClienteDto> ObtenerResumenPorCliente();
    List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
    List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();
}
```

```

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable.

Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente.

Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas.

Línea 14: List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta); → método que obtiene por rango de fechas.

Línea 15: List ObtenerPorClienteOrdenadas(string cliente); → método que obtiene por cliente ordenadas.

Línea 16: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → método que combina cliente y rango.

Línea 18: List ObtenerClientesUnicos(); → método que obtiene los clientes únicos.

Línea 19: List ObtenerResumenes(); → método que obtiene los resúmenes.

Línea 20: List ObtenerResumenesPorEstado(string estado); → método que obtiene los resúmenes por estado.

Línea 21: List ObtenerOrdenesConTotales(); → método que obtiene las órdenes con totales.

Línea 22: List ObtenerOrdenesConPlanchas(); → método que obtiene las órdenes con planchas.

Línea 23: List ObtenerOrdenesConDetalle(); → método que obtiene las órdenes con detalle.

Línea 24: List ObtenerOrdenesConTotalesDetallados(); → método que obtiene las órdenes con totales detallados.

Línea 25: List ObtenerOrdenesCompletas(); → método que obtiene las órdenes completas.

Línea 27: int ContarOrdenes(); → método que cuenta las órdenes.

Línea 28: int ContarOrdenesPorEstado(string estado); → método que cuenta las órdenes por estado.

Línea 29: bool ExisteAlgunaOrden(); → método que comprueba si existe alguna orden.

Línea 30: bool TodasLasOrdenesPendientes(); → método que comprueba si todas las órdenes están pendientes.

Línea 31: decimal ObtenerPesoTotalDePlanchas(); → método que obtiene el peso total de las planchas.

Línea 32: double ObtenerPesoPromedioDePlanchas(); → método que obtiene el peso promedio de las planchas.

Línea 33: decimal ObtenerPesoMinimoDePlanchas(); → método que obtiene el peso mínimo de las planchas.

Línea 34: decimal ObtenerPesoMaximoDePlanchas(); → método que obtiene el peso máximo de las planchas.

Línea 35: DateTime ObtenerFechaMasAntigua(); → método que obtiene la fecha más antigua.

Línea 36: DateTime ObtenerFechaMasReciente(); → método que obtiene la fecha más reciente.

Línea 38: List ObtenerResumenPorCliente(); → método que obtiene el resumen por cliente.

Línea 39: List ObtenerResumenPorEstado(); → método que obtiene el resumen por estado.

Línea 40: List ObtenerClientesConMasDeUnaOrden(); → método que obtiene los clientes con más de una orden.

Línea 42: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 43: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver valores simples o DTOs concretos.

| Paso 5: Implementar los métodos de agregación en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```
public int ContarOrdenes()
{
    return _context.OrdenesFabricacion.Count();
}

public int ContarOrdenesPorEstado(string estado)
{
    return _context.OrdenesFabricacion.Count(o => o.Estado == estado);
}

public bool ExisteAlgunaOrden()
{
    return _context.OrdenesFabricacion.Any();
}

public bool TodasLasOrdenesPendientes()
{
    return _context.OrdenesFabricacion.All(o => o.Estado == "Pendiente");
}

public decimal ObtenerPesoTotalDePlanchas()
{
    return _context.PlanchasAcero.Sum(p => p.Peso);
}

public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero.Average(p => (double)p.Peso);
}

public decimal ObtenerPesoMinimoDePlanchas()
{
    return _context.PlanchasAcero.Min(p => p.Peso);
}

public decimal ObtenerPesoMaximoDePlanchas()
{
    return _context.PlanchasAcero.Max(p => p.Peso);
}

public DateTime ObtenerFechaMasAntigua()
{
    return _context.OrdenesFabricacion.Min(o => o.FechaCreacion);
}

public DateTime ObtenerFechaMasReciente()
```

```

{
    return _context.OrdenesFabricacion.Max(o => o.FechaCreacion);
}

public List<ResumenPorClienteDto> ObtenerResumenPorCliente()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
            FechaMasReciente = g.Max(o => o.FechaCreacion)
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

public List<ResumenPorEstadoDto> ObtenerResumenPorEstado()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Estado)
        .Select(g => new ResumenPorEstadoDto
        {
            Estado = g.Key,
            TotalOrdenes = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Estado)
        .ToList();
}

public List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Where(g => g.Count() > 1)
        .Select(g => new ResumenPorClienteDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
            FechaMasReciente = g.Max(o => o.FechaCreacion)
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

```

Línea 1: public int ContarOrdenes() → declara el método que cuenta las órdenes.

Línea 3: return _context.OrdenesFabricacion.Count(); → cuenta todas las órdenes.

Línea 6: public int ContarOrdenesPorEstado(string estado) → declara el método que cuenta las órdenes por estado.

Línea 8: return _context.OrdenesFabricacion.Count(o => o.Estado == estado); → cuenta las órdenes que cumplen la condición.

Línea 11: public bool ExisteAlgunaOrden() → declara el método que comprueba si existe alguna orden.

Línea 13: return _context.OrdenesFabricacion.Any(); → comprueba si hay al menos una orden.

Línea 16: public bool TodasLasOrdenesPendientes() → declara el método que comprueba si todas las órdenes están pendientes.

Línea 18: return _context.OrdenesFabricacion.All(o => o.Estado == "Pendiente"); → comprueba si todas cumplen la condición.

Línea 21: `public decimal ObtenerPesoTotalDePlanchas()` → declara el método que obtiene el peso total.

Línea 23: `return _context.PlanchasAcero.Sum(p => p.Peso);` → suma el peso de todas las planchas.

Línea 26: `public double ObtenerPesoPromedioDePlanchas()` → declara el método que obtiene el peso promedio.

Línea 28: `return _context.PlanchasAcero.Average(p => (double)p.Peso);` → calcula el peso promedio.

Línea 31: `public decimal ObtenerPesoMinimoDePlanchas()` → declara el método que obtiene el peso mínimo.

Línea 33: `return _context.PlanchasAcero.Min(p => p.Peso);` → obtiene el peso mínimo.

Línea 36: `public decimal ObtenerPesoMaximoDePlanchas()` → declara el método que obtiene el peso máximo.

Línea 38: `return _context.PlanchasAcero.Max(p => p.Peso);` → obtiene el peso máximo.

Línea 41: `public DateTime ObtenerFechaMasAntigua()` → declara el método que obtiene la fecha más antigua.

Línea 43: `return _context.OrdenesFabricacion.Min(o => o.FechaCreacion);` → obtiene la fecha mínima.

Línea 46: `public DateTime ObtenerFechaMasReciente()` → declara el método que obtiene la fecha más reciente.

Línea 48: `return _context.OrdenesFabricacion.Max(o => o.FechaCreacion);` → obtiene la fecha máxima.

Línea 51: `public List ObtenerResumenPorCliente()` → declara el método que obtiene el resumen por cliente.

Línea 53: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 54: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 55: `.Select(g => new ResumenPorClienteDto` → proyecta cada grupo al DTO.

Línea 56: `Cliente = g.Key,` → asigna el cliente.

Línea 57: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 58: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 59: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),` → suma el peso de las planchas del grupo.

Línea 60: `FechaMasReciente = g.Max(o => o.FechaCreacion)` → obtiene la fecha más reciente del grupo.

Línea 61: `})` → cierra la proyección.

Línea 62: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 63: `.ToList();` → materializa la consulta.

Línea 66: `public List ObtenerResumenPorEstado()` → declara el método que obtiene el resumen por estado.

Línea 68: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 69: `.GroupBy(o => o.Estado)` → agrupa por estado.

Línea 70: `.Select(g => new ResumenPorEstadoDto` → proyecta cada grupo al DTO.

Línea 71: `Estado = g.Key,` → asigna el estado.

Línea 72: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 73: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas del grupo.

Línea 74: `}` → cierra la proyección.

Línea 75: `.OrderBy(r => r.Estado)` → ordena por estado.

Línea 76: `.ToList();` → materializa la consulta.

Línea 79: `public List ObtenerClientesConMasDeUnaOrden()` → declara el método que obtiene los clientes con más de una orden.

Línea 81: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 82: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 83: `.Where(g => g.Count() > 1)` → filtra los grupos con más de una orden. Se traduce a HAVING.

Línea 84: `.Select(g => new ResumenPorClienteDto` → proyecta cada grupo al DTO.

Línea 85: `Cliente = g.Key,` → asigna el cliente.

Línea 86: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 87: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 88: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),` → suma el peso de las planchas del grupo.

Línea 89: `FechaMasReciente = g.Max(o => o.FechaCreacion)` → obtiene la fecha más reciente del grupo.

Línea 90: `}` → cierra la proyección.

Línea 91: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 92: `.ToList();` → materializa la consulta.

Error común: si se proyecta una propiedad que no está en el `GroupBy` ni en una función de agregación, EF Core lanza una excepción. Solo se pueden proyectar la clave del grupo o funciones de agregación.

| Paso 6: Crear el caso de uso de agregaciones

Crear el archivo `src/AceriaData.Application/UseCases/AgregacionesUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AgregacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AgregacionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS DE AGREGACIÓN ===");

        DemostrarConteos();
        DemostrarAnyAll();
        DemostrarSumAverageMinMax();
        DemostrarFechas();
        DemostrarResumenPorCliente();
        DemostrarResumenPorEstado();
        DemostrarClientesConMasDeUnaOrden();
        DemostrarSqlAgregacion();
    }

    private void DemostrarConteos()
    {
        Console.WriteLine("\n--- Conteos ---");
    }
}
```

```

        var total = _unidad.Ordenes.ContarOrdenes();
        var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");

        Console.WriteLine($"Total de órdenes: {total}");
        Console.WriteLine($"Órdenes pendientes: {pendientes}");
    }

    private void DemostrarAnyAll()
    {
        Console.WriteLine("\n--- Any y All ---");

        var existeAlguna = _unidad.Ordenes.ExisteAlgunaOrden();
        var todasPendientes = _unidad.Ordenes.TodasLasOrdenesPendientes();

        Console.WriteLine($"Existe alguna orden: {existeAlguna}");
        Console.WriteLine($"Todas las órdenes pendientes: {todasPendientes}");
    }

    private void DemostrarSumAverageMinMax()
    {
        Console.WriteLine("\n--- Sum, Average, Min y Max ---");

        var pesoTotal = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();
        var pesoPromedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();
        var pesoMinimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();
        var pesoMaximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();

        Console.WriteLine($"Peso total: {pesoTotal} kg");
        Console.WriteLine($"Peso promedio: {pesoPromedio:F2} kg");
        Console.WriteLine($"Peso mínimo: {pesoMinimo} kg");
        Console.WriteLine($"Peso máximo: {pesoMaximo} kg");
    }

    private void DemostrarFechas()
    {
        Console.WriteLine("\n--- Fechas ---");

        var fechaAntigua = _unidad.Ordenes.ObtenerFechaMasAntigua();
        var fechaReciente = _unidad.Ordenes.ObtenerFechaMasReciente();

        Console.WriteLine($"Fecha más antigua: {fechaAntigua:dd/MM/yyyy}");
        Console.WriteLine($"Fecha más reciente: {fechaReciente:dd/MM/yyyy}");
    }

    private void DemostrarResumenPorCliente()
    {
        Console.WriteLine("\n--- Resumen por cliente ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorCliente();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");
        }
    }

    private void DemostrarResumenPorEstado()
    {
        Console.WriteLine("\n--- Resumen por estado ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorEstado();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");
        }
    }

    private void DemostrarClientesConMasDeUnaOrden()
    {
        Console.WriteLine("\n--- Clientes con más de una orden ---");
    }

```

```

        var resmenes = _unidad.Ordenes.ObtenerClientesConMasDeUnaOrden();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes}");
        }
    }

    private void DemostrarSqlAgregacion()
    {
        Console.WriteLine("\n--- SQL generado por una agregación agrupada ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .GroupBy(o => o.Cliente)
            .Select(g => new
            {
                Cliente = g.Key,
                Total = g.Count(),
                PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class AgregacionesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public AgregacionesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CONSULTAS DE AGREGACIÓN ==="); → muestra la cabecera.

Línea 18: DemostrarConteos(); → llama al método de conteos.

Línea 19: DemostrarAnyAll(); → llama al método de Any y All.

Línea 20: DemostrarSumAverageMinMax(); → llama al método de Sum, Average, Min y Max.

Línea 21: DemostrarFechas(); → llama al método de fechas.

Línea 22: DemostrarResumenPorCliente(); → llama al método de resumen por cliente.

Línea 23: DemostrarResumenPorEstado(); → llama al método de resumen por estado.

Línea 24: DemostrarClientesConMasDeUnaOrden(); → llama al método de clientes con más de una orden.

Línea 25: DemostrarSqlAgregacion(); → llama al método de SQL de agregación.

Línea 28: private void DemostrarConteos() → declara el método.

Línea 30: Console.WriteLine("\n--- Conteos ---"); → muestra la cabecera.

Línea 32: var total = _unidad.Ordenes.ContarOrdenes(); → llama al método del repositorio.

Línea 33: var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente"); → llama al método del repositorio.

Línea 35: Console.WriteLine(\$"Total de órdenes: {total}"); → muestra el total.

Línea 36: Console.WriteLine(\$"Órdenes pendientes: {pendientes}"); → muestra las pendientes.

Línea 39: private void DemostrarAnyAll() → declara el método.

Línea 41: `Console.WriteLine("\n--- Any y All ---");` → muestra la cabecera.

Línea 43: `var existeAlguna = _unidad Ordenes.ExisteAlgunaOrden();` → llama al método del repositorio.

Línea 44: `var todasPendientes = _unidad Ordenes.TodasLasOrdenesPendientes();` → llama al método del repositorio.

Línea 46: `Console.WriteLine($"Existe alguna orden: {existeAlguna}");` → muestra el resultado.

Línea 47: `Console.WriteLine($"Todas las órdenes pendientes: {todasPendientes}");` → muestra el resultado.

Línea 50: `private void DemostrarSumAverageMinMax()` → declara el método.

Línea 52: `Console.WriteLine("\n--- Sum, Average, Min y Max ---");` → muestra la cabecera.

Línea 54: `var pesoTotal = _unidad Ordenes.ObtenerPesoTotalDePlanchas();` → llama al método del repositorio.

Línea 55: `var pesoPromedio = _unidad Ordenes.ObtenerPesoPromedioDePlanchas();` → llama al método del repositorio.

Línea 56: `var pesoMinimo = _unidad Ordenes.ObtenerPesoMinimoDePlanchas();` → llama al método del repositorio.

Línea 57: `var pesoMaximo = _unidad Ordenes.ObtenerPesoMaximoDePlanchas();` → llama al método del repositorio.

Línea 59: `Console.WriteLine($"Peso total: {pesoTotal} kg");` → muestra el peso total.

Línea 60: `Console.WriteLine($"Peso promedio: {pesoPromedio:F2} kg");` → muestra el peso promedio.

Línea 61: `Console.WriteLine($"Peso mínimo: {pesoMinimo} kg");` → muestra el peso mínimo.

Línea 62: `Console.WriteLine($"Peso máximo: {pesoMaximo} kg");` → muestra el peso máximo.

Línea 65: `private void DemostrarFechas()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Fechas ---");` → muestra la cabecera.

Línea 69: `var fechaAntigua = _unidad Ordenes.ObtenerFechaMasAntigua();` → llama al método del repositorio.

Línea 70: `var fechaReciente = _unidad Ordenes.ObtenerFechaMasReciente();` → llama al método del repositorio.

Línea 72: `Console.WriteLine($"Fecha más antigua: {fechaAntigua:dd/MM/yyyy}");` → muestra la fecha más antigua.

Línea 73: `Console.WriteLine($"Fecha más reciente: {fechaReciente:dd/MM/yyyy}");` → muestra la fecha más reciente.

Línea 76: `private void DemostrarResumenPorCliente()` → declara el método.

Línea 78: `Console.WriteLine("\n--- Resumen por cliente ---");` → muestra la cabecera.

Línea 80: `var resúmenes = _unidad Ordenes.ObtenerResumenPorCliente();` → llama al método del repositorio.

Línea 81: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 83: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");` → muestra los datos.

Línea 87: `private void DemostrarResumenPorEstado()` → declara el método.

Línea 89: `Console.WriteLine("\n--- Resumen por estado ---");` → muestra la cabecera.

Línea 91: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorEstado();` → llama al método del repositorio.

Línea 92: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 94: `Console.WriteLine($" {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 98: `private void DemostrarClientesConMasDeUnaOrden()` → declara el método.

Línea 100: `Console.WriteLine("\n--- Clientes con más de una orden ---");` → muestra la cabecera.

Línea 102: `var resúmenes = _unidad.Ordenes.ObtenerClientesConMasDeUnaOrden();` → llama al método del repositorio.

Línea 103: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 105: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes}");` → muestra los datos.

Línea 109: `private void DemostrarSqlAgregacion()` → declara el método.

Línea 111: `Console.WriteLine("\n--- SQL generado por una agregación agrupada ---");` → muestra la cabecera.

Línea 113: `var consulta = _unidad.Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 114: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 115: `.Select(g => new` → proyecta a un tipo anónimo.

Línea 116: `Cliente = g.Key,` → incluye el cliente.

Línea 117: `Total = g.Count(),` → cuenta las órdenes.

Línea 118: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas.

Línea 119: `});` → cierra la proyección.

Línea 121: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 122: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<AgregacionesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    var orden4 = new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) };

    context OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `var orden4 = new OrdenFabricacion { ... };` → crea la cuarta orden.

Línea 12: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);` → registra las órdenes.

Línea 13: `context.SaveChanges();` → inserta las órdenes.

Línea 15: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 16: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 17: `var plancha3 = new PlanchaAcero { ... }; → crea la tercera plancha.`

Línea 18: `var plancha4 = new PlanchaAcero { ... }; → crea la cuarta plancha.`

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4); → registra las planchas.`

Línea 21: `context.SaveChanges(); → inserta las planchas.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada agregación. Se observan los conteos, Any y All, las sumas, promedios, mínimos y máximos, las fechas, los resúmenes por cliente y estado, los clientes con más de una orden y el SQL generado por una agregación agrupada.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CONSULTAS DE AGREGACIÓN ===

--- Conteos ---
Total de órdenes: 4
Órdenes pendientes: 3

--- Any y All ---
Existe alguna orden: True
Todas las órdenes pendientes: False

--- Sum, Average, Min y Max ---
Peso total: 1226.9 kg
Peso promedio: 306.73 kg
Peso mínimo: 125.6 kg
Peso máximo: 450.0 kg

--- Fechas ---
Fecha más antigua: 15/01/2024
Fecha más reciente: 05/04/2024

--- Resumen por cliente ---
Constructora del Norte | Órdenes: 3 | Planchas: 3 | Peso: 1101.3 kg | Última: 05/04/2024
Constructora del Sur | Órdenes: 1 | Planchas: 1 | Peso: 125.6 kg | Última: 20/02/2024

--- Resumen por estado ---
EnProceso | Órdenes: 1 | Peso: 450.0 kg
Pendiente | Órdenes: 3 | Peso: 776.9 kg

--- Clientes con más de una orden ---
Constructora del Norte | Órdenes: 3

--- SQL generado por una agregación agrupada ---
SELECT [o].[Cliente], COUNT(*) AS [Total], COALESCE(SUM((
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
)), 0.0) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
GROUP BY [o].[Cliente]

La primera sección muestra los conteos. La segunda sección muestra Any y All. La tercera sección muestra
las sumas, promedios, mínimos y máximos. La cuarta sección muestra las fechas. La quinta sección muestra
el resumen por cliente. La sexta sección muestra el resumen por estado. La séptima sección muestra los
clientes con más de una orden. La octava sección muestra el SQL generado por una agregación agrupada.
```


Observaciones: el SQL generado incluye COUNT(*), SUM, MAX y GROUP BY. El filtro global de Soft Delete se aplica automáticamente. El orden de las cláusulas es el correcto: SELECT, FROM, WHERE, GROUP BY.

Paso 12: Diagnosticar un error común

Modificar el método ObtenerPesoPromedioDePlanchas para llamar a Average sobre una colección vacía:

```
csharp
public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero
        .Where(p => p.OrdenId == 999)
        .Average(p => (double)p.Peso);
}
```

Resultado esperado: se lanza una excepción InvalidOperationException porque no se puede calcular el promedio de una colección vacía.

Solución: usar DefaultIfEmpty(0) antes de Average o comprobar si hay elementos.

```
public double ObtenerPesoPromedioDePlanchas()
{
    return _context.PlanchasAcero
        .Where(p => p.OrdenId == 999)
        .Select(p => (double)p.Peso)
        .DefaultIfEmpty(0)
        .Average();
}
```

Resultado esperado con la solución: el promedio es cero cuando no hay planchas.

Errores comunes del ejercicio

Error	Causa	Solución
Average sobre colección vacía	No se comprobó si hay elementos	Usar DefaultIfEmpty(0)
Min o Max sobre colección vacía	No se comprobó si hay elementos	Usar DefaultIfEmpty o comprobar
GroupBy sin agregación del grupo o funciones de agregación	Se proyecta una propiedad que no está en el GroupBy	Usar la clave
Where después del GroupBy para HAVING	Se usa Where en lugar de Having	Usar Where después del GroupBy
Count después de ToList directamente sobre IQueryable	Se materializó la consulta antes de contar	Llamar a Count
Sum sobre tipo anulable	Devuelve null si la colección está vacía	Usar DefaultIfEmpty o COALESCE
Any con Count() > 0	Se usa Count en lugar de Any	Usar Any para mejor rendimiento
Reto resuelto: Resumen por rango de fechas con agregaciones		
Reto: Crear un método en el repositorio que agrupe las órdenes por mes y año, y calcule el total de órdenes, el total de planchas y el peso total por cada grupo. Añadir un DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.		

Solución paso a paso:

Paso 1: Crear el DTO ResumenMensualDto:

```
csharp
namespace AceriaData.Application.Dtos;

public class ResumenMensualDto
{
    public int Anio { get; set; }
    public int Mes { get; set; }
    public int TotalOrdenes { get; set; }
    public int TotalPlanchas { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenMensualDto → declara el DTO.

Línea 5: public int Anio { get; set; } → año.

Línea 6: public int Mes { get; set; } → mes.

Línea 7: `public int TotalOrdenes { get; set; }` → total de órdenes.

Línea 8: `public int TotalPlanchas { get; set; }` → total de planchas.

Línea 9: `public decimal PesoTotal { get; set; }` → peso total.

Paso 2: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<ResumenMensualDto> ObtenerResumenMensual();
```

Línea 1: `List ObtenerResumenMensual();` → declara el método.

Paso 3: Implementar el método en `OrdenRepositorio`:

```
public List<ResumenMensualDto> ObtenerResumenMensual()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
        .Select(g => new ResumenMensualDto
        {
            Anio = g.Key.Year,
            Mes = g.Key.Month,
            TotalOrdenes = g.Count(),
            TotalPlanchas = g.Sum(o => o.Planchas.Count()),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Anio)
        .ThenBy(r => r.Mes)
        .ToList();
}
```

Línea 1: `public List ObtenerResumenMensual()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })` → agrupa por año y mes.

Línea 5: `.Select(g => new ResumenMensualDto` → proyecta cada grupo al DTO.

Línea 6: `Anio = g.Key.Year,` → asigna el año.

Línea 7: `Mes = g.Key.Month,` → asigna el mes.

Línea 8: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 9: `TotalPlanchas = g.Sum(o => o.Planchas.Count()),` → suma las planchas del grupo.

Línea 10: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso de las planchas del grupo.

Línea 11: `})` → cierra la proyección.

Línea 12: `.OrderBy(r => r.Anio)` → ordena por año.

Línea 13: `.ThenBy(r => r.Mes)` → ordena por mes.

Línea 14: `.ToList();` → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenMensual()
{
    Console.WriteLine("\n--- Resumen mensual ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenMensual();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Órdenes: {resumen.TotalOrdenes} |
Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg");
    }
}
```

Línea 1: `private void DemostrarResumenMensual()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Resumen mensual ---");` → muestra la cabecera.

Línea 5: `var resmenes = _unidad.Ordenes.ObtenerResumenMensual();` → llama al método del repositorio.

Línea 6: `foreach (var resumen in resmenes)` → itera sobre los resúmenes.

Línea 8: `Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarResumenMensual();
Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran agrupadas por mes y año.
```

Resultado esperado: las órdenes se muestran agrupadas por año y mes, con el total de órdenes, planchas y peso.

Analogía final

Las consultas de agregación en una acería son como los informes que el jefe de planta pide al archivo central. En lugar de revisar todas las carpetas una por una, el jefe pide un resumen: cuántas órdenes hay, cuánto pesan las planchas, cuál es la fecha más antigua. El archivo central calcula el resumen y lo entrega. Count es como contar las carpetas. Sum es como sumar los pesos. Average es como calcular el promedio. Min y Max son como encontrar el más ligero y el más pesado. GroupBy es como agrupar las carpetas por cliente o por estado. El filtro Having es como pedir solo los grupos que cumplen una condición. Las agregaciones sobre colecciones vacías son como pedir un promedio cuando no hay datos: hay que tener cuidado porque puede dar error. DefaultIfEmpty es como decir "si no hay datos, devuelve cero". Así funcionan las agregaciones en EF Core: se calculan en el servidor, se devuelven resúmenes y se evita transferir todas las filas.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `ResumenPorClienteDto` y `ResumenPorEstadoDto`.

Añadido los métodos de agregación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `AgregacionesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones de conteos, Any, All, Sum, Average, Min, Max, fechas y resúmenes agrupados.

Analizado el SQL generado por una agregación agrupada.

Diagnosticado el error de Average sobre colección vacía.

Creado el DTO ResumenMensualDto y su método.

| Conexión con el siguiente punto

En este punto se han añadido consultas de agregación al proyecto AceriaData, incluyendo Count, Sum, Average, Min, Max, Any, All y GroupBy. Se ha analizado el SQL generado por las agregaciones y se ha comprobado que EF Core traduce las funciones a SQL. En el siguiente punto se estudiarán las agrupaciones con proyección en detalle, incluyendo GroupBy con múltiples claves, filtros Having y proyecciones de grupos.

Punto 3.6 - Agrupaciones con proyección

Práctica

Ejercicio: Añadir agrupaciones con proyección al repositorio de órdenes del proyecto AceriaData. Crear métodos que agrupen por cliente, por estado, por mes y por cliente y estado. Incluir proyecciones con colecciones internas, filtros Having y agregaciones múltiples. Crear un caso de uso que ejecute estas agrupaciones y analice el SQL generado.

Contexto del proyecto: En el punto 3.5 se añadieron consultas de agregación con Count, Sum, Average, Min, Max y GroupBy. En este punto se profundiza en las agrupaciones con proyección, incluyendo agrupaciones por múltiples claves, filtros Having y proyecciones de grupos. Estas agrupaciones se usarán en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO ResumenPorClienteConOrdenesDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteConOrdenesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteConOrdenesDto
{
    public string Cliente { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public List<OrdenResumenDto> Ordenes { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteConOrdenesDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 7: public List Ordenes { get; set; } = new(); → colección de órdenes.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

Paso 3: Crear el DTO ResumenPorClienteYEstadoDto

Crear el archivo src/AceriaData.Application/Dtos/ResumenPorClienteYEstadoDto.cs:

```
namespace AceriaData.Application.Dtos;

public class ResumenPorClienteYEstadoDto
{
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public int TotalOrdenes { get; set; }
    public decimal PesoTotal { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenPorClienteYEstadoDto → declara el DTO.

Línea 5: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 6: public string Estado { get; set; } = string.Empty; → estado.

Línea 7: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 8: public decimal PesoTotal { get; set; } → peso total.

Error común: si el DTO usa int para el peso total y el valor real es decimal, se produce una pérdida de precisión. Se debe usar decimal para los pesos.

| Paso 4: Añadir los métodos de agrupación a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
    List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
    List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
    List<OrdenCompletaDto> ObtenerOrdenesCompletas();

    int ContarOrdenes();
    int ContarOrdenesPorEstado(string estado);
    bool ExisteAlgunaOrden();
    bool TodasLasOrdenesPendientes();
    decimal ObtenerPesoTotalDePlanchas();
    double ObtenerPesoPromedioDePlanchas();
    decimal ObtenerPesoMinimoDePlanchas();
    decimal ObtenerPesoMaximoDePlanchas();
    DateTime ObtenerFechaMasAntigua();
    DateTime ObtenerFechaMasReciente();

    List<ResumenPorClienteDto> ObtenerResumenPorCliente();
    List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
    List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();
    List<ResumenMensualDto> ObtenerResumenMensual();

    List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado();
    List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro();
    List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 8: `OrdenFabricacion? ObtenerPorId(int id);` → método que obtiene por Id.

Línea 9: `List ObtenerTodas();` → método que obtiene todas.

Línea 10: `IQueryable ObtenerQueryable();` → método que devuelve IQueryable.

Línea 11: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → método que obtiene por número.

Línea 12: `List ObtenerPendientesPorCliente(string cliente);` → método que obtiene las pendientes de un cliente.

Línea 13: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → método que obtiene por estado ordenadas.

Línea 14: `List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);` → método que obtiene por rango de fechas.

Línea 15: `List ObtenerPorClienteOrdenadas(string cliente);` → método que obtiene por cliente ordenadas.

Línea 16: `List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);` → método que combina cliente y rango.

Línea 18: `List ObtenerClientesUnicos();` → método que obtiene los clientes únicos.

Línea 19: `List ObtenerResumenes();` → método que obtiene los resúmenes.

Línea 20: `List ObtenerResumenesPorEstado(string estado);` → método que obtiene los resúmenes por estado.

Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales.

Línea 22: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.

Línea 23: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.

Línea 24: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.

Línea 25: `List ObtenerOrdenesCompletas();` → método que obtiene las órdenes completas.

Línea 27: `int ContarOrdenes();` → método que cuenta las órdenes.

Línea 28: `int ContarOrdenesPorEstado(string estado);` → método que cuenta las órdenes por estado.

Línea 29: `bool ExisteAlgunaOrden();` → método que comprueba si existe alguna orden.

Línea 30: `bool TodasLasOrdenesPendientes();` → método que comprueba si todas las órdenes están pendientes.

Línea 31: `decimal ObtenerPesoTotalDePlanchas();` → método que obtiene el peso total.

Línea 32: `double ObtenerPesoPromedioDePlanchas();` → método que obtiene el peso promedio.

Línea 33: `decimal ObtenerPesoMinimoDePlanchas();` → método que obtiene el peso mínimo.

Línea 34: `decimal ObtenerPesoMaximoDePlanchas();` → método que obtiene el peso máximo.

Línea 35: `DateTime ObtenerFechaMasAntigua();` → método que obtiene la fecha más antigua.

Línea 36: `DateTime ObtenerFechaMasReciente();` → método que obtiene la fecha más reciente.

Línea 38: `List ObtenerResumenPorCliente();` → método que obtiene el resumen por cliente.

Línea 39: `List ObtenerResumenPorEstado();` → método que obtiene el resumen por estado.

Línea 40: `List ObtenerClientesConMasDeUnaOrden();` → método que obtiene los clientes con más de una orden.

Línea 41: `List ObtenerResumenMensual();` → método que obtiene el resumen mensual.

Línea 43: `List ObtenerResumenPorClienteConOrdenes();` → método que obtiene el resumen por cliente con órdenes.

Línea 44: `List ObtenerResumenPorClienteYEstado();` → método que obtiene el resumen por cliente y estado.

Línea 45: `List ObtenerResumenPorClienteYEstadoConFiltro();` → método que obtiene el resumen por cliente y estado con filtro Having.

Línea 46: `List ObtenerResumenPorClienteConMultiplesAgregaciones();` → método que obtiene el resumen por cliente con múltiples agregaciones.

Línea 48: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 49: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

■ Paso 5: Implementar los métodos de agrupación en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteConOrdenesDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            Ordenes = g.Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            }).ToList()
        })
        .OrderBy(r => r.Cliente)
        .ToList();
}

public List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.Cliente, o.Estado })
        .Select(g => new ResumenPorClienteYEstadoDto
        {
            Cliente = g.Key.Cliente,
            Estado = g.Key.Estado,
            TotalOrdenes = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        })
        .OrderBy(r => r.Cliente)
        .ThenBy(r => r.Estado)
        .ToList();
}

public List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro()
{

```



```

        return _context.OrdenesFabricacion
            .GroupBy(o => new { o.Cliente, o.Estado })
            .Where(g => g.Count() > 1)
            .Select(g => new ResumenPorClienteYEstadoDto
            {
                Cliente = g.Key.Cliente,
                Estado = g.Key.Estado,
                TotalOrdenes = g.Count(),
                PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
            })
            .OrderBy(r => r.Cliente)
            .ThenBy(r => r.Estado)
            .ToList();
    }

    public List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones()
    {
        return _context.OrdenesFabricacion
            .GroupBy(o => o.Cliente)
            .Select(g => new ResumenPorClienteDto
            {
                Cliente = g.Key,
                TotalOrdenes = g.Count(),
                TotalPlanchas = g.Sum(o => o.Planchas.Count()),
                PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)),
                FechaMasReciente = g.Max(o => o.FechaCreacion)
            })
            .OrderBy(r => r.Cliente)
            .ToList();
    }
}

```

Línea 1: `public List ObtenerResumenPorClienteConOrdenes()` → declara el método que obtiene el resumen por cliente con órdenes.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.GroupBy(o => o.Cliente)` → agrupa por cliente.

Línea 5: `.Select(g => new ResumenPorClienteConOrdenesDto` → proyecta cada grupo al DTO.

Línea 6: `Cliente = g.Key,` → asigna el cliente.

Línea 7: `TotalOrdenes = g.Count(),` → cuenta las órdenes del grupo.

Línea 8: `Ordenes = g.Select(o => new OrdenResumenDto` → proyecta la colección interna de órdenes.

Línea 9: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 10: `Cliente = o.Cliente,` → asigna el cliente.

Línea 11: `Estado = o.Estado,` → asigna el estado.

Línea 12: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 13: `}).ToList()` → materializa la colección interna.

Línea 14: `})` → cierra la proyección.

Línea 15: `.OrderBy(r => r.Cliente)` → ordena por cliente.

Línea 16: `.ToList();` → materializa la consulta.

Línea 19: `public List ObtenerResumenPorClienteYEstado()` → declara el método que agrupa por cliente y estado.

Línea 21: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 22: `.GroupBy(o => new { o.Cliente, o.Estado })` → agrupa por cliente y estado.

Línea 23: `.Select(g => new ResumenPorClienteYEstadoDto` → proyecta cada grupo al DTO.

Línea 24: `Cliente = g.Key.Cliente,` → asigna el cliente.

Línea 25: Estado = g.Key.Estado, → asigna el estado.

Línea 26: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 27: PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)) → suma el peso de las planchas del grupo.

Línea 28: }) → cierra la proyección.

Línea 29: .OrderBy(r => r.Cliente) → ordena por cliente.

Línea 30: .ThenBy(r => r.Estado) → ordena por estado.

Línea 31: .ToList(); → materializa la consulta.

Línea 34: public List ObtenerResumenPorClienteYEstadoConFiltro() → declara el método que agrupa por cliente y estado con filtro Having.

Línea 36: return _context.OrdenesFabricacion → inicia la consulta.

Línea 37: .GroupBy(o => new { o.Cliente, o.Estado }) → agrupa por cliente y estado.

Línea 38: .Where(g => g.Count() > 1) → filtra los grupos con más de una orden. Se traduce a HAVING.

Línea 39: .Select(g => new ResumenPorClienteYEstadoDto → proyecta cada grupo al DTO.

Línea 40: Cliente = g.Key.Cliente, → asigna el cliente.

Línea 41: Estado = g.Key.Estado, → asigna el estado.

Línea 42: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 43: PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)) → suma el peso de las planchas del grupo.

Línea 44: }) → cierra la proyección.

Línea 45: .OrderBy(r => r.Cliente) → ordena por cliente.

Línea 46: .ThenBy(r => r.Estado) → ordena por estado.

Línea 47: .ToList(); → materializa la consulta.

Línea 50: public List ObtenerResumenPorClienteConMultiplesAgregaciones() → declara el método que agrupa por cliente con múltiples agregaciones.

Línea 52: return _context.OrdenesFabricacion → inicia la consulta.

Línea 53: .GroupBy(o => o.Cliente) → agrupa por cliente.

Línea 54: .Select(g => new ResumenPorClienteDto → proyecta cada grupo al DTO.

Línea 55: Cliente = g.Key, → asigna el cliente.

Línea 56: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 57: TotalPlanchas = g.Sum(o => o.Planchas.Count()), → suma las planchas del grupo.

Línea 58: PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso)), → suma el peso de las planchas del grupo.

Línea 59: FechaMasReciente = g.Max(o => o.FechaCreacion) → obtiene la fecha más reciente del grupo.

Línea 60: }) → cierra la proyección.

Línea 61: .OrderBy(r => r.Cliente) → ordena por cliente.

Línea 62: .ToList(); → materializa la consulta.

Error común: si se proyecta una colección interna dentro de un GroupBy, EF Core puede ejecutar una consulta adicional por cada grupo. Se debe revisar el SQL generado y considerar alternativas como SelectMany o cargar los datos en una sola consulta.

| Paso 6: Crear el caso de uso de agrupaciones con proyección

Crear el archivo src/AceriaData.Application/UseCases/AgrupacionesUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AgrupacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AgrupacionesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");

        DemostrarResumenPorClienteConOrdenes();
        DemostrarResumenPorClienteYEstado();
        DemostrarResumenPorClienteYEstadoConFiltro();
        DemostrarResumenConMultiplesAgregaciones();
        DemostrarSqlAgrupacionMultiple();
    }

    private void DemostrarResumenPorClienteConOrdenes()
    {
        Console.WriteLine("\n--- Resumen por cliente con órdenes ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | Total: {resumen.TotalOrdenes}");
            foreach (var orden in resumen.Ordenes)
            {
                Console.WriteLine($" {orden.NumeroOrden} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");
            }
        }
    }

    private void DemostrarResumenPorClienteYEstado()
    {
        Console.WriteLine("\n--- Resumen por cliente y estado ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");
        }
    }

    private void DemostrarResumenPorClienteYEstadoConFiltro()
    {
        Console.WriteLine("\n--- Resumen por cliente y estado (con más de una orden) ---");

        var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();
        foreach (var resumen in resmenes)
        {
            Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");
        }
    }
}
```

```

    }
}

private void DemostrarResumenConMultiplesAgregaciones()
{
    Console.WriteLine("\n--- Resumen por cliente con múltiples agregaciones ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteConMultiplesAgregaciones();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas:
{resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/
yyyy}");
    }
}

private void DemostrarSqlAgrupacionMultiple()
{
    Console.WriteLine("\n--- SQL generado por una agrupación por múltiples claves ---");

    var consulta = _unidad.Ordenes.ObtenerQueryable()
        .GroupBy(o => new { o.Cliente, o.Estado })
        .Select(g => new
        {
            Cliente = g.Key.Cliente,
            Estado = g.Key.Estado,
            Total = g.Count(),
            PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
        });

    var sql = consulta.ToQueryString();
    Console.WriteLine(sql);
}
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class AgrupacionesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public AgrupacionesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ==="); → muestra la cabecera.

Línea 18: DemostrarResumenPorClienteConOrdenes(); → llama al método de resumen por cliente con órdenes.

Línea 19: DemostrarResumenPorClienteYEstado(); → llama al método de resumen por cliente y estado.

Línea 20: DemostrarResumenPorClienteYEstadoConFiltro(); → llama al método de resumen con filtro Having.

Línea 21: DemostrarResumenConMultiplesAgregaciones(); → llama al método de resumen con múltiples agregaciones.

Línea 22: DemostrarSqlAgrupacionMultiple(); → llama al método de SQL de agrupación múltiple.

Línea 25: private void DemostrarResumenPorClienteConOrdenes() → declara el método.

Línea 27: Console.WriteLine("\n--- Resumen por cliente con órdenes ---"); → muestra la cabecera.

Línea 29: var resmenes = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes(); → llama al método del repositorio.

Línea 30: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 32: `Console.WriteLine($" {resumen.Cliente} | Total: {resumen.TotalOrdenes}");` → muestra el cliente y el total.

Línea 33: `foreach (var orden in resumen.Ordenes)` → itera sobre las órdenes.

Línea 35: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 39: `private void DemostrarResumenPorClienteYEstado()` → declara el método.

Línea 41: `Console.WriteLine("\n--- Resumen por cliente y estado ---");` → muestra la cabecera.

Línea 43: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();` → llama al método del repositorio.

Línea 44: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 46: `Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 50: `private void DemostrarResumenPorClienteYEstadoConFiltro()` → declara el método.

Línea 52: `Console.WriteLine("\n--- Resumen por cliente y estado (con más de una orden) ---");` → muestra la cabecera.

Línea 54: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();` → llama al método del repositorio.

Línea 55: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 57: `Console.WriteLine($" {resumen.Cliente} | {resumen.Estado} | Órdenes: {resumen.TotalOrdenes} | Peso: {resumen.PesoTotal} kg");` → muestra los datos.

Línea 61: `private void DemostrarResumenConMultiplesAgregaciones()` → declara el método.

Línea 63: `Console.WriteLine("\n--- Resumen por cliente con múltiples agregaciones ---");` → muestra la cabecera.

Línea 65: `var resúmenes = _unidad.Ordenes.ObtenerResumenPorClienteConMultiplesAgregaciones();` → llama al método del repositorio.

Línea 66: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 68: `Console.WriteLine($" {resumen.Cliente} | Órdenes: {resumen.TotalOrdenes} | Planchas: {resumen.TotalPlanchas} | Peso: {resumen.PesoTotal} kg | Última: {resumen.FechaMasReciente:dd/MM/yyyy}");` → muestra los datos.

Línea 72: `private void DemostrarSqlAgrupacionMultiple()` → declara el método.

Línea 74: `Console.WriteLine("\n--- SQL generado por una agrupación por múltiples claves ---");` → muestra la cabecera.

Línea 76: `var consulta = _unidad.Ordenes.ObtenerQueryable();` → inicia la consulta.

Línea 77: `.GroupBy(o => new { o.Cliente, o.Estado })` → agrupa por cliente y estado.

Línea 78: `.Select(g => new` → proyecta a un tipo anónimo.

Línea 79: `Cliente = g.Key.Cliente,` → incluye el cliente.

Línea 80: `Estado = g.Key.Estado,` → incluye el estado.

Línea 81: `Total = g.Count(),` → cuenta las órdenes.

Línea 82: `PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))` → suma el peso.

Línea 83: `});` → cierra la proyección.

Línea 85: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 86: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<AgrupacionesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 9: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    var orden4 = new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) };

    context OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `var orden4 = new OrdenFabricacion { ... };` → crea la cuarta orden.

Línea 12: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3, orden4);` → registra las órdenes.

Línea 13: `context.SaveChanges();` → inserta las órdenes.

Línea 15: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 16: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 17: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 18: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas.

Línea 21: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada agrupación. Se observan los resúmenes por cliente con órdenes, por cliente y estado, con filtro Having, con múltiples agregaciones y el SQL generado por una agrupación por múltiples claves.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== AGRUPACIONES CON PROYECCIÓN ===

--- Resumen por cliente con órdenes ---
Constructora del Norte | Total: 3
  OF-2024-0001 | Pendiente | 15/01/2024
  OF-2024-0003 | EnProceso | 10/03/2024
  OF-2024-0004 | Pendiente | 05/04/2024
Constructora del Sur | Total: 1
  OF-2024-0002 | Pendiente | 20/02/2024

--- Resumen por cliente y estado ---
Constructora del Norte | EnProceso | Órdenes: 1 | Peso: 450.0 kg
Constructora del Norte | Pendiente | Órdenes: 2 | Peso: 651.3 kg
Constructora del Sur | Pendiente | Órdenes: 1 | Peso: 125.6 kg

--- Resumen por cliente y estado (con más de una orden) ---
Constructora del Norte | Pendiente | Órdenes: 2 | Peso: 651.3 kg

--- Resumen por cliente con múltiples agregaciones ---
Constructora del Norte | Órdenes: 3 | Planchas: 3 | Peso: 1101.3 kg | Última: 05/04/2024
Constructora del Sur | Órdenes: 1 | Planchas: 1 | Peso: 125.6 kg | Última: 20/02/2024

--- SQL generado por una agrupación por múltiples claves ---
SELECT [o].[Cliente], [o].[Estado], COUNT(*) AS [Total], COALESCE(SUM((
  SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
)), 0.0) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
GROUP BY [o].[Cliente], [o].[Estado]
```

La primera sección muestra el resumen por cliente con órdenes. La segunda sección muestra el resumen por cliente y estado. La tercera sección muestra el resumen con filtro Having. La cuarta sección muestra el resumen con múltiples agregaciones. La quinta sección muestra el SQL generado por una agrupación por múltiples claves.

Observaciones: el SQL generado incluye COUNT(*), SUM, GROUP BY con dos columnas y el filtro global de Soft Delete. La colección interna de órdenes no se traduce a SQL: EF Core la materializa después de ejecutar la consulta. Esto demuestra que las colecciones internas dentro de un GroupBy se cargan en memoria.

Paso 12: Diagnosticar un error común

Modificar el método ObtenerResumenPorClienteConOrdenes para proyectar la colección interna sin ToList:

```
csharp
public List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => o.Cliente)
        .Select(g => new ResumenPorClienteConOrdenesDto
        {
            Cliente = g.Key,
            TotalOrdenes = g.Count(),
            Ordenes = g.Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
```



```

        FechaCreacion = o.FechaCreacion
    })
})
.OrderBy(r => r.Cliente)
.ToList();
}

```

Resultado esperado: el compilador lanza un error porque la propiedad Ordenes es de tipo List y la proyección devuelve IEnumerable. Se debe llamar a ToList para materializar la colección.

Solución: añadir .ToList() dentro de la proyección.

```

Ordenes = g.Select(o => new OrdenResumenDto
{
    NumeroOrden = o.NumeroOrden,
    Cliente = o.Cliente,
    Estado = o.Estado,
    FechaCreacion = o.FechaCreacion
}).ToList()

```

Resultado esperado con la solución: la consulta compila y la colección se materializa correctamente.

Errores comunes del ejercicio

Error	Causa	Solución
Colección interna sin ToList de la proyección	Se olvidó materializar la colección	Añadir .ToList() dentro de la proyección
Where antes del GroupBy para HAVING	Se aplica el filtro a las filas	Usar Where después del GroupBy
Propiedad no agrupada del grupo o funciones de agregación	Se proyecta una propiedad que no está en el GroupBy	Usar la clave
N+1 en agrupaciones	Se proyecta una colección interna	Revisar el SQL generado y considerar alternativas
SelectMany sin proyección	Se aplanan sin proyectar	Añadir una proyección clara
Agrupación anidada	EF Core ejecuta múltiples consultas	Cargar los datos en una sola consulta

Reto resuelto: Agrupación por mes con proyección de órdenes

Reto: Crear un método en el repositorio que agrupe las órdenes por mes y año, y proyecte cada grupo a un DTO con el año, el mes, el total de órdenes y la lista de números de orden. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO ResumenMensualConOrdenesDto:

```

csharp
namespace AceriaData.Application.Dtos;

public class ResumenMensualConOrdenesDto
{
    public int Anio { get; set; }
    public int Mes { get; set; }
    public int TotalOrdenes { get; set; }
    public List<string> NumerosOrden { get; set; } = new();
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class ResumenMensualConOrdenesDto → declara el DTO.

Línea 5: public int Anio { get; set; } → año.

Línea 6: public int Mes { get; set; } → mes.

Línea 7: public int TotalOrdenes { get; set; } → total de órdenes.

Línea 8: public List NumerosOrden { get; set; } = new(); → lista de números de orden.

| Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```

List<ResumenMensualConOrdenesDto> ObtenerResumenMensualConOrdenes();

```

Línea 1: List ObtenerResumenMensualConOrdenes(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<ResumenMensualConOrdenesDto> ObtenerResumenMensualConOrdenes()
{
    return _context.OrdenesFabricacion
        .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
        .Select(g => new ResumenMensualConOrdenesDto
        {
            Anio = g.Key.Year,
            Mes = g.Key.Month,
            TotalOrdenes = g.Count(),
            NumerosOrden = g.Select(o => o.NumeroOrden).ToList()
        })
        .OrderBy(r => r.Anio)
        .ThenBy(r => r.Mes)
        .ToList();
}
```

Línea 1: public List ObtenerResumenMensualConOrdenes() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month }) → agrupa por año y mes.

Línea 5: .Select(g => new ResumenMensualConOrdenesDto → proyecta cada grupo al DTO.

Línea 6: Anio = g.Key.Year, → asigna el año.

Línea 7: Mes = g.Key.Month, → asigna el mes.

Línea 8: TotalOrdenes = g.Count(), → cuenta las órdenes del grupo.

Línea 9: NumerosOrden = g.Select(o => o.NumeroOrden).ToList() → proyecta la colección interna de números de orden.

Línea 10: }) → cierra la proyección.

Línea 11: .OrderBy(r => r.Anio) → ordena por año.

Línea 12: .ThenBy(r => r.Mes) → ordena por mes.

Línea 13: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenMensualConOrdenes()
{
    Console.WriteLine("\n--- Resumen mensual con órdenes ---");

    var resmenes = _unidad.Ordenes.ObtenerResumenMensualConOrdenes();
    foreach (var resumen in resmenes)
    {
        Console.WriteLine($" {resumen.Anio}-{resumen.Mes:D2} | Total: {resumen.TotalOrdenes}");
        foreach (var numero in resumen.NumerosOrden)
        {
            Console.WriteLine($" {numero}");
        }
    }
}
```

Línea 1: private void DemostrarResumenMensualConOrdenes() → declara el método.

Línea 3: Console.WriteLine("\n--- Resumen mensual con órdenes ---"); → muestra la cabecera.

Línea 5: var resmenes = _unidad.Ordenes.ObtenerResumenMensualConOrdenes(); → llama al método del repositorio.

Línea 6: `foreach (var resumen in resúmenes)` → itera sobre los resúmenes.

Línea 8: `Console.WriteLine($" {resumen.Año}-{resumen.Mes:D2} | Total: {resumen.TotalOrdenes}");` → muestra el año, mes y total.

Línea 9: `foreach (var numero in resumen.NumerosOrden)` → itera sobre los números.

Línea 11: `Console.WriteLine($" {numero}");` → muestra el número.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarResumenMensualConOrdenes();
```

Paso 6: Ejecutar `dotnet run` y verificar que las órdenes se muestran agrupadas por mes y año con la lista de números.

Resultado esperado: las órdenes se muestran agrupadas por año y mes, con el total de órdenes y la lista de números de orden.

Analogía final

Las agrupaciones con proyección en una acería son como los informes agrupados que el jefe de planta pide al archivo central. En lugar de una lista plana de órdenes, el jefe pide un informe agrupado por cliente: para cada cliente, cuántas órdenes tiene y cuáles son. El archivo central agrupa las carpetas por cliente, cuenta las órdenes y prepara una lista con los números de cada orden. Proyectar una colección interna dentro de un grupo es como adjuntar la lista de órdenes al informe del cliente. El filtro `Having` es como pedir solo los clientes que tienen más de una orden. La agrupación por múltiples claves es como agrupar por cliente y estado a la vez. `SelectMany` es como aplanar los grupos en una lista plana. Así funcionan las agrupaciones con proyección en EF Core: se agrupan los datos en el servidor, se proyectan los resúmenes y se adjuntan las colecciones internas cuando es necesario. El resultado es un informe estructurado y eficiente.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `ResumenPorClienteConOrdenesDto` y `ResumenPorClienteYEstadoDto`.

Añadido los métodos de agrupación a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `AgrupacionesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones de resumen por cliente con órdenes, por cliente y estado, con filtro `Having` y con múltiples agregaciones.

Analizado el SQL generado por una agrupación por múltiples claves.

Diagnosticado el error de no materializar la colección interna.

Creado el DTO `ResumenMensualConOrdenesDto` y su método.

Conexión con el siguiente punto

En este punto se han añadido agrupaciones con proyección al proyecto `AceriaData`, incluyendo agrupaciones por múltiples claves, filtros `Having`, proyecciones de colecciones internas y agrupaciones con múltiples agregaciones. Se ha analizado el SQL generado y se ha comprobado que las colecciones internas se materializan en memoria después de la agrupación. En el siguiente punto se estudiarán los `joins` y la navegación en consultas, incluyendo `Join`, `GroupJoin` y la navegación por propiedades de navegación.

Punto 3.7 - Joins y navegación en consultas

Práctica

Ejercicio: Añadir consultas con joins y navegación al repositorio de órdenes del proyecto AceriaData. Crear métodos que combinen órdenes con planchas, órdenes con detalle, órdenes con aleaciones y órdenes con múltiples entidades. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.6 se añadieron agrupaciones con proyección, incluyendo colecciones internas y filtros Having. En este punto se añaden consultas con joins y navegación, incluyendo Join, SelectMany, DefaultIfEmpty y Include. Estas consultas se usarán en el punto 3.8 para la carga Eager y en el punto 3.9 para la carga Lazy.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO OrdenConPlanchasYDetalleDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConPlanchasYDetalleDto.cs:

```
namespace AceriaData.Application.Dtos;

public class OrdenConPlanchasYDetalleDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConPlanchasYDetalleDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → estado.

Línea 8: public List Planchas { get; set; } = new(); → colección de planchas.

Línea 9: public DetalleDto? Detalle { get; set; } → detalle opcional.

Error común: si el DTO no marca la propiedad Detalle como anulable, el compilador lanza un warning. Se debe marcar como ? si puede ser null.

Paso 3: Crear el DTO OrdenConAleacionesDto

Crear el archivo src/AceriaData.Application/Dtos/OrdenConAleacionesDto.cs:

```

namespace AceriaData.Application.Dtos;

public class OrdenConAleacionesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public List<AleacionDto> Aleaciones { get; set; } = new();
}

public class AleacionDto
{
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public decimal PorcentajeCarbono { get; set; }
    public decimal PorcentajeManganeso { get; set; }
    public decimal CantidadUtilizada { get; set; }
}

```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenConAleacionesDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 7: public List Aleaciones { get; set; } = new(); → colección de aleaciones.

Línea 10: public class AleacionDto → declara el DTO de aleación.

Línea 12: public string Nombre { get; set; } = string.Empty; → nombre.

Línea 13: public string Codigo { get; set; } = string.Empty; → código.

Línea 14: public decimal PorcentajeCarbono { get; set; } → porcentaje de carbono.

Línea 15: public decimal PorcentajeManganeso { get; set; } → porcentaje de manganeso.

Línea 16: public decimal CantidadUtilizada { get; set; } → cantidad utilizada.

Error común: si la colección no se inicializa con new(), se produce una NullReferenceException al intentar añadir elementos.

| Paso 4: Añadir los métodos de join a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```

using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    IQueryable<OrdenFabricacion> ObtenerQueryable();
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
    List<OrdenFabricacion> ObtenerPorClienteOrdenadas(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta);

    List<string> ObtenerClientesUnicos();
    List<OrdenResumenDto> ObtenerResumenes();
    List<OrdenResumenDto> ObtenerResumenesPorEstado(string estado);
    List<OrdenConTotalesDto> ObtenerOrdenesConTotales();
    List<OrdenConPlanchasDto> ObtenerOrdenesConPlanchas();
}

```

```

List<OrdenConDetalleDto> ObtenerOrdenesConDetalle();
List<OrdenConTotalesDto> ObtenerOrdenesConTotalesDetallados();
List<OrdenCompletaDto> ObtenerOrdenesCompletas();

int ContarOrdenes();
int ContarOrdenesPorEstado(string estado);
bool ExisteAlgunaOrden();
bool TodasLasOrdenesPendientes();
decimal ObtenerPesoTotalDePlanchas();
double ObtenerPesoPromedioDePlanchas();
decimal ObtenerPesoMinimoDePlanchas();
decimal ObtenerPesoMaximoDePlanchas();
DateTime ObtenerFechaMasAntigua();
DateTime ObtenerFechaMasReciente();

List<ResumenPorClienteDto> ObtenerResumenPorCliente();
List<ResumenPorEstadoDto> ObtenerResumenPorEstado();
List<ResumenPorClienteDto> ObtenerClientesConMasDeUnaOrden();
List<ResumenMensualDto> ObtenerResumenMensual();
List<ResumenPorClienteConOrdenesDto> ObtenerResumenPorClienteConOrdenes();
List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstado();
List<ResumenPorClienteYEstadoDto> ObtenerResumenPorClienteYEstadoConFiltro();
List<ResumenPorClienteDto> ObtenerResumenPorClienteConMultiplesAgregaciones();

List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle();
List<OrdenConAleacionesDto> ObtenerOrdenesConAleaciones();
List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();

void Agregar(OrdenFabricacion orden);
void Eliminar(OrdenFabricacion orden);
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: OrdenFabricacion? ObtenerPorId(int id); → método que obtiene por Id.

Línea 9: List ObtenerTodas(); → método que obtiene todas.

Línea 10: IQueryable ObtenerQueryable(); → método que devuelve IQueryable.

Línea 11: OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden); → método que obtiene por número.

Línea 12: List ObtenerPendientesPorCliente(string cliente); → método que obtiene las pendientes de un cliente.

Línea 13: List ObtenerPorEstadoOrdenadasPorFecha(string estado); → método que obtiene por estado ordenadas.

Línea 14: List ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta); → método que obtiene por rango de fechas.

Línea 15: List ObtenerPorClienteOrdenadas(string cliente); → método que obtiene por cliente ordenadas.

Línea 16: List ObtenerPorClienteYRangoDeFechas(string cliente, DateTime desde, DateTime hasta); → método que combina cliente y rango.

Línea 18: List ObtenerClientesUnicos(); → método que obtiene los clientes únicos.

Línea 19: List ObtenerResumenes(); → método que obtiene los resúmenes.

Línea 20: List ObtenerResumenesPorEstado(string estado); → método que obtiene los resúmenes por estado.

- Línea 21: `List ObtenerOrdenesConTotales();` → método que obtiene las órdenes con totales.
- Línea 22: `List ObtenerOrdenesConPlanchas();` → método que obtiene las órdenes con planchas.
- Línea 23: `List ObtenerOrdenesConDetalle();` → método que obtiene las órdenes con detalle.
- Línea 24: `List ObtenerOrdenesConTotalesDetallados();` → método que obtiene las órdenes con totales detallados.
- Línea 25: `List ObtenerOrdenesCompletas();` → método que obtiene las órdenes completas.
- Línea 27: `int ContarOrdenes();` → método que cuenta las órdenes.
- Línea 28: `int ContarOrdenesPorEstado(string estado);` → método que cuenta las órdenes por estado.
- Línea 29: `bool ExisteAlgunaOrden();` → método que comprueba si existe alguna orden.
- Línea 30: `bool TodasLasOrdenesPendientes();` → método que comprueba si todas las órdenes están pendientes.
- Línea 31: `decimal ObtenerPesoTotalDePlanchas();` → método que obtiene el peso total.
- Línea 32: `double ObtenerPesoPromedioDePlanchas();` → método que obtiene el peso promedio.
- Línea 33: `decimal ObtenerPesoMinimoDePlanchas();` → método que obtiene el peso mínimo.
- Línea 34: `decimal ObtenerPesoMaximoDePlanchas();` → método que obtiene el peso máximo.
- Línea 35: `DateTime ObtenerFechaMasAntigua();` → método que obtiene la fecha más antigua.
- Línea 36: `DateTime ObtenerFechaMasReciente();` → método que obtiene la fecha más reciente.
- Línea 38: `List ObtenerResumenPorCliente();` → método que obtiene el resumen por cliente.
- Línea 39: `List ObtenerResumenPorEstado();` → método que obtiene el resumen por estado.
- Línea 40: `List ObtenerClientesConMasDeUnaOrden();` → método que obtiene los clientes con más de una orden.
- Línea 41: `List ObtenerResumenMensual();` → método que obtiene el resumen mensual.
- Línea 42: `List ObtenerResumenPorClienteConOrdenes();` → método que obtiene el resumen por cliente con órdenes.
- Línea 43: `List ObtenerResumenPorClienteYEstado();` → método que obtiene el resumen por cliente y estado.
- Línea 44: `List ObtenerResumenPorClienteYEstadoConFiltro();` → método que obtiene el resumen con filtro.
- Línea 45: `List ObtenerResumenPorClienteConMultiplesAgregaciones();` → método que obtiene el resumen con múltiples agregaciones.
- Línea 47: `List ObtenerOrdenesConPlanchasYDetalle();` → método que combina órdenes con planchas y detalle.
- Línea 48: `List ObtenerOrdenesConAleaciones();` → método que combina órdenes con aleaciones.
- Línea 49: `List ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();` → método que usa Join explícito.
- Línea 51: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.
- Línea 52: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

■ Paso 5: Implementar los métodos de join en el repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:


```

public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}

public List<OrdenConAleacionesDto> ObtenerOrdenesConAleaciones()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConAleacionesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto
            {
                Nombre = oa.Aleacion.Nombre,
                Codigo = oa.Aleacion.Codigo,
                PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono,
                PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso,
                CantidadUtilizada = oa.CantidadUtilizada
            }).ToList()
        })
        .ToList();
}

public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleConJoinExplicito()
{
    return _context.OrdenesFabricacion
        .GroupJoin(
            _context.PlanchasAcero,
            o => o.Id,
            p => p.OrdenId,
            (o, planchas) => new { Orden = o, Planchas = planchas })
        .GroupJoin(
            _context.DetallesOrden,
            op => op.Orden.Id,
            d => d.OrdenId,
            (op, detalles) => new { op.Orden, op.Planchas, Detalle = detalles.FirstOrDefault() })
        .OrderBy(x => x.Orden.NumeroOrden)
        .Select(x => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = x.Orden.NumeroOrden,
            Cliente = x.Orden.Cliente,
            Estado = x.Orden.Estado,
            Planchas = x.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,

```

```

        Largo = p.Largo,
        Peso = p.Peso,
        Activa = p.Activa
    }).ToList(),
    Detalle = x.Detalle == null ? null : new DetalleDto
    {
        ComposicionQuimica = x.Detalle.ComposicionQuimica,
        TemperaturaColada = x.Detalle.TemperaturaColada,
        Notas = x.Detalle.Notas
    }
})
.ToList();
}

```

Línea 1: `public List ObtenerOrdenesConPlanchasYDetalle()` → declara el método que combina órdenes con planchas y detalle.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.Select(o => new OrdenConPlanchasYDetalleDto` → proyecta al DTO.

Línea 6: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 7: `Cliente = o.Cliente,` → asigna el cliente.

Línea 8: `Estado = o.Estado,` → asigna el estado.

Línea 9: `Planchas = o.Planchas.Select(p => new PlanchaDto` → proyecta la colección de planchas.

Línea 10: `Id = p.Id,` → asigna el Id.

Línea 11: `Espesor = p.Espesor,` → asigna el espesor.

Línea 12: `Ancho = p.Ancho,` → asigna el ancho.

Línea 13: `Largo = p.Largo,` → asigna el largo.

Línea 14: `Peso = p.Peso,` → asigna el peso.

Línea 15: `Activa = p.Activa` → asigna el estado activo.

Línea 16: `}).ToList(),` → materializa la colección de planchas.

Línea 17: `Detalle = o.Detalle == null ? null : new DetalleDto` → comprueba si el detalle existe y lo proyecta.

Línea 18: `ComposicionQuimica = o.Detalle.ComposicionQuimica,` → asigna la composición química.

Línea 19: `TemperaturaColada = o.Detalle.TemperaturaColada,` → asigna la temperatura.

Línea 20: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 21: `}` → cierra la proyección del detalle.

Línea 22: `})` → cierra la proyección.

Línea 23: `.ToList();` → materializa la consulta.

Línea 26: `public List ObtenerOrdenesConAleaciones()` → declara el método que combina órdenes con aleaciones.

Línea 28: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 29: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 30: `.Select(o => new OrdenConAleacionesDto` → proyecta al DTO.

Línea 31: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 32: `Cliente = o.Cliente, → asigna el cliente.`

Línea 33: `Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto → proyecta la colección de aleaciones.`

Línea 34: `Nombre = oa.Aleacion.Nombre, → asigna el nombre.`

Línea 35: `Codigo = oa.Aleacion.Codigo, → asigna el código.`

Línea 36: `PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono, → asigna el porcentaje de carbono.`

Línea 37: `PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso, → asigna el porcentaje de manganeso.`

Línea 38: `CantidadUtilizada = oa.CantidadUtilizada → asigna la cantidad utilizada.`

Línea 39: `}).ToList() → materializa la colección de aleaciones.`

Línea 40: `}) → cierra la proyección.`

Línea 41: `.ToList(); → materializa la consulta.`

Línea 44: `public List ObtenerOrdenesConPlanchasYDetalleConJoinExplicito() → declara el método con Join explícito.`

Línea 46: `return _context.OrdenesFabricacion → inicia la consulta.`

Línea 47: `.GroupJoin( → primer GroupJoin con planchas.`

Línea 48: `_context.PlanchasAcero, → tabla de planchas.`

Línea 49: `o => o.Id, → clave del lado izquierdo.`

Línea 50: `p => p.OrdenId, → clave del lado derecho.`

Línea 51: `(o, planchas) => new { Orden = o, Planchas = planchas }) → proyecta la combinación.`

Línea 52: `.GroupJoin( → segundo GroupJoin con detalles.`

Línea 53: `_context.DetallesOrden, → tabla de detalles.`

Línea 54: `op => op.Orden.Id, → clave del lado izquierdo.`

Línea 55: `d => d.OrdenId, → clave del lado derecho.`

Línea 56: `(op, detalles) => new { op.Orden, op.Planchas, Detalle = detalles.FirstOrDefault() }) → proyecta la combinación.`

Línea 57: `.OrderBy(x => x.Orden.NumeroOrden) → ordena por número de orden.`

Línea 58: `.Select(x => new OrdenConPlanchasYDetalleDto → proyecta al DTO.`

Línea 59: `NumeroOrden = x.Orden.NumeroOrden, → asigna el número.`

Línea 60: `Cliente = x.Orden.Cliente, → asigna el cliente.`

Línea 61: `Estado = x.Orden.Estado, → asigna el estado.`

Línea 62: `Planchas = x.Planchas.Select(p => new PlanchaDto → proyecta las planchas.`

Línea 69: `}).ToList(), → materializa la colección.`

Línea 70: `Detalle = x.Detalle == null ? null : new DetalleDto → comprueba si el detalle existe.`

Línea 77: `}) → cierra la proyección.`

Línea 78: `.ToList(); → materializa la consulta.`

Error común: si se usa `GroupJoin` sin entender su semántica, el resultado puede ser confuso. Se recomienda usar `SelectMany` con `DefaultIfEmpty` para los `LEFT JOIN`.

Paso 6: Crear el caso de uso de joins

Crear el archivo src/AceriaData.Application/UseCases/joinsUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class JoinsUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public JoinsUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");

        DemostrarOrdenesConPlanchasYDetalle();
        DemostrarOrdenesConAleaciones();
        DemostrarJoinExplicito();
        DemostrarSqlJoin();
    }

    private void DemostrarOrdenesConPlanchasYDetalle()
    {
        Console.WriteLine("\n--- Órdenes con planchas y detalle (navegación por propiedades) ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalle();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
            Console.WriteLine($"   Planchas: {orden.Planchas.Count}");
            var detalle = orden.Detalle == null ? "Sin detalle" :
            $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
            Console.WriteLine($"   Detalle: {detalle}");
        }
    }

    private void DemostrarOrdenesConAleaciones()
    {
        Console.WriteLine("\n--- Órdenes con aleaciones (navegación por propiedades) ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConAleaciones();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente}");
            foreach (var aleacion in orden.Aleaciones)
            {
                Console.WriteLine($"   {aleacion.Nombre} ({aleacion.Codigo}) | C:
{aleacion.PorcentajeCarbono}% | Mn: {aleacion.PorcentajeManganeso}% | Cantidad:
{aleacion.CantidadUtilizada} kg");
            }
        }
    }

    private void DemostrarJoinExplicito()
    {
        Console.WriteLine("\n--- Órdenes con planchas y detalle (Join explícito) ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
            Console.WriteLine($"   Planchas: {orden.Planchas.Count}");
        }
    }

    private void DemostrarSqlJoin()
    {
    }
}
```

```

    {
        Console.WriteLine("\n--- SQL generado por una consulta con navegación ---");

        var consulta = _unidad Ordenes.ObtenerQueryable()
            .OrderBy(o => o.NumeroOrden)
            .Select(o => new
            {
                o.NumeroOrden,
                o.Cliente,
                Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList(),
                Detalle = o.Detalle == null ? null : new { o.Detalle.ComposicionQuimica,
o.Detalle.TemperaturaColada }
            });

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class JoinsUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public JoinsUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== JOINS Y NAVEGACIÓN ==="); → muestra la cabecera.

Línea 18: DemostrarOrdenesConPlanchasYDetalle(); → llama al método de órdenes con planchas y detalle.

Línea 19: DemostrarOrdenesConAleaciones(); → llama al método de órdenes con aleaciones.

Línea 20: DemostrarJoinExplicito(); → llama al método de join explícito.

Línea 21: DemostrarSqlJoin(); → llama al método de SQL de join.

Línea 24: private void DemostrarOrdenesConPlanchasYDetalle() → declara el método.

Línea 26: Console.WriteLine("\n--- Órdenes con planchas y detalle (navegación por propiedades) ---"); → muestra la cabecera.

Línea 28: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalle(); → llama al método del repositorio.

Línea 29: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 31: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos de la orden.

Línea 32: Console.WriteLine(\$" Planchas: {orden.Planchas.Count}"); → muestra el número de planchas.

Línea 33: var detalle = orden.Detalle == null ? "Sin detalle" : \$"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C"; → comprueba si el detalle existe.

Línea 34: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle.

Línea 38: private void DemostrarOrdenesConAleaciones() → declara el método.

Línea 40: Console.WriteLine("\n--- Órdenes con aleaciones (navegación por propiedades) ---"); → muestra la cabecera.

Línea 42: `var ordenes = _unidad Ordenes.ObtenerOrdenesConAleaciones();` → llama al método del repositorio.

Línea 43: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 45: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente}");` → muestra los datos de la orden.

Línea 46: `foreach (var aleacion in orden.Aleaciones)` → itera sobre las aleaciones.

Línea 48: `Console.WriteLine($" {aleacion.Nombre} ({aleacion.Codigo}) | C: {aleacion.PorcentajeCarbono}% | Mn: {aleacion.PorcentajeManganeso}% | Cantidad: {aleacion.CantidadUtilizada} kg");` → muestra los datos de la aleación.

Línea 52: `private void DemostrarJoinExplicito()` → declara el método.

Línea 54: `Console.WriteLine("\n--- Órdenes con planchas y detalle (Join explícito) ---");` → muestra la cabecera.

Línea 56: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasYDetalleConJoinExplicito();` → llama al método del repositorio.

Línea 57: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 59: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 60: `Console.WriteLine($" Planchas: {orden.Planchas.Count}");` → muestra el número de planchas.

Línea 64: `private void DemostrarSqlJoin()` → declara el método.

Línea 66: `Console.WriteLine("\n--- SQL generado por una consulta con navegación ---");` → muestra la cabecera.

Línea 68: `var consulta = _unidad Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 69: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 70: `.Select(o => new` → proyecta a un tipo anónimo.

Línea 71: `o.NumeroOrden,` → incluye el número de orden.

Línea 72: `o.Cliente,` → incluye el cliente.

Línea 73: `Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList(),` → proyecta la colección de planchas.

Línea 74: `Detalle = o.Detalle == null ? null : new { o.Detalle.ComposicionQuimica, o.Detalle.TemperaturaColada }` → comprueba si el detalle existe y lo proyecta.

Línea 75: `});` → cierra la proyección.

Línea 77: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 78: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 7: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<JoinsUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 8: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

| Paso 9: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas, detalle y aleaciones:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
    Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500,
    Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000,
    Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
    TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
    var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
    CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
    var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
    context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 13: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 14: context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.

Línea 15: context.SaveChanges(); → inserta las órdenes.

Línea 17: var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha.

Línea 18: var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha.

Línea 19: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 22: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 23: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 25: `var ordenAleacion1 = new OrdenAleacion { ... };` → crea la primera relación.

Línea 26: `var ordenAleacion2 = new OrdenAleacion { ... };` → crea la segunda relación.

Línea 27: `var ordenAleacion3 = new OrdenAleacion { ... };` → crea la tercera relación.

Línea 28: `context OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);` → registra las relaciones.

Línea 30: `context.SaveChanges();` → inserta las planchas, el detalle y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 10: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan las órdenes con planchas y detalle, las órdenes con aleaciones, el join explícito y el SQL generado por una consulta con navegación.

Paso 11: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== JOINS Y NAVEGACIÓN ===

--- Órdenes con planchas y detalle (navegación por propiedades) ---
OF-2024-0001 | Constructora del Norte | Pendiente
  Planchas: 2
  Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C
OF-2024-0002 | Constructora del Sur | Pendiente
  Planchas: 1
  Detalle: Sin detalle

--- Órdenes con aleaciones (navegación por propiedades) ---
OF-2024-0001 | Constructora del Norte
  AISI 1045 (A1045) | C: 0.45% | Mn: 0.75% | Cantidad: 1500.5 kg
  AISI 4140 (A4140) | C: 0.40% | Mn: 0.85% | Cantidad: 800.0 kg
OF-2024-0002 | Constructora del Sur
  AISI 1045 (A1045) | C: 0.45% | Mn: 0.75% | Cantidad: 1200.0 kg

--- Órdenes con planchas y detalle (Join explícito) ---
OF-2024-0001 | Constructora del Norte | Pendiente
  Planchas: 2
OF-2024-0002 | Constructora del Sur | Pendiente
  Planchas: 1

--- SQL generado por una consulta con navegación ---
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso], [p].[Id], [d].[ComposicionQuimica],
[d].[TemperaturaColada]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[NumeroOrden], [o].[Id], [p].[Id]
La primera sección muestra las órdenes con planchas y detalle. La segunda sección muestra las órdenes
```

con aleaciones. La tercera sección muestra el join explícito. La cuarta sección muestra el SQL generado por una consulta con navegación.

Observaciones: el SQL generado incluye dos LEFT JOIN con las tablas de planchas y detalles. El filtro global de Soft Delete se aplica automáticamente. El orden de las cláusulas es el correcto: SELECT, FROM, LEFT JOIN, WHERE, ORDER BY.

Paso 12: Diagnosticar un error común

Modificar el método `ObtenerOrdenesConPlanchasYDetalle` para acceder a una propiedad del detalle sin comprobar si existe:

```
csharp
public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalle()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}
```

Resultado esperado: si alguna orden no tiene detalle, se produce una `NullReferenceException` en tiempo de ejecución porque `o.Detalle` es null. EF Core no puede traducir la proyección a SQL y lanza una excepción.

Solución: comprobar si `o.Detalle` es null antes de proyectarlo, como en la implementación original.

Errores comunes del ejercicio Error Causa Solución `NullReferenceException` en detalle No se comprobó si el detalle es null Usar operador ternario antes de proyectar INNER JOIN en lugar de LEFT JOIN Se usó `SelectMany` sin `DefaultIfEmpty` Usar `DefaultIfEmpty` para incluir entidades sin relación Producto cartesiano Se incluyeron varias colecciones Usar `AsSplitQuery` o proyecciones `ThenInclude` sin `Include` Se usó `ThenInclude` sin `Include` previo Usar `Include` antes de `ThenInclude` `GroupJoin` confuso Se usó `GroupJoin` sin entender su semántica Preferir `SelectMany` con `DefaultIfEmpty` N+1 en proyecciones Se proyecta una colección sin `ToList` Añadir `ToList` dentro de la proyección

■ Reto resuelto: Consulta con navegación de tres niveles

Reto: Crear un método en el repositorio que obtenga las órdenes con sus planchas, su detalle y sus aleaciones en una sola consulta. Añadir el DTO, el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Crear el DTO OrdenCompletaConAleacionesDto:

```
namespace AceriaData.Application.Dtos;

public class OrdenCompletaConAleacionesDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
    public DetalleDto? Detalle { get; set; }
    public List<AleacionDto> Aleaciones { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class OrdenCompletaConAleacionesDto → declara el DTO.

Línea 5: public string NumeroOrden { get; set; } = string.Empty; → número de orden.

Línea 6: public string Cliente { get; set; } = string.Empty; → cliente.

Línea 7: public string Estado { get; set; } = string.Empty; → estado.

Línea 8: public List Planchas { get; set; } = new(); → colección de planchas.

Línea 9: public DetalleDto? Detalle { get; set; } → detalle opcional.

Línea 10: public List Aleaciones { get; set; } = new(); → colección de aleaciones.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```
List<OrdenCompletaConAleacionesDto> ObtenerOrdenesCompletasConAleaciones();
```

Línea 1: List ObtenerOrdenesCompletasConAleaciones(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```

public List<OrdenCompletaConAleacionesDto> ObtenerOrdenesCompletasConAleaciones()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenCompletaConAleacionesDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            },
            Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto
            {
                Nombre = oa.Aleacion.Nombre,
               Codigo = oa.Aleacion.Codigo,
                PorcentajeCarbono = oa.Aleacion.PorcentajeCarbono,
                PorcentajeManganeso = oa.Aleacion.PorcentajeManganeso,
                CantidadUtilizada = oa.CantidadUtilizada
            }).ToList()
        })
        .ToList();
}

```

Línea 1: public List ObtenerOrdenesCompletasConAleaciones() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 5: .Select(o => new OrdenCompletaConAleacionesDto → proyecta al DTO.

Línea 6: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 7: Cliente = o.Cliente, → asigna el cliente.

Línea 8: Estado = o.Estado, → asigna el estado.

Línea 9: Planchas = o.Planchas.Select(p => new PlanchaDto → proyecta las planchas.

Línea 17: }).ToList(), → materializa la colección.

Línea 18: Detalle = o.Detalle == null ? null : new DetalleDto → comprueba si el detalle existe.

Línea 24: }, → cierra la proyección del detalle.

Línea 25: Aleaciones = o.OrdenesAleaciones.Select(oa => new AleacionDto → proyecta las aleaciones.

Línea 32: }).ToList() → materializa la colección.

Línea 33: }) → cierra la proyección.

Línea 34: .ToList(); → materializa la consulta.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarOrdenesCompletasConAleaciones()
{
    Console.WriteLine("\n--- Órdenes completas con aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConAleaciones();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        Console.WriteLine($"      Planchas: {orden.Planchas.Count}");
        var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica}";
        Console.WriteLine($"      Detalle: {detalle}");
        Console.WriteLine($"      Aleaciones: {orden.Aleaciones.Count}");
    }
}
```

Línea 1: `private void DemostrarOrdenesCompletasConAleaciones()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Órdenes completas con aleaciones ---");` → muestra la cabecera.

Línea 5: `var ordenes = _unidad.Ordenes.ObtenerOrdenesCompletasConAleaciones();` → llama al método del repositorio.

Línea 6: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 8: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos de la orden.

Línea 9: `Console.WriteLine($" Planchas: {orden.Planchas.Count}");` → muestra el número de planchas.

Línea 10: `var detalle = orden.Detalle == null ? "Sin detalle" : $"{orden.Detalle.ComposicionQuimica}";` → comprueba si el detalle existe.

Línea 11: `Console.WriteLine($" Detalle: {detalle}");` → muestra el detalle.

Línea 12: `Console.WriteLine($" Aleaciones: {orden.Aleaciones.Count}");` → muestra el número de aleaciones.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarOrdenesCompletasConAleaciones();
Paso 6: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas, detalle y aleaciones.
```

Resultado esperado: las órdenes se muestran con su número, cliente, estado, número de planchas, detalle y número de aleaciones.

Analugía final

Los joins en una acería son como las consultas que el jefe de planta hace al archivo central para combinar información de varias carpetas. En lugar de mirar la carpeta de órdenes, la carpeta de planchas y la carpeta de aleaciones por separado, el jefe pide un informe que combine las tres. El archivo central hace el join: para cada orden, busca sus planchas y sus aleaciones, y las presenta juntas. La navegación por propiedades es como pedir el informe usando las referencias internas de las carpetas: cada orden tiene una lista de planchas y una lista de aleaciones. El join explícito es como pedir el informe indicando manualmente cómo se relacionan las carpetas. `DefaultIfEmpty` es como pedir que se incluyan las órdenes aunque no tengan planchas. `GroupJoin` es como pedir un informe agrupado por orden. `Include` es como pedir que se adjunten las carpetas relacionadas al informe principal. Así funcionan los joins en EF Core: se combinan datos de varias tablas en una sola consulta, y el resultado se presenta de forma estructurada.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado los DTOs `OrdenConPlanchasYDetalleDto` y `OrdenConAleacionesDto`.

Añadido los métodos de join a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `JoinsUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas, detalle y aleaciones.

Ejecutado las demostraciones de órdenes con planchas y detalle, órdenes con aleaciones, join explícito y SQL generado por navegación.

Diagnosticado el error de no comprobar si el detalle es null.

Creado el DTO `OrdenCompletaConAleacionesDto` y su método.

| Conexión con el siguiente punto

En este punto se han añadido consultas con joins y navegación al proyecto `AceriaData`, incluyendo `Join`, `SelectMany`, `DefaultIfEmpty`, `GroupJoin` y navegación por propiedades. Se ha analizado el SQL generado y se ha comprobado que EF Core genera `INNER JOIN` o `LEFT JOIN` según el caso. En el siguiente punto se estudiará la carga Eager en detalle, incluyendo `Include`, `ThenInclude`, `AsSplitQuery` y sus implicaciones en el rendimiento.

Punto 3.8 - Eager Loading con Include y ThenInclude

Práctica

Ejercicio: Añadir métodos de carga Eager al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen Include, ThenInclude, Filtered Include, AsSplitQuery y AutoInclude. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.7 se añadieron consultas con joins y navegación, incluyendo Join, SelectMany, DefaultIfEmpty y GroupJoin. En este punto se profundiza en la carga Eager, incluyendo Include, ThenInclude, Filtered Include y AsSplitQuery. Estas técnicas se usarán en el punto 3.9 para la carga Lazy y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de carga Eager a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleInclude();
    List<OrdenFabricacion> ObtenerOrdenesConAleacionesInclude();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivas();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleSplitQuery();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesConPlanchasInclude(); → método que carga las órdenes con planchas usando Include.

Línea 11: List ObtenerOrdenesConPlanchasYDetalleInclude(); → método que carga las órdenes con planchas y detalle usando Include.

Línea 12: List ObtenerOrdenesConAleacionesInclude(); → método que carga las órdenes con aleaciones usando Include y ThenInclude.

Línea 13: `List ObtenerOrdenesConPlanchasActivas();` → método que carga las órdenes con planchas activas usando `Filtered Include`.

Línea 14: `List ObtenerOrdenesConPlanchasYDetalleSplitQuery();` → método que carga las órdenes con planchas y detalle usando `AsSplitQuery`.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 3: Implementar los métodos de carga Eager en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConAleacionesInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivas()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas.Where(p => p.Activa))
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
    
```

Línea 1: `public List ObtenerOrdenesConPlanchasInclude();` → declara el método que carga las órdenes con planchas.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta.

Línea 9: `public List ObtenerOrdenesConPlanchasYDetalleInclude()` → declara el método que carga las órdenes con planchas y detalle.

Línea 11: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 12: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 13: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 14: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 15: `.ToList();` → materializa la consulta.

Línea 18: `public List ObtenerOrdenesConAleacionesInclude()` → declara el método que carga las órdenes con aleaciones.

Línea 20: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 21: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 22: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 23: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 24: `.ToList();` → materializa la consulta.

Línea 27: `public List ObtenerOrdenesConPlanchasActivas()` → declara el método que carga las órdenes con planchas activas.

Línea 29: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 30: `.Include(o => o.Planchas.Where(p => p.Activa))` → incluye solo las planchas activas.

Línea 31: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 32: `.ToList();` → materializa la consulta.

Línea 35: `public List ObtenerOrdenesConPlanchasYDetalleSplitQuery()` → declara el método que carga las órdenes con planchas y detalle usando `AsSplitQuery`.

Línea 37: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 38: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 39: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 40: `.AsSplitQuery()` → divide la consulta en varias.

Línea 41: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 42: `.ToList();` → materializa la consulta.

Error común: si se incluyen varias colecciones sin `AsSplitQuery`, EF Core genera un producto cartesiano. Se debe usar `AsSplitQuery` para evitar el problema.

| Paso 4: Crear el caso de uso de carga Eager

Crear el archivo `src/AceriaData.Application/UseCases/CargaEagerUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaEagerUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaEagerUseCase(IUnidadDeTrabajo unidad)
```

```

{
    _unidad = unidad;
}

public void Ejecutar()
{
    Console.WriteLine("=== CARGA EAGER CON INCLUDE ===");

    DemostrarIncludePlanchas();
    DemostrarIncludePlanchasYDetalle();
    DemostrarThenIncludeAleaciones();
    DemostrarFilteredInclude();
    DemostrarSplitQuery();
    DemostrarSqlInclude();
}

private void DemostrarIncludePlanchas()
{
    Console.WriteLine("\n--- Include: órdenes con planchas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
{orden.Planchas.Count}");
    }
}

private void DemostrarIncludePlanchasYDetalle()
{
    Console.WriteLine("\n--- Include: órdenes con planchas y detalle ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude();
    foreach (var orden in ordenes)
    {
        var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
        Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle:
{detalle}");
    }
}

private void DemostrarThenIncludeAleaciones()
{
    Console.WriteLine("\n--- ThenInclude: órdenes con aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Aleaciones: {orden.OrdenesAleaciones.Count}");
        foreach (var ordenAleacion in orden.OrdenesAleaciones)
        {
            Console.WriteLine($" {ordenAleacion.Aleacion.Nombre} | Cantidad:
{ordenAleacion.CantidadUtilizada} kg");
        }
    }
}

private void DemostrarFilteredInclude()
{
    Console.WriteLine("\n--- Filtered Include: órdenes con planchas activas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}");
    }
}

private void DemostrarSplitQuery()
{
    Console.WriteLine("\n--- AsSplitQuery: órdenes con planchas y detalle ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();

```

```

        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
        }
    }

    private void DemostrarSqlInclude()
    {
        Console.WriteLine("\n--- SQL generado por un Include ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Include(o => o.Planchas)
            .Include(o => o.Detalle);

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class CargaEagerUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public CargaEagerUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CARGA EAGER CON INCLUDE ==="); → muestra la cabecera.

Línea 18: DemostrarIncludePlanchas(); → llama al método de Include con planchas.

Línea 19: DemostrarIncludePlanchasYDetalle(); → llama al método de Include con planchas y detalle.

Línea 20: DemostrarThenIncludeAleaciones(); → llama al método de ThenInclude con aleaciones.

Línea 21: DemostrarFilteredInclude(); → llama al método de Filtered Include.

Línea 22: DemostrarSplitQuery(); → llama al método de AsSplitQuery.

Línea 23: DemostrarSqlInclude(); → llama al método de SQL de Include.

Línea 26: private void DemostrarIncludePlanchas() → declara el método.

Línea 28: Console.WriteLine("\n--- Include: órdenes con planchas ---"); → muestra la cabecera.

Línea 30: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude(); → llama al método del repositorio.

Línea 31: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 33: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.Planchas.Count}"); → muestra los datos.

Línea 36: private void DemostrarIncludePlanchasYDetalle() → declara el método.

Línea 38: Console.WriteLine("\n--- Include: órdenes con planchas y detalle ---"); → muestra la cabecera.

Línea 40: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude(); → llama al método del repositorio.

Línea 41: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 43: var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica; → comprueba si el detalle existe.

Línea 44: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle: {detalle}");` → muestra los datos.

Línea 48: `private void DemostrarThenIncludeAleaciones()` → declara el método.

Línea 50: `Console.WriteLine("\n--- ThenInclude: órdenes con aleaciones ---");` → muestra la cabecera.

Línea 52: `var ordenes = _unidad Ordenes.ObtenerOrdenesConAleacionesInclude();` → llama al método del repositorio.

Línea 53: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 55: `Console.WriteLine($" {orden.NumeroOrden} | Aleaciones: {orden.OrdenesAleaciones.Count}");` → muestra el número de aleaciones.

Línea 56: `foreach (var ordenAleacion in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 58: `Console.WriteLine($" {ordenAleacion.Aleacion.Nombre} | Cantidad: {ordenAleacion.CantidadUtilizada} kg");` → muestra los datos.

Línea 62: `private void DemostrarFilteredInclude()` → declara el método.

Línea 64: `Console.WriteLine("\n--- Filtered Include: órdenes con planchas activas ---");` → muestra la cabecera.

Línea 66: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasActivas();` → llama al método del repositorio.

Línea 67: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 69: `Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}");` → muestra los datos.

Línea 73: `private void DemostrarSplitQuery()` → declara el método.

Línea 75: `Console.WriteLine("\n--- AsSplitQuery: órdenes con planchas y detalle ---");` → muestra la cabecera.

Línea 77: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();` → llama al método del repositorio.

Línea 78: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 80: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");` → muestra los datos.

Línea 84: `private void DemostrarSqlInclude()` → declara el método.

Línea 86: `Console.WriteLine("\n--- SQL generado por un Include ---");` → muestra la cabecera.

Línea 88: `var consulta = _unidad Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 89: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 90: `.Include(o => o.Detalle);` → incluye la referencia al detalle.

Línea 92: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 93: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CargaEagerUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas, detalle y aleaciones:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
    Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500,
    Peso = 280.8m, Activa = false };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000,
    Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
    TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
    var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
    CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
    var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
    context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 13: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 14: context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.

Línea 15: context.SaveChanges(); → inserta las órdenes.

Línea 17: var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha activa.

Línea 18: var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha inactiva.

Línea 19: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha activa.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 22: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 23: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 25: `var ordenAleacion1 = new OrdenAleacion { ... };` → crea la primera relación.

Línea 26: `var ordenAleacion2 = new OrdenAleacion { ... };` → crea la segunda relación.

Línea 27: `var ordenAleacion3 = new OrdenAleacion { ... };` → crea la tercera relación.

Línea 28: `context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);` → registra las relaciones.

Línea 30: `context.SaveChanges();` → inserta las planchas, el detalle y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan las órdenes con planchas, con planchas y detalle, con aleaciones, con planchas activas, con `AsSplitQuery` y el SQL generado por `Include`.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CARGA EAGER CON INCLUDE ===

--- Include: órdenes con planchas ---
OF-2024-0001 | Constructora del Norte | Planchas: 2
OF-2024-0002 | Constructora del Sur | Planchas: 1

--- Include: órdenes con planchas y detalle ---
OF-2024-0001 | Planchas: 2 | Detalle: C: 0.45%, Mn: 0.75%
OF-2024-0002 | Planchas: 1 | Detalle: Sin detalle

--- ThenInclude: órdenes con aleaciones ---
OF-2024-0001 | Aleaciones: 2
  AISI 1045 | Cantidad: 1500.5 kg
  AISI 4140 | Cantidad: 800.0 kg
OF-2024-0002 | Aleaciones: 1
  AISI 1045 | Cantidad: 1200.0 kg

--- Filtered Include: órdenes con planchas activas ---
OF-2024-0001 | Planchas activas: 1
OF-2024-0002 | Planchas activas: 1

--- AsSplitQuery: órdenes con planchas y detalle ---
OF-2024-0001 | Planchas: 2
OF-2024-0002 | Planchas: 1

--- SQL generado por un Include ---
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ..., [p].[Id], [p].[Espesor], ..., [d].[Id], [d].[ComposicionQuimica], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
```

```
ORDER BY [o].[Id], [p].[Id]
```

La primera sección muestra las órdenes con planchas. La segunda sección muestra las órdenes con planchas y detalle. La tercera sección muestra las órdenes con aleaciones. La cuarta sección muestra las órdenes con planchas activas. La quinta sección muestra las órdenes con AsSplitQuery. La sexta sección muestra el SQL generado por Include.

Observaciones: el SQL generado incluye dos LEFT JOIN con las tablas de planchas y detalles. El filtro global de Soft Delete se aplica automáticamente. El Filtered Include filtra las planchas inactivas. AsSplitQuery divide la consulta en varias.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerOrdenesConPlanchasInclude para llamar a ToList antes del Include:

```
csharp
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .ToList()
        .Select(o =>
        {
            o.Planchas = _context.PlanchasAcero.Where(p => p.OrdenId == o.Id).ToList();
            return o;
        })
        .ToList();
}
```

Resultado esperado: el código ejecuta una consulta para cargar todas las órdenes y después una consulta por cada orden para cargar sus planchas. Esto es el problema N+1: una consulta para las órdenes y N consultas para las planchas. Si hay cien órdenes, se ejecutan ciento una consultas.

Solución: usar Include para cargar las planchas en una sola consulta.

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: el código ejecuta una sola consulta con un LEFT JOIN para cargar las órdenes y sus planchas. No hay problema N+1.

Errores comunes del ejercicio

Error	Causa	Solución
Producto cartesiano	Se incluyeron varias colecciones sin AsSplitQuery	Usar AsSplitQuery
N+1	Se llamó a ToList antes del Include	Aplicar Include antes de ToList
ThenInclude sin Include	Se usó ThenInclude sin Include previo	Usar Include antes de ThenInclude
NullReferenceException en referencia	La propiedad de navegación es null	Comprobar si la propiedad es null
Filtro no aplicado	Se aplicó el filtro fuera del Include	Aplicar el filtro dentro del Include
AutoInclude excesivo	Se configuró en muchas propiedades	Usar con moderación

Reto resuelto: Consulta con Include, ThenInclude y Filtered Include
 Reto: Crear un método en el repositorio que cargue las órdenes con sus planchas activas y sus aleaciones, usando Include con filtro y ThenInclude. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
csharp
List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivasYAleaciones();
```

Línea 1: List ObtenerOrdenesConPlanchasActivasYAleaciones(); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivasYAleaciones()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas.Where(p => p.Activa))
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesConPlanchasActivasYAleaciones() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas.Where(p => p.Activa)) → incluye solo las planchas activas.

Línea 5: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 6: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 7: .AsSplitQuery() → divide la consulta en varias para evitar el producto cartesiano.

Línea 8: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 9: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPlanchasActivasYAleaciones()
{
    Console.WriteLine("\n--- Planchas activas y aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivasYAleaciones();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count} |
Aleaciones: {orden.OrdenesAleaciones.Count}");
    }
}
```

Línea 1: private void DemostrarPlanchasActivasYAleaciones() → declara el método.

Línea 3: Console.WriteLine("\n--- Planchas activas y aleaciones ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivasYAleaciones(); → llama al método del repositorio.

Línea 6: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 8: Console.WriteLine(\$" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count} | Aleaciones: {orden.OrdenesAleaciones.Count}"); → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPlanchasActivasYAleaciones();
Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas activas y sus aleaciones.
```

Resultado esperado: las órdenes se muestran con su número, número de planchas activas y número de aleaciones. AsSplitQuery divide la consulta en varias para evitar el producto cartesiano.

| Analogía final

La carga Eager en una acería es como pedirle al archivo central que, además de la carpeta de la orden, adjunte también las carpetas de las planchas y del detalle. En lugar de pedir la carpeta de la orden y después ir a buscar las planchas y el detalle por separado, se pide todo junto. El archivo central prepara un paquete con la orden y sus documentos relacionados. Include es como pedir que se adjunte una carpeta. ThenInclude es como pedir que se adjunten los documentos de la carpeta adjunta. Filtered Include es como pedir que solo se adjunten las planchas activas. AsSplitQuery es como pedir que el paquete se prepare en varios envíos separados para evitar que el paquete sea demasiado grande. AutoInclude es como pedir que siempre se adjunten las planchas, sin tener que pedirlo cada vez. Así funciona la carga Eager en EF Core: se cargan las entidades relacionadas en una sola consulta, se evita el problema N+1 y se controla qué se carga y qué no.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de carga Eager a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso CargaEagerUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas activas e inactivas, detalle y aleaciones.

Ejecutado las demostraciones de Include, ThenInclude, Filtered Include y AsSplitQuery.

Analizado el SQL generado por Include.

Diagnosticado el error de aplicar ToList antes del Include.

Creado el método ObtenerOrdenesConPlanchasActivasYAleaciones.

| Conexión con el siguiente punto

En este punto se ha profundizado en la carga Eager, incluyendo Include, ThenInclude, Filtered Include, AsSplitQuery y AutoInclude. Se ha analizado el SQL generado y se ha comprobado que EF Core genera LEFT JOIN para las entidades relacionadas. En el siguiente punto se estudiará la carga Lazy, incluyendo su configuración, su funcionamiento y sus riesgos.

Punto 3.9 - Lazy Loading: configuración, funcionamiento y riesgos

Práctica

Ejercicio: Habilitar la carga Lazy en el proyecto AceriaData, marcar las propiedades de navegación como virtual, demostrar el problema N+1 y comparar el rendimiento con la carga Eager. Crear un caso de uso que ejecute las consultas con carga Lazy y analice el SQL generado.

Contexto del proyecto: En el punto 3.8 se profundizó en la carga Eager con Include, ThenInclude, Filtered Include y AsSplitQuery. En este punto se explora la carga Lazy con fines demostrativos, incluyendo su configuración, su funcionamiento y sus riesgos. Esta exploración servirá para justificar el uso de la carga Eager en los módulos posteriores.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Infrastructure
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Infrastructure → entra en la carpeta del proyecto de infraestructura.

Paso 2: Instalar el paquete de proxies

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies
```

dotnet add package → añade una referencia a un paquete NuGet. Microsoft.EntityFrameworkCore.Proxies → nombre del paquete que contiene los proxies para la carga Lazy.

Error común: si se olvida instalar el paquete, el método UseLazyLoadingProxies no está disponible y el código no compila.

Paso 3: Marcar las propiedades de navegación como virtual

Modificar las entidades del dominio en src/AceriaData.Domain/Entities/:

OrdenFabricacion.cs:

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = string.Empty;
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual List<PlanchaAcero> Planchas { get; set; } = new();
    public virtual DetalleOrden? Detalle { get; set; }
    public virtual CertificadoCalidad? Certificado { get; set; }
    public virtual List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Domain.Entities; → declara el espacio de nombres.

Línea 3: public class OrdenFabricacion → declara la entidad.

Línea 5: `public int Id { get; set; } → clave primaria.`

Línea 6: `public string NumeroOrden { get; set; } = string.Empty; → número de orden.`

Línea 7: `public string Cliente { get; set; } = string.Empty; → cliente.`

Línea 8: `public DateTime FechaCreacion { get; set; } → fecha de creación.`

Línea 9: `public DateTime? FechaEntrega { get; set; } → fecha de entrega opcional.`

Línea 10: `public string Estado { get; set; } = string.Empty; → estado.`

Línea 11: `public string? Observaciones { get; set; } → observaciones opcionales.`

Línea 12: `public bool IsDeleted { get; set; } → indica si está eliminada.`

Línea 13: `public DateTime? DeletedAt { get; set; } → fecha de eliminación.`

Línea 15: `public virtual List Planchas { get; set; } = new(); → colección de planchas. La palabra virtual permite que el proxy la sobrescriba.`

Línea 16: `public virtual DetalleOrden? Detalle { get; set; } → detalle. La palabra virtual permite que el proxy la sobrescriba.`

Línea 17: `public virtual CertificadoCalidad? Certificado { get; set; } → certificado. La palabra virtual permite que el proxy la sobrescriba.`

Línea 18: `public virtual List OrdenesAleaciones { get; set; } = new(); → colección de entidades intermedias. La palabra virtual permite que el proxy la sobrescriba.`

Error común: si las propiedades de navegación no son virtual, el proxy no puede sobrescribirlas y la carga Lazy no funciona. Se debe marcar todas las propiedades de navegación como virtual.

PlanchaAcero.cs:

```
namespace AceriaData.Domain.Entities;

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 15: `public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.`

Error común: si la referencia a la orden no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

Aleacion.cs:

```
namespace AceriaData.Domain.Entities;

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public decimal PorcentajeCarbono { get; set; }
    public decimal PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool Activo { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

Línea 15: public virtual List OrdenesAleaciones { get; set; } = new(); → colección de entidades intermedias. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la colección no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

DetalleOrden.cs:

```
namespace AceriaData.Domain.Entities;

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la referencia no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

CertificadoCalidad.cs:

```
namespace AceriaData.Domain.Entities;

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la referencia no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

OrdenAleacion.cs:

```
namespace AceriaData.Domain.Entities;

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; }
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = string.Empty;

    public virtual OrdenFabricacion Orden { get; set; } = null!;
    public virtual Aleacion Aleacion { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Línea 11: public virtual Aleacion Aleacion { get; set; } = null!; → referencia a la aleación. La palabra virtual permite que el proxy la sobrescriba.

Error común: si las referencias no son virtual, el proxy no puede sobrescribirlas y la carga Lazy no funciona.

| Paso 4: Configurar la carga Lazy en el DbContext

Modificar el método OnConfiguring del AceriaDbContext en src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder
            .UseSqlServer(
                "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;",
                sqlOptions =>
                {
                    sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                    sqlOptions.CommandTimeout(60);
                })
            .UseLazyLoadingProxies()
            .LogTo(
                Console.WriteLine,
                new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                LogLevel.Information)
            .EnableSensitiveDataLogging()
            .EnableDetailedErrors();
    }
}
```

Línea 1: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración.

Línea 3: if (!optionsBuilder.IsConfigured) → comprueba si las opciones ya están configuradas.

Línea 5: optionsBuilder → objeto de configuración.

Línea 6: .UseSqlServer(→ registra el proveedor de SQL Server.

Línea 7: "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;", → cadena de conexión.

Línea 8: sqlOptions => → delegado de configuración del proveedor.

Línea 10: sqlOptions.EnableRetryOnFailure(maxRetryCount: 5); → habilita los reintentos automáticos.

Línea 11: sqlOptions.CommandTimeout(60); → establece el tiempo de espera.

Línea 12: }) → cierra el delegado.

Línea 13: .UseLazyLoadingProxies() → habilita la carga Lazy con proxies.

Línea 14: .LogTo(→ habilita el logging.

Línea 15: Console.WriteLine, → destino del logging.

Línea 16: new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, → categorías.

Línea 17: LogLevel.Information) → nivel mínimo.

Línea 18: .EnableSensitiveDataLogging() → muestra los valores de los parámetros.

Línea 19: .EnableDetailedErrors(); → muestra información detallada en los errores.

Error común: si se olvida el using Microsoft.EntityFrameworkCore;, el método UseLazyLoadingProxies no está disponible y el código no compila.

| Paso 5: Crear el caso de uso de carga Lazy

Crear el archivo src/AceriaData.Application/UseCases/CargaLazyUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaLazyUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaLazyUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CARGA LAZY CON PROXIES ===");

        DemostrarCargaLazySimple();
        DemostrarProblemaN1();
        DemostrarCargaEagerComoAlternativa();
    }

    private void DemostrarCargaLazySimple()
    {
        Console.WriteLine("\n--- Carga Lazy simple ---");

        var orden = _unidad.Ordenes.ObtenerPorId(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
            Console.WriteLine($" Tipo de la entidad: {orden.GetType().Name}");
            Console.WriteLine($" Planchas (carga Lazy): {orden.Planchas.Count}");
        }
    }

    private void DemostrarProblemaN1()
    {
        Console.WriteLine("\n--- Problema N+1 con carga Lazy ---");

        var ordenes = _unidad.Ordenes.ObtenerTodas();
        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");

        var totalPlanchas = 0;
        foreach (var orden in ordenes)
        {
            totalPlanchas += orden.Planchas.Count;
        }
    }
}
```

```

        Console.WriteLine($"Total de planchas: {totalPlanchas}");
    }

    private void DemostrarCargaEagerComoAlternativa()
    {
        Console.WriteLine("\n--- Carga Eager como alternativa ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
        var totalPlanchas = ordenes.Sum(o => o.Planchas.Count);

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Total de planchas: {totalPlanchas}");
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class CargaLazyUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public CargaLazyUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CARGA LAZY CON PROXIES ==="); → muestra la cabecera.

Línea 18: DemostrarCargaLazySimple(); → llama al método de carga Lazy simple.

Línea 19: DemostrarProblemaN1(); → llama al método del problema N+1.

Línea 20: DemostrarCargaEagerComoAlternativa(); → llama al método de carga Eager como alternativa.

Línea 23: private void DemostrarCargaLazySimple() → declara el método.

Línea 25: Console.WriteLine("\n--- Carga Lazy simple ---"); → muestra la cabecera.

Línea 27: var orden = _unidad.Ordenes.ObtenerPorId(1); → carga la orden por Id.

Línea 28: if (orden is not null) → comprueba si la orden existe.

Línea 30: Console.WriteLine(\$" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}"); → muestra los datos.

Línea 31: Console.WriteLine(\$" Tipo de la entidad: {orden.GetType().Name}"); → muestra el tipo de la entidad, que es el proxy.

Línea 32: Console.WriteLine(\$" Planchas (carga Lazy): {orden.Planchas.Count}"); → accede a las planchas, lo que provoca la carga Lazy.

Línea 36: private void DemostrarProblemaN1() → declara el método.

Línea 38: Console.WriteLine("\n--- Problema N+1 con carga Lazy ---"); → muestra la cabecera.

Línea 40: var ordenes = _unidad.Ordenes.ObtenerTodas(); → carga todas las órdenes.

Línea 41: Console.WriteLine(\$"Órdenes cargadas: {ordenes.Count}"); → muestra el número de órdenes.

Línea 43: var totalPlanchas = 0; → inicializa el contador.

Línea 44: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 46: totalPlanchas += orden.Planchas.Count; → accede a las planchas, lo que provoca una consulta por cada orden.

Línea 48: Console.WriteLine(\$"Total de planchas: {totalPlanchas}"); → muestra el total.

Línea 51: `private void DemostrarCargaEagerComoAlternativa()` → declara el método.

Línea 53: `Console.WriteLine("\n--- Carga Eager como alternativa ---");` → muestra la cabecera.

Línea 55: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();` → carga las órdenes con planchas usando `Include`.

Línea 56: `var totalPlanchas = ordenes.Sum(o => o.Planchas.Count);` → suma las planchas sin consultas adicionales.

Línea 58: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes.

Línea 59: `Console.WriteLine($"Total de planchas: {totalPlanchas}");` → muestra el total.

Error común: si la carga Lazy no está habilitada, el acceso a `orden.Planchas` devuelve una colección vacía en lugar de cargar las planchas. Se debe asegurar que `UseLazyLoadingProxies` está configurado.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CargaLazyUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 7: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) };
    context OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 8: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 9: var orden3 = new OrdenFabricacion { ... }; → crea la tercera orden.

Línea 10: context.OrdenesFabricacion.AddRange(orden1, orden2, orden3); → registra las órdenes.

Línea 11: context.SaveChanges(); → inserta las órdenes.

Línea 13: var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha.

Línea 14: var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha.

Línea 15: var plancha3 = new PlanchaAcero { ... }; → crea la tercera plancha.

Línea 16: var plancha4 = new PlanchaAcero { ... }; → crea la cuarta plancha.

Línea 18: context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4); → registra las planchas.

Línea 19: context.SaveChanges(); → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea OrdenId no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tipo de la entidad (el proxy), el problema N+1 con las consultas adicionales y la carga Eager como alternativa.

Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== CARGA LAZY CON PROXIES ===

--- Carga Lazy simple ---
Orden: OF-2024-0001 | Cliente: Constructora del Norte
Tipo de la entidad: OrdenFabricacionProxy
Planchas (carga Lazy): 2

--- Problema N+1 con carga Lazy ---
Órdenes cargadas: 3
... (3 consultas adicionales para cargar las planchas de cada orden)
Total de planchas: 4

--- Carga Eager como alternativa ---
Órdenes cargadas: 3
Total de planchas: 4
La primera sección muestra el tipo de la entidad, que es OrdenFabricacionProxy, confirmando que la carga Lazy está habilitada. La segunda sección muestra el problema N+1: se ejecutan tres consultas adicionales para cargar las planchas de cada orden. La tercera sección muestra la carga Eager como alternativa, que ejecuta una sola consulta.

Observaciones: en la primera sección, el acceso a orden.Planchas.Count provoca una consulta adicional. En la segunda sección, el bucle sobre las órdenes provoca una consulta por cada orden. En la tercera sección, la carga Eager carga las planchas en una sola consulta.

Paso 11: Diagnosticar un error común
Modificar el método DemostrarProblemaN1 para acceder a la propiedad Planchas de cada orden dentro de un bucle anidado:

csharp
private void DemostrarProblemaN1()
{
    Console.WriteLine("\n--- Problema N+1 con carga Lazy ---");

    var ordenes = _unidad.Ordenes.ObtenerTodas();
    Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");

    foreach (var orden in ordenes)
    {
        Console.WriteLine($" Orden: {orden.NumeroOrden}");
        foreach (var plancha in orden.Planchas)
        {
            Console.WriteLine($" Plancha: {plancha.Espesor} mm");
            Console.WriteLine($" Aleaciones: {plancha.Orden.OrdenesAleaciones.Count}");
        }
    }
}

```

Resultado esperado: el bucle anidado accede a plancha.Orden.OrdenesAleaciones, lo que provoca consultas adicionales. Si hay tres órdenes y cuatro planchas, se ejecutan tres consultas para las órdenes, cuatro para las planchas y cuatro para las aleaciones de cada plancha. En total, once consultas.

Solución: usar Include con ThenInclude para cargar todas las entidades relacionadas en una sola consulta.

```

var ordenes = _unidad.Ordenes.ObtenerQueryable()
    .Include(o => o.Planchas)
    .ThenInclude(p => p.Orden)
    .ThenInclude(o => o.OrdenesAleaciones)
    .ToList();
Resultado esperado con la solución: una sola consulta carga todas las entidades relacionadas.

```

Errores comunes del ejercicio

Error	Causa	Solución
Carga Lazy no funciona	Las propiedades de navegación no son virtual	Marcar como virtual
Carga Lazy no funciona	No se configuró UseLazyLoadingProxies	Configurar en OnConfiguring
N+1 en bucle	Se accede a propiedades de navegación en un bucle	Usar Include
ObjectDisposedException	Se accede a propiedades de navegación fuera del ámbito	Usar
Include o DTOs		
Referencia circular en serialización	Las entidades se referencian mutuamente	Usar DTOs o configurar el serializador
NullReferenceException	La propiedad de navegación es null	Comprobar antes de acceder
Proxies no generados	Las clases son sealed	Quitar sealed
Reto resuelto: Comparar carga Lazy y carga Eager con medición de tiempo		
Reto: Crear un método en el caso de uso que mida el tiempo de ejecución de una consulta con carga Lazy y otra con carga Eager. Comparar los resultados y analizar la diferencia.		

Solución paso a paso:

Paso 1: Añadir el método CompararRendimiento al caso de uso:

```
csharp
private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroLazy = System.Diagnostics.Stopwatch.StartNew();
    var ordenesLazy = _unidad.Ordenes.ObtenerTodas();
    var totalLazy = ordenesLazy.Sum(o => o.Planchas.Count);
    cronometroLazy.Stop();

    Console.WriteLine($"Carga Lazy: {cronometroLazy.ElapsedMilliseconds} ms | Total planchas: {totalLazy}");

    var cronometroEager = System.Diagnostics.Stopwatch.StartNew();
    var ordenesEager = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
    var totalEager = ordenesEager.Sum(o => o.Planchas.Count);
    cronometroEager.Stop();

    Console.WriteLine($"Carga Eager: {cronometroEager.ElapsedMilliseconds} ms | Total planchas: {totalEager}");
}
```

Línea 1: private void CompararRendimiento() → declara el método.

Línea 3: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 5: var cronometroLazy = System.Diagnostics.Stopwatch.StartNew(); → inicia el cronómetro para la carga Lazy.

Línea 6: var ordenesLazy = _unidad.Ordenes.ObtenerTodas(); → carga todas las órdenes.

Línea 7: var totalLazy = ordenesLazy.Sum(o => o.Planchas.Count); → suma las planchas con carga Lazy.

Línea 8: cronometroLazy.Stop(); → detiene el cronómetro.

Línea 10: Console.WriteLine(\$"Carga Lazy: {cronometroLazy.ElapsedMilliseconds} ms | Total planchas: {totalLazy}"); → muestra el tiempo de la carga Lazy.

Línea 12: var cronometroEager = System.Diagnostics.Stopwatch.StartNew(); → inicia el cronómetro para la carga Eager.

Línea 13: var ordenesEager = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude(); → carga las órdenes con planchas usando Include.

Línea 14: var totalEager = ordenesEager.Sum(o => o.Planchas.Count); → suma las planchas con carga Eager.

Línea 15: cronometroEager.Stop(); → detiene el cronómetro.

Línea 17: Console.WriteLine(\$"Carga Eager: {cronometroEager.ElapsedMilliseconds} ms | Total planchas: {totalEager}"); → muestra el tiempo de la carga Eager.

Paso 2: Llamar al método desde Ejecutar:

```
CompararRendimiento();
```

Paso 3: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: la carga Lazy tarda más que la carga Eager porque ejecuta más consultas. La diferencia es más notable cuando hay más órdenes.

Analogía final

La carga Lazy en una acería es como pedirle al archivo central que no te traiga las carpetas relacionadas hasta que las pidas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Si después necesitas las planchas, el archivo central te las trae en ese momento. Si necesitas el detalle, te lo trae después. Cada petición adicional es un viaje al archivo central. Si tienes cien órdenes y pides las planchas de cada una, haces cien viajes. Eso es el problema N+1. La carga Eager es como pedir todas las carpetas relacionadas de una vez: un solo viaje con todo lo necesario. La carga Lazy es cómoda porque no tienes que decidir de antemano qué necesitas, pero es ineficiente si haces muchas peticiones. En una acería con mucho volumen, la carga Eager es la opción correcta. En un prototipo o en una aplicación pequeña, la carga Lazy puede ser aceptable. Así funciona la carga Lazy en EF Core: cómoda pero peligrosa, útil en prototipos pero ineficiente en producción.

Resultado esperado

Al final del ejercicio, deberías haber:

Instalado el paquete Microsoft.EntityFrameworkCore.Proxies.

Marcado las propiedades de navegación como virtual.

Configurado UseLazyLoadingProxies en el DbContext.

Creado el caso de uso CargaLazyUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado la demostración de carga Lazy simple y verificado el tipo del proxy.

Observado el problema N+1 en el bucle.

Comprobado la carga Eager como alternativa.

Diagnosticado el error de acceder a propiedades de navegación en un bucle anidado.

Creado el método CompararRendimiento.

Conexión con el siguiente punto

En este punto se ha explorado la carga Lazy, incluyendo su configuración con proxies, su funcionamiento interno, el problema N+1 y sus riesgos en aplicaciones web y serialización. Se ha comprobado que la carga Lazy es cómoda pero ineficiente, y que la carga Eager es la alternativa recomendada. En el siguiente punto se estudiará la carga Explicit, que permite cargar entidades relacionadas bajo demanda de forma controlada.

Punto 3.10 - Explicit Loading

| Práctica

Ejercicio: Añadir métodos de carga Explicit al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen `Entry().Collection().Load()`, `Entry().Reference().Load()`, `IsLoaded` y `Query()`. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.9 se exploró la carga Lazy, incluyendo su configuración con proxies, el problema N+1 y sus riesgos. En este punto se explora la carga Explicit, que permite cargar entidades relacionadas de forma controlada. Esta técnica se usará en el punto 3.11 para la composición de consultas y en el Módulo 4 para las optimizaciones de rendimiento.

| Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

`cd AceriaData` → entra en la carpeta raíz del proyecto. `cd src/AceriaData.Console` → entra en la carpeta del proyecto de consola.

| Paso 2: Añadir los métodos de carga Explicit a la interfaz del repositorio

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId);
    OrdenFabricacion? CargarDetalleExplicitamente(int ordenId);
    OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId);
    OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio` → declara la interfaz.

Línea 8: `// ... métodos existentes ...` → comentario que indica que se mantienen los métodos anteriores.

Línea 10: `OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId);` → método que carga las planchas de una orden de forma explícita.

Línea 11: `OrdenFabricacion? CargarDetalleExplicitamente(int ordenId);` → método que carga el detalle de una orden de forma explícita.

Línea 12: `OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId);` → método que carga las planchas activas de una orden de forma explícita.

Línea 13: `OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId);` → método que carga las planchas y el detalle de una orden de forma explícita.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o entidades concretas.

| Paso 3: Implementar los métodos de carga Explicit en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Load();
    return orden;
}

public OrdenFabricacion? CargarDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Reference(o => o.Detalle).Load();
    return orden;
}

public OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Query().Where(p => p.Activa).Load();
    return orden;
}

public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    var entry = _context.Entry(orden);

    if (!entry.Collection(o => o.Planchas).IsLoaded)
    {
        entry.Collection(o => o.Planchas).Load();
    }

    if (!entry.Reference(o => o.Detalle).IsLoaded)
    {
        entry.Reference(o => o.Detalle).Load();
    }
}
```

```

    return orden;
}

```

Línea 1: `public OrdenFabricacion? CargarPlanchasExplicitamente(int ordenId) →` declara el método que carga las planchas.

Línea 3: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); →` carga la orden.

Línea 4: `if (orden is null) →` comprueba si la orden existe.

Línea 6: `return null; →` devuelve null si no existe.

Línea 9: `_context.Entry(orden).Collection(o => o.Planchas).Load(); →` carga la colección de planchas de forma explícita.

Línea 10: `return orden; →` devuelve la orden con las planchas cargadas.

Línea 13: `public OrdenFabricacion? CargarDetalleExplicitamente(int ordenId) →` declara el método que carga el detalle.

Línea 15: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); →` carga la orden.

Línea 16: `if (orden is null) →` comprueba si la orden existe.

Línea 18: `return null; →` devuelve null si no existe.

Línea 21: `_context.Entry(orden).Reference(o => o.Detalle).Load(); →` carga la referencia al detalle de forma explícita.

Línea 22: `return orden; →` devuelve la orden con el detalle cargado.

Línea 25: `public OrdenFabricacion? CargarPlanchasActivasExplicitamente(int ordenId) →` declara el método que carga las planchas activas.

Línea 27: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); →` carga la orden.

Línea 28: `if (orden is null) →` comprueba si la orden existe.

Línea 30: `return null; →` devuelve null si no existe.

Línea 33: `_context.Entry(orden).Collection(o => o.Planchas).Query().Where(p => p.Activa).Load(); →` carga solo las planchas activas.

Línea 34: `return orden; →` devuelve la orden con las planchas activas cargadas.

Línea 37: `public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId) →` declara el método que carga las planchas y el detalle.

Línea 39: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId); →` carga la orden.

Línea 40: `if (orden is null) →` comprueba si la orden existe.

Línea 42: `return null; →` devuelve null si no existe.

Línea 45: `var entry = _context.Entry(orden); →` obtiene el EntityEntry.

Línea 47: `if (!entry.Collection(o => o.Planchas).IsLoaded) →` comprueba si la colección ya está cargada.

Línea 49: `entry.Collection(o => o.Planchas).Load(); →` carga la colección de planchas.

Línea 52: `if (!entry.Reference(o => o.Detalle).IsLoaded) →` comprueba si la referencia ya está cargada.

Línea 54: `entry.Reference(o => o.Detalle).Load(); →` carga la referencia al detalle.

Línea 57: return orden; → devuelve la orden con las planchas y el detalle cargados.

Error común: si se llama a Load sin comprobar IsLoaded, EF Core puede ejecutar una consulta innecesaria si la propiedad ya estaba cargada por un Include previo.

▮ Paso 4: Crear el caso de uso de carga Explicit

Crear el archivo src/AceriaData.Application/UseCases/CargaExplicitUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaExplicitUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaExplicitUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CARGA EXPLICIT ===");

        DemostrarCargaReferencia();
        DemostrarCargaColeccion();
        DemostrarCargaConFiltro();
        DemostrarCargaMultipleConIsLoaded();
    }

    private void DemostrarCargaReferencia()
    {
        Console.WriteLine("\n--- Carga Explicit de referencia (detalle) ---");

        var orden = _unidad.Ordenes.CargarDetalleExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
            var detalle = orden.Detalle == null ? "Sin detalle" :
            $"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C";
            Console.WriteLine($" Detalle: {detalle}");
        }
    }

    private void DemostrarCargaColeccion()
    {
        Console.WriteLine("\n--- Carga Explicit de colección (planchas) ---");

        var orden = _unidad.Ordenes.CargarPlanchasExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
            foreach (var plancha in orden.Planchas)
            {
                Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso:
{plancha.Peso} kg");
            }
        }
    }

    private void DemostrarCargaConFiltro()
    {
        Console.WriteLine("\n--- Carga Explicit con filtro (planchas activas) ---");

        var orden = _unidad.Ordenes.CargarPlanchasActivasExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas activas:
{orden.Planchas.Count}");
            foreach (var plancha in orden.Planchas)
```

```

        {
            Console.WriteLine($"    Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Activa:
{plancha.Activa}");
        }
    }

    private void DemostrarCargaMultipleConIsLoaded()
    {
        Console.WriteLine("\n--- Carga Explicit de múltiples propiedades con IsLoaded ---");

        var orden = _unidad.Ordenes.CargarPlanchasYDetalleExplicitamente(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
            var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
            Console.WriteLine($" Detalle: {detalle}");
        }
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class CargaExplicitUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public CargaExplicitUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CARGA EXPLICIT ==="); → muestra la cabecera.

Línea 18: DemostrarCargaReferencia(); → llama al método de carga de referencia.

Línea 19: DemostrarCargaColeccion(); → llama al método de carga de colección.

Línea 20: DemostrarCargaConFiltro(); → llama al método de carga con filtro.

Línea 21: DemostrarCargaMultipleConIsLoaded(); → llama al método de carga múltiple con IsLoaded.

Línea 24: private void DemostrarCargaReferencia() → declara el método.

Línea 26: Console.WriteLine("\n--- Carga Explicit de referencia (detalle) ---"); → muestra la cabecera.

Línea 28: var orden = _unidad.Ordenes.CargarDetalleExplicitamente(1); → llama al método del repositorio.

Línea 29: if (orden is not null) → comprueba si la orden existe.

Línea 31: Console.WriteLine(\$" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}"); → muestra los datos de la orden.

Línea 32: var detalle = orden.Detalle == null ? "Sin detalle" : \$"{orden.Detalle.ComposicionQuimica} | {orden.Detalle.TemperaturaColada}°C"; → comprueba si el detalle existe.

Línea 33: Console.WriteLine(\$" Detalle: {detalle}"); → muestra el detalle.

Línea 37: private void DemostrarCargaColeccion() → declara el método.

Línea 39: Console.WriteLine("\n--- Carga Explicit de colección (planchas) ---"); → muestra la cabecera.

Línea 41: var orden = _unidad.Ordenes.CargarPlanchasExplicitamente(1); → llama al método del repositorio.

Línea 42: if (orden is not null) → comprueba si la orden existe.

Línea 44: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");` → muestra el número de planchas.

Línea 45: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 47: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");` → muestra los datos.

Línea 51: `private void DemostrarCargaConFiltro()` → declara el método.

Línea 53: `Console.WriteLine("\n--- Carga Explicit con filtro (planchas activas) ---");` → muestra la cabecera.

Línea 55: `var orden = _unidad Ordenes.CargarPlanchasActivasExplicitamente(1);` → llama al método del repositorio.

Línea 56: `if (orden is not null)` → comprueba si la orden existe.

Línea 58: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}");` → muestra el número de planchas activas.

Línea 59: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 61: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Activa: {plancha.Activa}");` → muestra los datos.

Línea 65: `private void DemostrarCargaMultipleConIsLoaded()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Carga Explicit de múltiples propiedades con IsLoaded ---");` → muestra la cabecera.

Línea 69: `var orden = _unidad Ordenes.CargarPlanchasYDetalleExplicitamente(1);` → llama al método del repositorio.

Línea 70: `if (orden is not null)` → comprueba si la orden existe.

Línea 72: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");` → muestra el número de planchas.

Línea 73: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → comprueba si el detalle existe.

Línea 74: `Console.WriteLine($" Detalle: {detalle}");` → muestra el detalle.

Error común: si el método del repositorio no usa `IsLoaded`, EF Core puede ejecutar una consulta innecesaria si la propiedad ya estaba cargada por un `Include` previo.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CargaExplicitUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes, planchas y detalle:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = false };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `context.OrdenesFabricacion.AddRange(orden1, orden2);` → registra las órdenes.

Línea 10: `context.SaveChanges();` → inserta las órdenes.

Línea 12: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha activa.

Línea 13: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha inactiva.

Línea 14: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha activa.

Línea 15: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 17: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 18: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 20: `context.SaveChanges();` → inserta las planchas y el detalle.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observan las órdenes con detalle, con planchas, con planchas activas y con planchas y detalle. Cada sección ejecuta consultas adicionales para cargar las entidades relacionadas de forma explícita.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CARGA EXPLICIT ===
```

```
--- Carga Explicit de referencia (detalle) ---
```

```
Orden: OF-2024-0001 | Cliente: Constructora del Norte
Detalle: C: 0.45%, Mn: 0.75% | 1550.5°C
```

```
--- Carga Explicit de colección (planchas) ---
```

```
Orden: OF-2024-0001 | Planchas: 2
Plancha 1 | Espesor: 10.5 | Peso: 370.5 kg
Plancha 2 | Espesor: 12.0 | Peso: 280.8 kg
```

```
--- Carga Explicit con filtro (planchas activas) ---
```

```
Orden: OF-2024-0001 | Planchas activas: 1
Plancha 1 | Espesor: 10.5 | Activa: True
```

```
--- Carga Explicit de múltiples propiedades con IsLoaded ---
```

```
Orden: OF-2024-0001 | Planchas: 2
Detalle: C: 0.45%, Mn: 0.75%
```

La primera sección muestra la orden con su detalle. La segunda sección muestra la orden con sus planchas. La tercera sección muestra la orden con sus planchas activas. La cuarta sección muestra la orden con sus planchas y su detalle.

Observaciones: en la primera sección, la carga del detalle ejecuta una consulta adicional. En la segunda sección, la carga de las planchas ejecuta una consulta adicional. En la tercera sección, la carga de las planchas activas ejecuta una consulta adicional con filtro. En la cuarta sección, la carga de las planchas y el detalle ejecuta dos consultas adicionales, pero solo si no están cargadas previamente.

Paso 10: Diagnosticar un error común

Modificar el método `CargarPlanchasYDetalleExplicitamente` para no comprobar `IsLoaded`:

```
csharp
public OrdenFabricacion? CargarPlanchasYDetalleExplicitamente(int ordenId)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden).Collection(o => o.Planchas).Load();
    _context.Entry(orden).Reference(o => o.Detalle).Load();

    return orden;
}
```

Resultado esperado: el código funciona, pero si la colección o la referencia ya estaban cargadas por un `Include` previo, EF Core ejecuta una consulta innecesaria. Esto degrada el rendimiento sin aportar ningún beneficio.

Solución: usar `IsLoaded` para comprobar si la propiedad ya está cargada antes de llamar a `Load`.

```
var entry = _context.Entry(orden);

if (!entry.Collection(o => o.Planchas).IsLoaded)
{
    entry.Collection(o => o.Planchas).Load();
}

if (!entry.Reference(o => o.Detalle).IsLoaded)
{
    entry.Reference(o => o.Detalle).Load();
}
```

Resultado esperado con la solución: el código solo ejecuta consultas adicionales si la propiedad no estaba cargada previamente.

Errores comunes del ejercicio

Error	Causa	Solución
Consulta innecesaria	Se llamó a <code>Load</code> sin comprobar <code>IsLoaded</code>	Comprobar <code>IsLoaded</code> antes de <code>Load</code>
Filtro no aplicado	Se aplicó el filtro fuera del Query	Aplicar el filtro dentro del Query
<code>NullReferenceException</code>	La entidad principal es <code>null</code>	Comprobar antes de llamar a <code>Load</code>
Referencia circular	Se carga una entidad que referencia a la principal	Usar DTOs o evitar cargar la referencia inversa
N+1	Se llama a <code>Load</code> en un bucle	Usar <code>Include</code> en su lugar
Carga Lazy mezclada	Se mezcla carga <code>Explicit</code> con carga <code>Lazy</code>	Elegir una técnica y usarla de forma consistente

Reto resuelto: Carga `Explicit` condicional con filtro y ordenación

Reto: Crear un método en el repositorio que cargue las planchas de una orden de forma explícita, aplicando un filtro por espesor mínimo y una ordenación por peso descendente. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz `IOrdenRepositorio`:

```
csharp
OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo);
```

Línea 1: `OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo);` → declara el método.

Paso 2: Implementar el método en `OrdenRepositorio`:

```
public OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);
    if (orden is null)
    {
        return null;
    }

    _context.Entry(orden)
        .Collection(o => o.Planchas)
        .Query()
        .Where(p => p.Espesor >= espesorMinimo)
        .OrderByDescending(p => p.Peso)
        .Load();

    return orden;
}
```

Línea 1: `public OrdenFabricacion? CargarPlanchasFiltradasYOrdenadasExplicitamente(int ordenId, double espesorMinimo)` → declara el método.

Línea 3: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == ordenId);` → carga la orden.

Línea 4: `if (orden is null)` → comprueba si la orden existe.

Línea 6: `return null;` → devuelve null si no existe.

Línea 9: `_context.Entry(orden)` → obtiene el `EntityEntry`.

Línea 10: `.Collection(o => o.Planchas)` → selecciona la colección de planchas.

Línea 11: `.Query()` → obtiene la consulta de la colección.

Línea 12: `.Where(p => p.Espesor >= espesorMinimo)` → filtra por espesor mínimo.

Línea 13: `.OrderByDescending(p => p.Peso)` → ordena por peso descendente.

Línea 14: `.Load();` → ejecuta la consulta y carga la colección.

Línea 16: `return orden;` → devuelve la orden con las planchas filtradas y ordenadas.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarCargaFiltradaYOrdenada()
{
    Console.WriteLine("\n--- Carga Explicit con filtro y ordenación ---");

    var orden = _unidad.Ordenes.CargarPlanchasFiltradasYOrdenadasExplicitamente(1, 11.0);
    if (orden is not null)
    {
        Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas con espesor >= 11.0:
{orden.Planchas.Count}");
        foreach (var plancha in orden.Planchas)
        {
            Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso:
{plancha.Peso} kg");
        }
    }
}
```

Línea 1: `private void DemostrarCargaFiltradaYOrdenada()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Carga Explicit con filtro y ordenación ---");` → muestra la cabecera.

Línea 5: `var orden = _unidad.Ordenes.CargarPlanchasFiltradasYOrdenadasExplicitamente(1, 11.0);` → llama al método del repositorio con espesor mínimo 11.0.

Línea 6: `if (orden is not null)` → comprueba si la orden existe.

Línea 8: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Planchas con espesor >= 11.0: {orden.Planchas.Count}");` → muestra el número de planchas.

Línea 9: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 11: `Console.WriteLine($" Plancha {plancha.Id} | Espesor: {plancha.Espesor} | Peso: {plancha.Peso} kg");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarCargaFiltradaYOrdenada();
Paso 5: Ejecutar dotnet run y verificar que las planchas se muestran filtradas por espesor y ordenadas por peso descendente.
```

Resultado esperado: las planchas con espesor mayor o igual a 11.0 se muestran ordenadas por peso descendente.

| Analogía final

La carga Explicit en una acería es como pedirle al archivo central que te traiga las carpetas relacionadas en el momento en que las necesitas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Cuando necesitas las planchas, pides al archivo que te las traiga. Cuando necesitas el detalle, pides al archivo que te lo traiga. Cada petición es un viaje al archivo central. La carga Explicit es como un operario que decide cuándo pedir cada carpeta. Si sabe que va a necesitar las planchas, las pide. Si no las necesita, no las pide. IsLoaded es como comprobar si ya tienes la carpeta antes de pedirla de nuevo. Query es como pedir solo las planchas que cumplen una condición. La carga Explicit es más controlada que la carga Lazy pero requiere más código. En una acería con mucho volumen, la carga Eager es la opción más eficiente. En una aplicación donde no se sabe de antemano qué se va a necesitar, la carga Explicit es una buena alternativa. Así funciona la carga Explicit en EF Core: manual, controlada y explícita.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de carga Explicit a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso CargaExplicitUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas activas e inactivas, y detalle.

Ejecutado las demostraciones de carga de referencia, carga de colección, carga con filtro y carga múltiple con IsLoaded.

Diagnosticado el error de no comprobar IsLoaded.

Creado el método CargarPlanchasFiltradasYOrdenadasExplicitamente.

| Conexión con el siguiente punto

En este punto se ha explorado la carga Explicit, incluyendo `Entry().Reference().Load()`, `Entry().Collection().Load()`, `IsLoaded` y `Query()`. Se ha comprobado que la carga Explicit es manual y controlada, y que se puede aplicar filtros y ordenaciones antes de cargar. En el siguiente punto se estudiará la composición de consultas con `IQueryable`, incluyendo la ejecución diferida y las consultas condicionales.

Punto 3.11 - Composición de consultas y ejecución diferida

Práctica

Ejercicio: Añadir métodos de composición de consultas al repositorio de órdenes del proyecto AceriaData. Crear métodos que construyan consultas con filtros, ordenaciones y paginación de forma incremental. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.10 se exploró la carga Explicit, incluyendo Entry().Reference().Load(), Entry().Collection().Load(), IsLoaded y Query(). En este punto se profundiza en la composición de consultas y la ejecución diferida. Esta técnica se usará en el punto 3.12 para las buenas prácticas y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO FiltroOrdenesDto

Crear el archivo src/AceriaData.Application/Dtos/FiltroOrdenesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class FiltroOrdenesDto
{
    public string? Cliente { get; set; }
    public string? Estado { get; set; }
    public DateTime? FechaDesde { get; set; }
    public DateTime? FechaHasta { get; set; }
    public string? OrdenarPor { get; set; }
    public bool Descendente { get; set; }
    public int? Pagina { get; set; }
    public int? TamanoPagina { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class FiltroOrdenesDto → declara el DTO de filtros.

Línea 5: public string? Cliente { get; set; } → filtro por cliente.

Línea 6: public string? Estado { get; set; } → filtro por estado.

Línea 7: public DateTime? FechaDesde { get; set; } → filtro por fecha desde.

Línea 8: public DateTime? FechaHasta { get; set; } → filtro por fecha hasta.

Línea 9: public string? OrdenarPor { get; set; } → criterio de ordenación.

Línea 10: public bool Descendente { get; set; } → indica si la ordenación es descendente.

Línea 11: public int? Pagina { get; set; } → número de página.

Línea 12: public int? TamanoPagina { get; set; } → tamaño de la página.

Error común: si los filtros son de tipo string no anulable, el DTO requiere que se asignen siempre. Se deben declarar como string? para que sean opcionales.

Paso 3: Añadir el método de composición a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro);
    int ContarConFiltros(FiltroOrdenesDto filtro);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConFiltros(FiltroOrdenesDto filtro); → método que obtiene las órdenes con filtros.

Línea 11: int ContarConFiltros(FiltroOrdenesDto filtro); → método que cuenta las órdenes con filtros.

Línea 13: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 14: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 4: Implementar el método de composición en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```
public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.ToList();
}

public int ContarConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.Count();
}

private IQueryable<OrdenFabricacion> ConstruirConsultaConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = _context.OrdenesFabricacion.AsQueryable();

    if (!string.IsNullOrEmpty(filtro.Cliente))
    {
        consulta = consulta.Where(o => o.Cliente == filtro.Cliente);
    }

    if (!string.IsNullOrEmpty(filtro.Estado))
    {
        consulta = consulta.Where(o => o.Estado == filtro.Estado);
    }
}
```

```

    if (filtro.FechaDesde.HasValue)
    {
        consulta = consulta.Where(o => o.FechaCreacion >= filtro.FechaDesde.Value);
    }

    if (filtro.FechaHasta.HasValue)
    {
        consulta = consulta.Where(o => o.FechaCreacion <= filtro.FechaHasta.Value);
    }

    if (!string.IsNullOrEmpty(filtro.OrdenarPor))
    {
        consulta = filtro.OrdenarPor switch
        {
            "Cliente" => filtro.Descendente ? consulta.OrderByDescending(o => o.Cliente) :
            consulta.OrderBy(o => o.Cliente),
            "Estado" => filtro.Descendente ? consulta.OrderByDescending(o => o.Estado) :
            consulta.OrderBy(o => o.Estado),
            "FechaCreacion" => filtro.Descendente ? consulta.OrderByDescending(o => o.FechaCreacion) :
            consulta.OrderBy(o => o.FechaCreacion),
            "NumeroOrden" => filtro.Descendente ? consulta.OrderByDescending(o => o.NumeroOrden) :
            consulta.OrderBy(o => o.NumeroOrden),
            _ => consulta.OrderBy(o => o.NumeroOrden)
        };
    }
    else
    {
        consulta = consulta.OrderBy(o => o.NumeroOrden);
    }

    if (filtro.Pagina.HasValue && filtro.TamanoPagina.HasValue &&
        filtro.Pagina.Value > 0 && filtro.TamanoPagina.Value > 0)
    {
        consulta = consulta
            .Skip((filtro.Pagina.Value - 1) * filtro.TamanoPagina.Value)
            .Take(filtro.TamanoPagina.Value);
    }

    return consulta;
}

```

Línea 1: public List ObtenerConFiltros(FiltroOrdenesDto filtro) → declara el método que obtiene las órdenes con filtros.

Línea 3: var consulta = ConstruirConsultaConFiltros(filtro); → construye la consulta.

Línea 4: return consulta.ToList(); → materializa la consulta.

Línea 7: public int ContarConFiltros(FiltroOrdenesDto filtro) → declara el método que cuenta las órdenes con filtros.

Línea 9: var consulta = ConstruirConsultaConFiltros(filtro); → construye la consulta.

Línea 10: return consulta.Count(); → cuenta los elementos.

Línea 13: private IQueryable ConstruirConsultaConFiltros(FiltroOrdenesDto filtro) → declara el método que construye la consulta.

Línea 15: var consulta = _context.OrdenesFabricacion.AsQueryable(); → inicia la consulta base.

Línea 17: if (!string.IsNullOrEmpty(filtro.Cliente)) → comprueba si el filtro por cliente está presente.

Línea 19: consulta = consulta.Where(o => o.Cliente == filtro.Cliente); → añade el filtro por cliente.

Línea 22: if (!string.IsNullOrEmpty(filtro.Estado)) → comprueba si el filtro por estado está presente.

Línea 24: consulta = consulta.Where(o => o.Estado == filtro.Estado); → añade el filtro por estado.

Línea 27: if (filtro.FechaDesde.HasValue) → comprueba si el filtro por fecha desde está presente.

Línea 29: `consulta = consulta.Where(o => o.FechaCreacion >= filtro.FechaDesde.Value);` → añade el filtro por fecha desde.

Línea 32: `if (filtro.FechaHasta.HasValue)` → comprueba si el filtro por fecha hasta está presente.

Línea 34: `consulta = consulta.Where(o => o.FechaCreacion <= filtro.FechaHasta.Value);` → añade el filtro por fecha hasta.

Línea 37: `if (!string.IsNullOrEmpty(filtro.OrdenarPor))` → comprueba si hay criterio de ordenación.

Línea 39: `consulta = filtro.OrdenarPor switch` → selecciona el criterio de ordenación.

Línea 41: `"Cliente" => filtro.Descendente ? consulta.OrderByDescending(o => o.Cliente) : consulta.OrderBy(o => o.Cliente),` → ordena por cliente.

Línea 42: `"Estado" => filtro.Descendente ? consulta.OrderByDescending(o => o.Estado) : consulta.OrderBy(o => o.Estado),` → ordena por estado.

Línea 43: `"FechaCreacion" => filtro.Descendente ? consulta.OrderByDescending(o => o.FechaCreacion) : consulta.OrderBy(o => o.FechaCreacion),` → ordena por fecha de creación.

Línea 44: `"NumeroOrden" => filtro.Descendente ? consulta.OrderByDescending(o => o.NumeroOrden) : consulta.OrderBy(o => o.NumeroOrden),` → ordena por número de orden.

Línea 45: `_ => consulta.OrderBy(o => o.NumeroOrden)` → ordenación por defecto.

Línea 46: `};` → cierra el switch.

Línea 49: `consulta = consulta.OrderBy(o => o.NumeroOrden);` → ordenación por defecto si no hay criterio.

Línea 52: `if (filtro.Pagina.HasValue && filtro.TamanoPagina.HasValue &&` → comprueba si hay paginación.

Línea 53: `filtro.Pagina.Value > 0 && filtro.TamanoPagina.Value > 0)` → comprueba que los valores son positivos.

Línea 55: `consulta = consulta` → inicia la paginación.

Línea 56: `.Skip((filtro.Pagina.Value - 1) * filtro.TamanoPagina.Value)` → salta las páginas anteriores.

Línea 57: `.Take(filtro.TamanoPagina.Value);` → toma el tamaño de la página.

Línea 60: `return consulta;` → devuelve la consulta construida.

Error común: si se llama a `ToList` dentro del método `ConstruirConsultaConFiltros`, la consulta se materializa antes de tiempo y se pierde la ventaja de la composición. Se debe devolver `IQueryable` y materializar solo al final.

| Paso 5: Crear el caso de uso de composición de consultas

Crear el archivo `src/AceriaData.Application/UseCases/ComposicionConsultasUseCase.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ComposicionConsultasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");
    }
}
```

```

        DemostrarFiltroPorCliente();
        DemostrarFiltroPorEstado();
        DemostrarFiltroPorRangoDeFechas();
        DemostrarFiltrosCombinados();
        DemostrarOrdenacionYPaginacion();
    }

    private void DemostrarFiltroPorCliente()
    {
        Console.WriteLine("\n--- Filtro por cliente ---");

        var filtro = new FiltroOrdenesDto { Cliente = "Constructora del Norte" };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarFiltroPorEstado()
    {
        Console.WriteLine("\n--- Filtro por estado ---");

        var filtro = new FiltroOrdenesDto { Estado = "Pendiente" };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarFiltroPorRangoDeFechas()
    {
        Console.WriteLine("\n--- Filtro por rango de fechas ---");

        var filtro = new FiltroOrdenesDto
        {
            FechaDesde = new DateTime(2024, 1, 1),
            FechaHasta = new DateTime(2024, 6, 30)
        };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarFiltrosCombinados()
    {
        Console.WriteLine("\n--- Filtros combinados ---");

        var filtro = new FiltroOrdenesDto
        {
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaDesde = new DateTime(2024, 1, 1)
        };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }
}

```

```

private void DemostrarOrdenacionYPaginacion()
{
    Console.WriteLine("\n--- Ordenación y paginación ---");

    var filtro = new FiltroOrdenesDto
    {
        OrdenarPor = "FechaCreacion",
        Descendente = true,
        Pagina = 1,
        TamanoPagina = 2
    };
    var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);
    var total = _unidad.Ordenes.ContarConFiltros(filtro);

    Console.WriteLine($"Total de órdenes: {total} | Página 1 con 2 elementos: {ordenes.Count}");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");
    }
}
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ComposicionConsultasUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ==="); → muestra la cabecera.

Línea 19: DemostrarFiltroPorCliente(); → llama al método de filtro por cliente.

Línea 20: DemostrarFiltroPorEstado(); → llama al método de filtro por estado.

Línea 21: DemostrarFiltroPorRangoDeFechas(); → llama al método de filtro por rango de fechas.

Línea 22: DemostrarFiltrosCombinados(); → llama al método de filtros combinados.

Línea 23: DemostrarOrdenacionYPaginacion(); → llama al método de ordenación y paginación.

Línea 26: private void DemostrarFiltroPorCliente() → declara el método.

Línea 28: Console.WriteLine("\n--- Filtro por cliente ---"); → muestra la cabecera.

Línea 30: var filtro = new FiltroOrdenesDto { Cliente = "Constructora del Norte" }; → crea el filtro.

Línea 31: var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro); → llama al método del repositorio.

Línea 33: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 34: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 36: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos.

Línea 39: private void DemostrarFiltroPorEstado() → declara el método.

Línea 41: Console.WriteLine("\n--- Filtro por estado ---"); → muestra la cabecera.

Línea 43: var filtro = new FiltroOrdenesDto { Estado = "Pendiente" }; → crea el filtro.

Línea 44: `var ordenes = _unidad Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 46: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 47: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 49: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 52: `private void DemostrarFiltroPorRangoDeFechas()` → declara el método.

Línea 54: `Console.WriteLine("\n--- Filtro por rango de fechas ---");` → muestra la cabecera.

Línea 56: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 58: `FechaDesde = new DateTime(2024, 1, 1),` → asigna la fecha desde.

Línea 59: `FechaHasta = new DateTime(2024, 6, 30)` → asigna la fecha hasta.

Línea 60: `};` → cierra el filtro.

Línea 61: `var ordenes = _unidad Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 63: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 64: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 66: `Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 69: `private void DemostrarFiltrosCombinados()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Filtros combinados ---");` → muestra la cabecera.

Línea 73: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 75: `Cliente = "Constructora del Norte",` → asigna el cliente.

Línea 76: `Estado = "Pendiente",` → asigna el estado.

Línea 77: `FechaDesde = new DateTime(2024, 1, 1)` → asigna la fecha desde.

Línea 78: `};` → cierra el filtro.

Línea 79: `var ordenes = _unidad Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 81: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 82: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 84: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 87: `private void DemostrarOrdenacionYPaginacion()` → declara el método.

Línea 89: `Console.WriteLine("\n--- Ordenación y paginación ---");` → muestra la cabecera.

Línea 91: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 93: `OrdenarPor = "FechaCreacion",` → asigna el criterio de ordenación.

Línea 94: `Descendente = true,` → indica ordenación descendente.

Línea 95: `Pagina = 1,` → asigna la página.

Línea 96: `TamanoPagina = 2` → asigna el tamaño de la página.

Línea 97: `};` → cierra el filtro.

Línea 98: `var ordenes = _unidad Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 99: `var total = _unidad Ordenes.ContarConFiltros(filtro);` → cuenta las órdenes con el filtro.

Línea 101: `Console.WriteLine($"Total de órdenes: {total} | Página 1 con 2 elementos: {ordenes.Count}");` → muestra el total y el número de órdenes.

Línea 102: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 104: `Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Error común: si el método `ConstruirConsultaConFiltros` materializa la consulta con `ToList`, la composición se pierde. Se debe devolver `IQueryable` y materializar solo al final.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ComposicionConsultasUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 7: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes de prueba:


```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada filtro. Se observan los filtros por cliente, por estado, por rango de fechas, combinados y con ordenación y paginación.

| Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

=== COMPOSICIÓN DE CONSULTAS ===

--- Filtro por cliente ---

Órdenes: 3

OF-2024-0001	Constructora del Norte	Pendiente
OF-2024-0003	Constructora del Norte	EnProceso
OF-2024-0005	Constructora del Norte	Pendiente

--- Filtro por estado ---

Órdenes: 4

OF-2024-0001	Constructora del Norte	Pendiente
OF-2024-0002	Constructora del Sur	Pendiente
OF-2024-0004	Constructora del Este	Pendiente
OF-2024-0005	Constructora del Norte	Pendiente

--- Filtro por rango de fechas ---

Órdenes: 5

OF-2024-0001	15/01/2024
OF-2024-0002	20/02/2024
OF-2024-0003	10/03/2024
OF-2024-0004	05/04/2024
OF-2024-0005	12/05/2024

--- Filtros combinados ---

Órdenes: 2

OF-2024-0001	Constructora del Norte	Pendiente	15/01/2024
OF-2024-0005	Constructora del Norte	Pendiente	12/05/2024

--- Ordenación y paginación ---

Total de órdenes: 6 | Página 1 con 2 elementos: 2

OF-2024-0006	18/06/2024
OF-2024-0005	12/05/2024

La primera sección muestra el filtro por cliente. La segunda sección muestra el filtro por estado. La tercera sección muestra el filtro por rango de fechas. La cuarta sección muestra los filtros combinados. La quinta sección muestra la ordenación y paginación.

Observaciones: cada consulta se ha construido de forma incremental con los filtros aplicados. La consulta de ordenación y paginación incluye Skip y Take, que se traducen a OFFSET y FETCH en SQL Server.

Paso 11: Diagnosticar un error común

Modificar el método ObtenerConFiltros para materializar la consulta antes de aplicar todos los filtros:

```
csharp
public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = _context.OrdenesFabricacion.ToList();

    if (!string.IsNullOrEmpty(filtro.Cliente))
    {
        consulta = consulta.Where(o => o.Cliente == filtro.Cliente).ToList();
    }

    if (!string.IsNullOrEmpty(filtro.Estado))
    {
        consulta = consulta.Where(o => o.Estado == filtro.Estado).ToList();
    }

    return consulta;
}
```

Resultado esperado: el código carga todas las órdenes en memoria y después filtra. Si la tabla tiene un millón de filas, se transfieren todas a memoria antes de filtrar. Esto degrada el rendimiento.

Solución: construir la consulta con IQueryable y materializar solo al final.

```
public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.ToList();
}
```

Resultado esperado con la solución: la consulta se construye de forma incremental y se materializa solo al final. Los filtros se aplican en SQL.

Errores comunes del ejercicio

Error	Causa	Solución
Materialización prematura	Se llamó a ToList antes de aplicar todos los filtros	
Materializar solo al final		
Filtros en memoria	Se materializó la consulta antes de filtrar	Construir la consulta con IQueryable
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
ThenBy sin OrderBy	Se llamó a ThenBy sin OrderBy previo	Llamar a OrderBy primero
Skip sin OrderBy	Se aplicó Skip sin ordenar	Aplicar OrderBy antes de Skip
Paginación incorrecta	Se calculó mal el Skip	Usar (pagina - 1) * tamañoPagina

Reto resuelto: Consulta compuesta con filtros dinámicos

Reto: Crear un método en el repositorio que construya una consulta con filtros dinámicos basados en una lista de predicados. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
csharp
```

```
List<OrdenFabricacion>
```

```
ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados);
```

Línea 1: List ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados); → declara el método que acepta una lista de predicados.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion>
ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados)
{
    var consulta = _context.OrdenesFabricacion.AsQueryable();

    foreach (var predicado in predicados)
    {
        consulta = consulta.Where(predicado);
    }

    return consulta.OrderBy(o => o.NumeroOrden).ToList();
}
```

Línea 1: public List

ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados) → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion.AsQueryable(); → inicia la consulta base.

Línea 5: foreach (var predicado in predicados) → itera sobre los predicados.

Línea 7: consulta = consulta.Where(predicado); → aplica cada predicado.

Línea 10: return consulta.OrderBy(o => o.NumeroOrden).ToList(); → ordena y materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPredicadosDinamicos()
{
    Console.WriteLine("\n--- Predicados dinámicos ---");

    var predicados = new
List<System.Linq.Expressions.Expression<Func<AceriaData.Domain.Entities.OrdenFabricacion, bool>>>
    {
        o => o.Estado == "Pendiente",
        o => o.Cliente == "Constructora del Norte"
    };

    var ordenes = _unidad.Ordenes.ObtenerConPredicados(predicados);
    Console.WriteLine($"Órdenes: {ordenes.Count}");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
    }
}
```

Línea 1: private void DemostrarPredicadosDinamicos() → declara el método.

Línea 3: Console.WriteLine("\n--- Predicados dinámicos ---"); → muestra la cabecera.

Línea 5: var predicados = new List<...> → crea la lista de predicados.

Línea 7: o => o.Estado == "Pendiente", → primer predicado.

Línea 8: o => o.Cliente == "Constructora del Norte" → segundo predicado.

Línea 9: }; → cierra la lista.

Línea 11: var ordenes = _unidad.Ordenes.ObtenerConPredicados(predicados); → llama al método del repositorio.

Línea 12: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 13: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 15: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPredicadosDinamicos();
```

Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran filtradas por los predicados dinámicos.

Resultado esperado: las órdenes pendientes de la "Constructora del Norte" se muestran correctamente.

Analogía final

La composición de consultas en una acería es como construir una orden de búsqueda al archivo central paso a paso. En lugar de pedir todos los documentos y después filtrarlos, el jefe de planta va añadiendo condiciones a la orden: primero el cliente, después el estado, después el rango de fechas. Cada condición se añade a la orden sin enviarla todavía. Solo cuando la orden está completa, el jefe la envía al archivo central. El archivo recibe la orden con todas las condiciones y devuelve solo los documentos que cumplen todas ellas. La ejecución diferida es como preparar la orden sin enviarla: se puede modificar hasta el último momento. La materialización prematura es como enviar la orden antes de terminarla: el archivo devuelve documentos que después hay que filtrar a mano. La composición de consultas permite construir consultas flexibles y eficientes, adaptadas a las necesidades de cada momento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado el DTO FiltroOrdenesDto.

Añadido el método ObtenerConFiltros a la interfaz IOrdenRepositorio.

Implementado el método ConstruirConsultaConFiltros en OrdenRepositorio.

Creado el caso de uso ComposicionConsultasUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de filtro por cliente, estado, rango de fechas, filtros combinados y ordenación con paginación.

Diagnosticado el error de materialización prematura.

Creado el método ObtenerConPredicados.

| Conexión con el siguiente punto

En este punto se ha profundizado en la composición de consultas y la ejecución diferida, incluyendo filtros condicionales, ordenaciones dinámicas, paginación y predicados dinámicos. Se ha comprobado que la composición permite construir consultas flexibles sin ejecutarlas hasta el final. En el siguiente punto se estudiarán las buenas prácticas en el acceso a datos, incluyendo la elección entre carga Eager, Lazy y Explicit, el uso de proyecciones y la composición de consultas.

Punto 3.12 - Buenas prácticas en el acceso a datos y composición de consultas

Práctica

Ejercicio: Aplicar las buenas prácticas de acceso a datos a todos los métodos del repositorio de órdenes del proyecto AceriaData. Refactorizar los métodos existentes para usar proyecciones, AsNoTracking, AsSplitQuery, Any y FirstOrDefault. Crear un caso de uso que ejecute las consultas refactorizadas y analice el SQL generado.

Contexto del proyecto: En el punto 3.11 se profundizó en la composición de consultas y la ejecución diferida. En este punto se consolidan las buenas prácticas de acceso a datos, cerrando el Módulo 3. Estas buenas prácticas se usarán en el Módulo 4 para las optimizaciones de rendimiento y en el Módulo 5 para la persistencia empresarial.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Refactorizar los métodos del repositorio con buenas prácticas

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs para aplicar las buenas prácticas a los métodos existentes:

```
/// <summary>
/// Obtiene los resúmenes de las órdenes pendientes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

/// <summary>
/// Cuenta las órdenes pendientes.
/// Usa Any en lugar de Count cuando solo se quiere saber si hay elementos.
/// </summary>
public bool ExisteAlgunaOrdenPendiente()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Any(o => o.Estado == "Pendiente");
}

/// <summary>
/// Obtiene las órdenes con planchas y detalle usando AsSplitQuery.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// </summary>
```

```

public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}

```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene los resúmenes de las órdenes pendientes. → descripción del método.

Línea 3: /// Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 4: /// Usa AsNoTracking porque es una consulta de solo lectura. → explicación del uso de AsNoTracking.

Línea 5: ///

→ cierre del comentario XML.

Línea 6: public List ObtenerResumenesPendientes() → declara el método.

Línea 8: return _context.OrdenesFabricacion → inicia la consulta.

Línea 9: .AsNoTracking() → aplica AsNoTracking.

Línea 10: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 11: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 12: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 13: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 14: Cliente = o.Cliente, → asigna el cliente.

Línea 15: Estado = o.Estado, → asigna el estado.

Línea 16: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 17: }) → cierra la proyección.

Línea 18: .ToList(); → materializa la consulta.

Línea 21: ///

→ inicio del comentario XML.

Línea 22: /// Cuenta las órdenes pendientes. → descripción del método.

Línea 23: /// Usa Any en lugar de Count cuando solo se quiere saber si hay elementos. → explicación del uso de Any.

Línea 24: ///

→ cierre del comentario XML.

Línea 25: public bool ExisteAlgunaOrdenPendiente() → declara el método.

Línea 27: return _context.OrdenesFabricacion → inicia la consulta.

Línea 28: .AsNoTracking() → aplica AsNoTracking.

Línea 29: .Any(o => o.Estado == "Pendiente"); → comprueba si hay órdenes pendientes.

Línea 32: ///

→ inicio del comentario XML.

Línea 33: /// Obtiene las órdenes con planchas y detalle usando AsSplitQuery. → descripción del método.

Línea 34: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 35: ///

→ cierre del comentario XML.

Línea 36: public List ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano() → declara el método.

Línea 38: return _context.OrdenesFabricacion → inicia la consulta.

Línea 39: .AsNoTracking() → aplica AsNoTracking.

Línea 40: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 41: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 42: .AsSplitQuery() → divide la consulta en varias.

Línea 43: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 44: .Select(o => new OrdenConPlanchasYDetalleDto → proyecta al DTO.

Línea 45: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 46: Cliente = o.Cliente, → asigna el cliente.

Línea 47: Estado = o.Estado, → asigna el estado.

Línea 48: Planchas = o.Planchas.Select(p => new PlanchaDto → proyecta las planchas.

Línea 49: Id = p.Id, → asigna el Id.

Línea 50: Espesor = p.Espesor, → asigna el espesor.

Línea 51: Ancho = p.Ancho, → asigna el ancho.

Línea 52: Largo = p.Largo, → asigna el largo.

Línea 53: Peso = p.Peso, → asigna el peso.

Línea 54: Activa = p.Activa → asigna el estado activo.

Línea 55: }).ToList(), → materializa la colección.

Línea 56: Detalle = o.Detalle == null ? null : new DetalleDto → comprueba si el detalle existe.

Línea 57: `ComposicionQuimica = o.Detalle.ComposicionQuimica`, → asigna la composición química.

Línea 58: `TemperaturaColada = o.Detalle.TemperaturaColada`, → asigna la temperatura.

Línea 59: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 60: `}` → cierra la proyección del detalle.

Línea 61: `})` → cierra la proyección.

Línea 62: `.ToList();` → materializa la consulta.

Error común: si se aplica `AsNoTracking` a una consulta que devuelve entidades completas y después se modifican, los cambios no se guardan. Se debe usar `AsNoTracking` solo cuando no se van a modificar las entidades.

| Paso 3: Crear el caso de uso de buenas prácticas

Crear el archivo `src/AceriaData.Application/UseCases/BuenasPracticasUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class BuenasPracticasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");

        DemostrarProyeccionConAsNoTracking();
        DemostrarAnyEnLugarDeCount();
        DemostrarAsSplitQuerySinProductoCartesiano();
        DemostrarFirstOrDefault();
        DemostrarDocumentacionDeDecisiones();
    }

    private void DemostrarProyeccionConAsNoTracking()
    {
        Console.WriteLine("\n--- Proyección con AsNoTracking ---");

        var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientes();
        Console.WriteLine($"Resúmenes pendientes: {resúmenes.Count}");
        foreach (var resumen in resúmenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarAnyEnLugarDeCount()
    {
        Console.WriteLine("\n--- Any en lugar de Count ---");

        var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();
        Console.WriteLine($"Existe alguna orden pendiente: {existe}");
    }

    private void DemostrarAsSplitQuerySinProductoCartesiano()
    {
        Console.WriteLine("\n--- AsSplitQuery sin producto cartesiano ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();
        foreach (var orden in ordenes)
```

```

        {
            var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
            Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle:
{detalle}");
        }
    }

    private void DemostrarFirstOrDefault()
    {
        Console.WriteLine("\n--- FirstOrDefault en lugar de First ---");

        var orden = _unidad.Ordenes.ObtenerPorNumeroOrden("OF-2024-9999");
        var resultado = orden is null ? "No encontrada" : orden.NumeroOrden;
        Console.WriteLine($"Orden OF-2024-9999: {resultado}");
    }

    private void DemostrarDocumentacionDeDecisiones()
    {
        Console.WriteLine("\n--- Documentación de decisiones ---");
        Console.WriteLine("Los métodos del repositorio incluyen comentarios XML que documentan las
decisiones de acceso a datos.");
        Console.WriteLine("Consultar el código fuente de OrdenRepositorio para más detalles.");
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class BuenasPracticasUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ==="); → muestra la cabecera.

Línea 18: DemostrarProyeccionConAsNoTracking(); → llama al método de proyección con AsNoTracking.

Línea 19: DemostrarAnyEnLugarDeCount(); → llama al método de Any en lugar de Count.

Línea 20: DemostrarAsSplitQuerySinProductoCartesiano(); → llama al método de AsSplitQuery.

Línea 21: DemostrarFirstOrDefault(); → llama al método de FirstOrDefault.

Línea 22: DemostrarDocumentacionDeDecisiones(); → llama al método de documentación.

Línea 25: private void DemostrarProyeccionConAsNoTracking() → declara el método.

Línea 27: Console.WriteLine("\n--- Proyección con AsNoTracking ---"); → muestra la cabecera.

Línea 29: var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientes(); → llama al método del repositorio.

Línea 30: Console.WriteLine(\$"Resúmenes pendientes: {resúmenes.Count}"); → muestra el número de resúmenes.

Línea 31: foreach (var resumen in resúmenes) → itera sobre los resúmenes.

Línea 33: Console.WriteLine(\$" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 36: private void DemostrarAnyEnLugarDeCount() → declara el método.

Línea 38: Console.WriteLine("\n--- Any en lugar de Count ---"); → muestra la cabecera.

Línea 40: `var existe = _unidad Ordenes.ExisteAlgunaOrdenPendiente();` → llama al método del repositorio.

Línea 41: `Console.WriteLine($"Existe alguna orden pendiente: {existe}");` → muestra el resultado.

Línea 44: `private void DemostrarAsSplitQuerySinProductoCartesiano()` → declara el método.

Línea 46: `Console.WriteLine("\n--- AsSplitQuery sin producto cartesiano ---");` → muestra la cabecera.

Línea 48: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();` → llama al método del repositorio.

Línea 49: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 51: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → comprueba si el detalle existe.

Línea 52: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle: {detalle}");` → muestra los datos.

Línea 56: `private void DemostrarFirstOrDefault()` → declara el método.

Línea 58: `Console.WriteLine("\n--- FirstOrDefault en lugar de First ---");` → muestra la cabecera.

Línea 60: `var orden = _unidad Ordenes.ObtenerPorNumeroOrden("OF-2024-9999");` → busca una orden que no existe.

Línea 61: `var resultado = orden is null ? "No encontrada" : orden.NumeroOrden;` → comprueba si la orden existe.

Línea 62: `Console.WriteLine($"Orden OF-2024-9999: {resultado}");` → muestra el resultado.

Línea 65: `private void DemostrarDocumentacionDeDecisiones()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Documentación de decisiones ---");` → muestra la cabecera.

Línea 68: `Console.WriteLine("Los métodos del repositorio incluyen comentarios XML que documentan las decisiones de acceso a datos.");` → describe la documentación.

Línea 69: `Console.WriteLine("Consultar el código fuente de OrdenRepositorio para más detalles.");` → indica dónde consultar.

Error común: si no se documentan las decisiones, otros desarrolladores pueden modificarlas sin entender las consecuencias. Se deben documentar las decisiones de acceso a datos.

| Paso 4: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<BuenasPracticasUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 5: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 6: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes, planchas y detalle:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 16: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 18: `var detalle1 = new DetalleOrden { ... }; → crea el detalle.`

Línea 19: `context.DetallesOrden.Add(detalle1); → registra el detalle.`

Línea 21: `context.SaveChanges(); → inserta las planchas y el detalle.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observan las proyecciones con `AsNoTracking`, el uso de `Any`, el uso de `AsSplitQuery` y el uso de `FirstOrDefault`.

Paso 8: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===

--- Proyección con AsNoTracking ---
Resúmenes pendientes: 2
  OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
  OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024

--- Any en lugar de Count ---
Existe alguna orden pendiente: True

--- AsSplitQuery sin producto cartesiano ---
  OF-2024-0001 | Planchas: 2 | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002 | Planchas: 1 | Detalle: Sin detalle
  OF-2024-0003 | Planchas: 0 | Detalle: Sin detalle

--- FirstOrDefault en lugar de First ---
Orden OF-2024-9999: No encontrada

--- Documentación de decisiones ---
Los métodos del repositorio incluyen comentarios XML que documentan las decisiones de acceso a datos.
Consultar el código fuente de OrdenRepositorio para más detalles.
La primera sección muestra los resúmenes pendientes con proyección y AsNoTracking. La segunda sección
muestra el uso de Any. La tercera sección muestra el uso de AsSplitQuery sin producto cartesiano. La
cuarta sección muestra el uso de FirstOrDefault. La quinta sección muestra la documentación de
decisiones.

Observaciones: la proyección con AsNoTracking reduce el volumen de datos transferidos. El uso de Any es
más eficiente que Count. El uso de AsSplitQuery evita el producto cartesiano. El uso de FirstOrDefault
devuelve null en lugar de lanzar una excepción.

Paso 9: Diagnosticar un error común
Modificar el método ObtenerResumenesPendientes para no usar AsNoTracking:

csharp
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

```

    })
    .ToList();
}

```

Resultado esperado: el código funciona, pero las entidades se registran en el Change Tracker aunque no se vayan a modificar. Esto consume memoria innecesariamente.

Solución: añadir AsNoTracking antes del Where.

```

public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Resultado esperado con la solución: las entidades no se registran en el Change Tracker y se libera memoria.

Errores comunes del ejercicio

Error	Causa	Solución
AsNoTracking en consulta de escritura		Se aplicó AsNoTracking a una consulta que modifica datos
Count() > 0 en lugar de Any	Usar AsNoTracking solo en consultas de solo lectura	Se usó Count para saber si hay elementos
First en lugar de FirstOrDefault		Usar Any
FirstOrDefault		Se usó First cuando puede no haber resultados
Producto cartesiano		Usar
Materialización prematura	Se incluyeron varias colecciones sin AsSplitQuery	Usar AsSplitQuery
Materializar solo al final	Se llamó a ToList antes de aplicar todos los filtros	
N+1	Se accede a propiedades de navegación en un bucle sin Include	Usar Include
Falta de documentación	No se documentaron las decisiones	Añadir comentarios XML

Reto resuelto: Refactorizar un método con todas las buenas prácticas

Reto: Refactorizar el método ObtenerPorEstadoOrdenadosPorFecha del repositorio para aplicar todas las buenas prácticas: proyección, AsNoTracking, OrderBy, ToList al final y documentación XML. Añadir el método refactorizado, el DTO correspondiente y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```

csharp
List<OrdenResumenDto> ObtenerResumenesPorEstadoOrdenados(string estado);

```

Línea 1: List ObtenerResumenesPorEstadoOrdenados(string estado); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```

/// <summary>
/// Obtiene los resúmenes de las órdenes de un estado concreto, ordenados por fecha.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// Materializa con ToList al final para ejecutar una sola consulta.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPorEstadoOrdenados(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene los resúmenes de las órdenes de un estado concreto, ordenados por fecha. → descripción del método.

Línea 3: /// Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 4: /// Usa AsNoTracking porque es una consulta de solo lectura. → explicación del uso de AsNoTracking.

Línea 5: /// Materializa con ToList al final para ejecutar una sola consulta. → explicación de la materialización al final.

Línea 6: ///

→ cierre del comentario XML.

Línea 7: public List ObtenerResumenesPorEstadoOrdenados(string estado) → declara el método.

Línea 9: return _context.OrdenesFabricacion → inicia la consulta.

Línea 10: .AsNoTracking() → aplica AsNoTracking.

Línea 11: .Where(o => o.Estado == estado) → filtra por estado.

Línea 12: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 13: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 14: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 15: Cliente = o.Cliente, → asigna el cliente.

Línea 16: Estado = o.Estado, → asigna el estado.

Línea 17: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 18: }) → cierra la proyección.

Línea 19: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenesPorEstadoOrdenados()
{
    Console.WriteLine("\n--- Resúmenes por estado ordenados ---");

    var resumenes = _unidad Ordenes.ObtenerResumenesPorEstadoOrdenados("Pendiente");
    Console.WriteLine($"Resúmenes pendientes: {resumenes.Count}");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: `private void DemostrarResumenesPorEstadoOrdenados()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Resúmenes por estado ordenados ---");` → muestra la cabecera.

Línea 5: `var resumenes = _unidad Ordenes.ObtenerResumenesPorEstadoOrdenados("Pendiente");` → llama al método del repositorio.

Línea 6: `Console.WriteLine($"Resúmenes pendientes: {resumenes.Count}");` → muestra el número de resúmenes.

Línea 7: `foreach (var resumen in resumenes)` → itera sobre los resúmenes.

Línea 9: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarResumenesPorEstadoOrdenados();
```

Paso 5: Ejecutar `dotnet run` y verificar que los resúmenes se muestran ordenados por fecha.

Resultado esperado: los resúmenes de las órdenes pendientes se muestran ordenados por fecha.

Analogía final

Las buenas prácticas de acceso a datos en una acería son como las normas de seguridad y eficiencia de la planta. No se trata solo de producir acero, sino de producirlo de forma segura, eficiente y sostenible. Usar proyecciones es como pedir solo los datos que se necesitan: no se pide la carpeta completa si solo se necesita el número de orden. Usar `AsNoTracking` es como no registrar cada plancha en el libro de producción si solo se va a consultar. Usar `Any` es como comprobar si hay planchas en el almacén sin contarlas todas. Usar `FirstOrDefault` es como buscar una plancha por su número y devolver null si no existe, en lugar de lanzar una alarma. Usar `AsSplitQuery` es como dividir una orden de búsqueda grande en varias más pequeñas para evitar un envío masivo. Documentar las decisiones es como escribir las normas en el manual de la planta para que todos las conozcan. Así funcionan las buenas prácticas en EF Core: producen código más eficiente, más mantenible y más predecible.

Resultado esperado

Al final del ejercicio, deberías haber:

Refactorizado los métodos del repositorio con buenas prácticas.

Añadido `AsNoTracking` a las consultas de solo lectura.

Usado `Any` en lugar de `Count`.

Usado `AsSplitQuery` cuando se incluyen varias colecciones.

Usado `FirstOrDefault` en lugar de `First`.

Documentado las decisiones de acceso a datos con comentarios XML.

Creado el caso de uso BuenasPracticasUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas y detalle.

Ejecutado las demostraciones de proyección con AsNoTracking, Any, AsSplitQuery y FirstOrDefault.

Diagnosticado el error de no usar AsNoTracking.

Creado el método ObtenerResumenesPorEstadoOrdenados.

Resumen del estado del proyecto AceriaData al final del Módulo 3 Al final del Módulo 3, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infrastructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IOrdenRepositorio y IUnidadDeTrabajo, los casos de uso CrearOrdenUseCase, ListarOrdenesUseCase, ConsultasLinqUseCase, ConsultasBasicasUseCase, ProyeccionesUseCase, ProyeccionesDtoUseCase, AgregacionesUseCase, AgrupacionesUseCase, JoinsUseCase, CargaEagerUseCase, CargaLazyUseCase, CargaExplicitUseCase, ComposicionConsultasUseCase y BuenasPracticasUseCase, y los DTOs.

La infraestructura con el AceriaDbContext, las configuraciones de Fluent API, las implementaciones de repositorios y la unidad de trabajo.

La consola con el método Main y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El Soft Delete implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.
